Chương 8

Mô hình sinh

Một trong những tiến bộ quan trọng nhất của NLP trong những năm gần đây là sự phát triển của các mô hình ngôn ngữ lớn (*large language models, LLMs*). Bước đột phá này đã giúp tạo ra những hệ thống có thể hiểu và sinh ngôn ngữ tự nhiên ở mức gần với con người. Các hệ thống này thậm chí còn thể hiện được năng lực lập luận (*reasoning*) — một thách thức tồn tại từ lâu trong AI. Với những thành tựu đó, NLP đã đạt được tiến bộ đáng kể và bước vào một kỷ nguyên nghiên cứu mới, trong đó những bài toán ngày càng phức tạp đang được giải quyết, chẳng hạn như xây dựng các tác tử hội thoại (*conversational agents*) có thể tương tác trôi chảy với con người.

Khái niệm mô hình hóa ngôn ngữ (*language modeling*) bắt nguồn từ những thí nghiệm ban đầu của Shannon [1951]. Trong công trình của mình, Shannon đề xuất một mô hình ngôn ngữ để ước lượng mức độ có thể đoán trước của văn bản tiếng Anh — tức là chữ cái tiếp theo của một văn bản có thể được dự đoán tốt đến đâu khi đã biết N chữ cái đứng trước nó. Dù các thí nghiệm của Shannon mới chỉ là bước sơ khởi, mục tiêu và phương pháp cơ bản của mô hình hóa ngôn ngữ hầu như không thay đổi qua nhiều thập kỷ. Trong nhiều năm, đặc biệt là trước năm 2010, cách tiếp cận thống trị trong mô hình hóa ngôn ngữ là cách tiếp cận *n*-gram [Jurafsky và Martin, 2008]. Trong mô hình hóa ngôn ngữ *n*-gram, ta ước lượng xác suất của một từ khi biết $n-1$ từ đứng trước nó, và nhờ vậy xác suất của một chuỗi có thể được xấp xỉ bằng tích các xác suất *n*-gram. Các xác suất này thường được ước lượng từ tần suất tương đối (đã được làm trơn (*smoothed*)) của các *n*-gram trong văn bản. Dù đơn giản, cách tiếp

cận này đã được dùng rộng rãi trong NLP. Chẳng hạn, thành công của các hệ thống nhận dạng tiếng nói (*speech recognition*) và dịch máy (*machine translation*) thống kê hiện đại phần lớn dựa vào việc sử dụng mô hình ngôn ngữ *n*-gram [Jelinek, 1998; Koehn, 2010].

Chú giải của người dịch: Mô hình *n*-gram qua một ví dụ

Một *n*-gram là một cụm gồm n từ liên tiếp. Với mô hình 2-gram (bigram), ta chỉ nhìn *một* từ đứng trước để đoán từ tiếp theo. Giả sử trong kho văn bản cụm "ăn cơm" xuất hiện 30 lần, còn từ "ăn" xuất hiện 100 lần; khi đó ước lượng $\Pr(\text{cơm} \mid \text{ăn}) \approx 30/100 = 0.3$. "Làm trơn" là kỹ thuật chia bớt một chút xác suất cho những cụm chưa từng gặp, để chúng không bị gán xác suất bằng 0 (vì một cụm chưa thấy trong dữ liệu chưa chắc là không thể xảy ra).

Việc áp dụng mạng nơ-ron vào mô hình hóa ngôn ngữ từ lâu đã là một hướng nghiên cứu sôi động, nhưng bước đột phá lớn chỉ đến cùng với sự tiến bộ của các kỹ thuật học sâu (*deep learning*). Một nghiên cứu được trích dẫn rộng rãi là công trình của Bengio và cộng sự [2003a], trong đó xác suất *n*-gram được mô hình hóa bằng một mạng truyền thẳng (*feed-forward network*) được huấn luyện đầu-cuối. Một thành quả quan trọng của mô hình này là các biểu diễn phân tán (*distributed representations*) của từ, được gọi là nhúng từ (*word embeddings*). Thay vì biểu diễn từ như các biến rời rạc, nhúng từ ánh xạ các từ thành những vectơ số thực có số chiều thấp, nhờ đó có thể nắm bắt các quan hệ ngữ nghĩa giữa các từ và giữa các *n*-gram từ trong một không gian biểu diễn liên tục. Kết quả là các mô hình ngôn ngữ không còn bị đè nặng bởi lời nguyền số chiều (*curse of dimensionality*), mà có thể biểu diễn một số lượng *n*-gram tăng theo hàm mũ bằng một mô hình nơ-ron gọn nhẹ.

Chú giải của người dịch: Lời nguyền số chiều

Nếu bộ từ vựng có $|V| = 10\,000$ từ thì số 3-gram có thể có là $10\,000^3 = 10^{12}$ — quá nhiều để đếm và lưu trữ, và tuyệt đại đa số chúng chẳng bao giờ xuất hiện trong dữ liệu. Đó là "lời nguyền số chiều": số trường hợp tăng theo hàm mũ khi n tăng. Nhúng từ giải quyết chuyện này bằng cách cho các từ gần nghĩa (như "mèo" và "chó") có vectơ gần nhau, nên những gì mô hình học được về "con mèo chạy" có thể dùng lại cho "con chó chạy" dù cụm sau chưa từng gặp.

Ý tưởng học biểu diễn từ thông qua mô hình ngôn ngữ nơ-ron đã truyền cảm hứng cho các nghiên cứu tiếp theo về học biểu diễn (*representation learning*) trong NLP. Tuy nhiên, cách tiếp cận này không thu hút được nhiều sự quan tâm của cộng đồng NLP trong vài năm đầu sau khi được đề xuất. Dù vậy, vào khoảng năm 2012, đã có những tiến bộ đáng kể trong việc học nhúng từ từ văn bản quy mô lớn thông qua các nhiệm vụ dự đoán từ đơn giản. Nhiều phương pháp, trong đó có Word2Vec, đã được đề xuất để học hiệu quả các vectơ nhúng như vậy, và chúng sau đó được ứng dụng thành công trong nhiều hệ thống NLP [Mikolov và cộng sự, 2013a;c]. Nhờ những tiến bộ này, các nhà nghiên cứu bắt đầu khám phá việc học biểu diễn chuỗi bằng những mô hình ngôn ngữ mạnh hơn, chẳng hạn các mô hình dựa trên LSTM [Sutskever và cộng sự, 2014; Peters và cộng sự, 2018]. Tiếp đó, các tiến bộ và sự quan tâm đến biểu diễn chuỗi tăng lên nhanh chóng sau khi kiến trúc Transformer được đề xuất. Cùng với sự trỗi dậy của kiến trúc Transformer, khái niệm mô hình hóa ngôn ngữ được tổng quát hóa để bao hàm cả những mô hình học cách dự đoán từ theo những mô thức khác nhau. Nhiều mô hình mạnh dựa trên Transformer đã được tiền huấn luyện (*pre-trained*) bằng các nhiệm vụ dự đoán từ này và được áp dụng thành công cho nhiều nhiệm vụ hạ nguồn (*downstream tasks*) [Devlin và cộng sự, 2019].

Quả thật, việc huấn luyện mô hình ngôn ngữ trên dữ liệu quy mô lớn đã đưa nghiên cứu NLP đến một thời kỳ đầy hứng khởi. Trong khi mô hình hóa ngôn ngữ từ lâu được xem là một kỹ thuật nền tảng, không có liên hệ trực tiếp với những mục tiêu rộng lớn hơn của trí tuệ nhân tạo, thì nó lại dẫn đến sự xuất hiện của những hệ thống thông minh có thể thu nhận tri thức tổng quát thông qua việc dự đoán từ lặp đi lặp lại. Các nghiên cứu gần đây cho thấy một LLM duy nhất, được huấn luyện tốt, có thể thực hiện rất nhiều loại nhiệm vụ và tổng quát hóa sang nhiệm vụ mới chỉ với mức điều chỉnh tối thiểu [Bubeck và cộng sự, 2023]. Đây là một bước tiến tới những dạng trí tuệ nhân tạo cao cấp hơn, và nó thúc đẩy việc khám phá sâu hơn nhằm phát triển những mô hình ngôn ngữ mạnh hơn như một lớp mô hình nền tảng (*foundation models*).

Trong chương này, chúng ta bàn về các khái niệm cơ bản của LLM sinh. Để đơn giản, ta dùng từ LLM để chỉ các mô hình sinh (*generative models*) như GPT, mặc dù theo nghĩa rộng thuật ngữ này cũng có thể bao gồm các mô hình không sinh như BERT. Ta bắt đầu bằng phần giới thiệu

chung về LLM, bao gồm các bước then chốt để xây dựng những mô hình như vậy. Sau đó, ta bàn về hai thách thức then chốt khi *mở rộng quy mô* LLM: LLM được huấn luyện ở quy mô lớn như thế nào, và chúng có thể được cải tiến ra sao để xử lý các đầu vào có ngữ cảnh dài. Cuối cùng, chương khép lại bằng một phần tóm tắt ngắn.

8.1 Giới thiệu ngắn gọn về LLM

Trong mục này, chúng tôi giới thiệu các khái niệm cơ bản về LLM cần thiết cho phần còn lại của chương này và các chương sau. Chúng tôi sẽ dùng các thuật ngữ *từ* và *token* thay thế cho nhau. Cả hai đều chỉ các đơn vị cơ bản dùng trong mô hình hóa ngôn ngữ, dù nghĩa gốc của chúng hơi khác nhau.

Trước khi trình bày chi tiết kỹ thuật, ta hãy xem các mô hình ngôn ngữ hoạt động như thế nào. Mục tiêu cốt lõi của mô hình hóa ngôn ngữ là ước lượng xác suất của một chuỗi token. Gọi $\{x_0, x_1, \cdots, x_m\}$ là một chuỗi token, trong đó x_0 là ký hiệu bắt đầu $\langle s \rangle$ (hoặc $\langle \text{SOS} \rangle$)[1]. Xác suất của chuỗi này có thể được phân tích bằng quy tắc chuỗi (*chain rule*)

$$\begin{aligned}\Pr(x_0, \cdots, x_m) &= \Pr(x_0) \cdot \Pr(x_1|x_0) \cdot \Pr(x_2|x_0, x_1) \cdots \\ &\quad \Pr(x_m|x_0, \cdots, x_{m-1}) \\ &= \prod_{i=0}^{m} \Pr(x_i|x_0, \cdots, x_{i-1}),\end{aligned} \tag{8.1}$$

hoặc dưới dạng logarit

$$\log \Pr(x_0, \cdots, x_m) = \sum_{i=0}^{m} \log \Pr(x_i|x_0, \cdots, x_{i-1}). \tag{8.2}$$

Ở đây, $\Pr(x_i|x_0, \cdots, x_{i-1})$ là xác suất của token x_i khi biết mọi token đứng trước nó $\{x_0, \cdots, x_{i-1}\}$[2]. Trong kỷ nguyên học sâu, một cách tiếp cận điển hình để mô hình hóa ngôn ngữ là ước lượng các xác suất này bằng

[1] Trong một số mô hình như BERT, một token đặc biệt như [CLS] được thêm vào đầu chuỗi, dù nó phục vụ một mục đích khác với ký hiệu bắt đầu trong các mô hình tự hồi quy.

[2] Ta quy ước rằng khi $i = 0$, $\Pr(x_i|x_0, \cdots, x_{i-1}) = \Pr(x_0) = 1$. Do đó, $\Pr(x_0, \cdots, x_m) = \Pr(x_0) \Pr(x_1, \cdots, x_m|x_0) = \Pr(x_1, \cdots, x_m|x_0)$.

mạng nơ-ron sâu. Các mạng nơ-ron được huấn luyện cho nhiệm vụ này nhận vào một chuỗi ngữ cảnh $x_0, \cdots, x_{i-1}$ và xuất ra một phân phối xác suất (*probability distribution*) trên bộ từ vựng (*vocabulary*) V (ký hiệu là $\Pr(\cdot|x_0, \cdots, x_{i-1})$). Xác suất $\Pr(x_i|x_0, \cdots, x_{i-1})$ chính là phần tử ứng với token x_i trong phân phối này.

Chú giải của người dịch: Quy tắc chuỗi và xác suất có điều kiện

Ký hiệu $\Pr(b|a)$ đọc là "xác suất của b *khi biết* a" (xác suất có điều kiện (*conditional probability*)). Quy tắc chuỗi nói rằng xác suất để cả chuỗi xảy ra bằng tích các xác suất "từng bước một". Ví dụ với câu "tôi đi học":

$$\Pr(\text{tôi đi học}) = \Pr(\text{tôi}) \cdot \Pr(\text{đi} \mid \text{tôi}) \cdot \Pr(\text{học} \mid \text{tôi đi}).$$

Nếu ba xác suất lần lượt là 0.1, 0.2, 0.5 thì xác suất cả câu là $0.1 \cdot 0.2 \cdot 0.5 = 0.01$. Lấy logarit biến tích thành tổng (vì $\log(ab) = \log a + \log b$, như đã học ở lớp 11): $\log 0.01 = \log 0.1 + \log 0.2 + \log 0.5$. Dùng tổng các logarit giúp tránh việc nhân rất nhiều số nhỏ với nhau đến mức máy tính làm tròn thành 0.

Khi áp dụng một mô hình ngôn ngữ đã huấn luyện, một nhiệm vụ thường gặp là tìm token tiếp theo có khả năng cao nhất khi biết ngữ cảnh của nó. Điều này có thể được viết thành

$$\hat{x}_i \;=\; \arg\max_{x_i \in V} \Pr(x_i|x_0, \cdots, x_{i-1}). \tag{8.3}$$

Ta có thể thực hiện việc dự đoán token này lặp đi lặp lại để sinh văn bản một cách tuần tự: ở mỗi bước, ta dự đoán token có khả năng cao nhất $\hat{x}_i$, rồi nối nó vào ngữ cảnh để dự đoán token tiếp theo $\hat{x}_{i+1}$. Điều này tạo thành một quá trình sinh tự hồi quy (*autoregressive*), từ trái sang phải, hiện thực hóa Phương trình (8.1) và (8.2). Để minh họa, hãy xét việc sinh ba token khi biết tiền tố "$\langle s\rangle\ a$", như trong Bảng 8.1. Tiếp theo, chúng ta sẽ bàn về cách LLM được xây dựng, huấn luyện và áp dụng trong thực tế.

Chú giải của người dịch: Ký hiệu arg max

$\arg\max_{x \in V} f(x)$ nghĩa là "giá trị x trong tập V làm cho $f(x)$ lớn nhất" (khác với max, là chính giá trị lớn nhất của f). Ví dụ, nếu mô hình cho $\Pr(\text{học}|\ldots) = 0.5$, $\Pr(\text{chơi}|\ldots) = 0.3$, $\Pr(\text{ngủ}|\ldots) = 0.2$ thì arg max là từ "học", còn max là số 0.5. Chọn token theo cách này ở mỗi bước gọi là tìm

kiếm tham lam (*greedy search*).

Ngữ cảnh	Dự đoán	Quy tắc quyết định	Xác suất chuỗi
$\langle s\rangle\ a$	b	$\arg\max_{x_2\in V} \Pr(x_2\|\langle s\rangle\ a)$	$\Pr(\langle s\rangle) \cdot \Pr(a\|\langle s\rangle) \cdot \Pr(b\|\langle s\rangle\ a)$
$\langle s\rangle\ a\ b$	c	$\arg\max_{x_3\in V} \Pr(x_3\|\langle s\rangle\ a\ b)$	$\Pr(\langle s\rangle) \cdot \Pr(a\|\langle s\rangle) \cdot \Pr(b\|\langle s\rangle\ a) \cdot \Pr(c\|\langle s\rangle\ a\ b)$
$\langle s\rangle\ a\ b\ c$	d	$\arg\max_{x_4\in V} \Pr(x_4\|\langle s\rangle\ a\ b\ c)$	$\Pr(\langle s\rangle) \cdot \Pr(a\|\langle s\rangle) \cdot \Pr(b\|\langle s\rangle\ a) \cdot \Pr(c\|\langle s\rangle\ a\ b) \cdot \Pr(d\|\langle s\rangle\ a\ b\ c)$

Bảng 8.1: Minh họa việc sinh ba token $b\ c\ d$ khi biết tiền tố $\langle s\rangle\ a$ bằng một mô hình ngôn ngữ. Ở mỗi bước, mô hình chọn một token x_i từ V sao cho $\Pr(x_i|x_0, \cdots, x_{i-1})$ đạt cực đại. Token này sau đó được nối vào cuối chuỗi ngữ cảnh. Ở bước tiếp theo, ta lặp lại quá trình tương tự, nhưng dựa trên ngữ cảnh mới.

8.1.1 Transformer chỉ có bộ giải mã

Theo cách làm tiêu chuẩn, đầu vào của một mô hình ngôn ngữ là một chuỗi token (ký hiệu là $\{x_0, \cdots, x_{m-1}\}$). Ở mỗi bước, một token mới được sinh ra và nối vào chuỗi để phục vụ lần dự đoán tiếp theo. Để làm vậy, mô hình ngôn ngữ xuất ra một phân phối xác suất $\Pr(\cdot|x_0, \cdots, x_{i-1})$ tại mỗi vị trí i, và token tiếp theo được sinh ra theo phân phối này (ví dụ bằng lấy mẫu (*sampling*) hoặc giải mã tham lam (*greedy decoding*)). Mô hình được huấn luyện bằng cách cực đại hóa log-hợp lý (*log-likelihood*) $\sum_{i=1}^{m} \log \Pr(x_i|x_0, \cdots, x_{i-1})$[3].

Ở đây, ta tập trung vào kiến trúc Transformer chỉ có bộ giải mã (*decoder-only*), vì đây là một trong những kiến trúc được dùng rộng rãi nhất cho LLM. Chuỗi token đầu vào được biểu diễn bằng một chuỗi các vectơ d_e chiều $\{\mathbf{e}_0, \cdots, \mathbf{e}_{m-1}\}$. Mỗi vectơ $\mathbf{e}_i$ là tổng của vectơ nhúng token (*token embedding*) của x_i và vectơ nhúng vị trí (*positional embedding*) của vị trí i. Phần lõi của mô hình gồm một chồng các khối (hay tầng) Transformer. Mỗi khối Transformer gồm hai tầng con: một tầng con tự chú ý (*self-attention*) và một tầng con FFN. Các tầng con này

[3]Lưu ý rằng $\sum_{i=1}^{m} \log \Pr(x_i|x_0, \cdots, x_{i-1}) = \sum_{i=0}^{m} \log \Pr(x_i|x_0, \cdots, x_{i-1})$ vì $\log \Pr(x_0) = 0$.

có thể được định nghĩa theo kiến trúc *post-norm*

$$\text{output} \;=\; \text{LNorm}(F(\text{input}) + \text{input}), \tag{8.4}$$

hoặc kiến trúc *pre-norm*

$$\text{output} \;=\; F(\text{LNorm}(\text{input})) + \text{input}, \tag{8.5}$$

trong đó input và output ký hiệu đầu vào và đầu ra, cả hai đều là ma trận $m \times d$. Hàng thứ i của input và output có thể được hiểu là biểu diễn theo ngữ cảnh của token thứ i trong chuỗi.

$F(\cdot)$ là hàm lõi của một tầng con. Với tầng con FFN, $F(\cdot)$ là một FFN hai tầng. Với tầng con tự chú ý, $F(\cdot)$ là một hàm tự chú ý đa đầu (*multi-head self-attention*). Nói chung, tự chú ý được biểu diễn dưới dạng chú ý QKV

$$\text{Att}_{\text{qkv}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \;=\; \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{Mask}\right)\mathbf{V}, \tag{8.6}$$

trong đó $\mathbf{Q}$, $\mathbf{K}$ và $\mathbf{V} \in \mathbb{R}^{m\times d}$ lần lượt là các truy vấn (*queries*), khóa (*keys*) và giá trị (*values*). Điều quan trọng cần lưu ý là khi dự đoán một token, chỉ những token đứng trước mới được xét đến. Vì vậy, một biến che (mặt nạ) $\mathbf{Mask} \in \mathbb{R}^{m\times m}$ được đưa vào cơ chế tự chú ý để đạt được điều này. Phần tử (i, k) của $\mathbf{Mask}$ có giá trị 0 nếu $i \geq k$, và giá trị $-\infty$ trong trường hợp ngược lại.

Chú giải của người dịch: Mặt nạ trong tự chú ý

Hàm Softmax biến một dãy số $s_1, \ldots, s_n$ thành các trọng số $e^{s_k} / \sum_j e^{s_j}$, đều dương và có tổng bằng 1. Vì $e^{-\infty} = 0$, cộng $-\infty$ vào một vị trí sẽ khiến trọng số chú ý ở vị trí đó bằng đúng 0. Với $m = 3$, ma trận mặt nạ là

$$\mathbf{Mask} = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}:$$

hàng 1 (token đầu tiên) chỉ nhìn được chính nó, hàng 2 nhìn được token 1 và 2, hàng 3 nhìn được cả ba. Nhờ vậy mô hình không thể "nhìn trộm" các token phía sau — những token mà lúc sinh văn bản thật ta chưa hề có.

Cho một biểu diễn $\mathbf{H} \in \mathbb{R}^{m\times d}$, hàm tự chú ý đa đầu có thể được định nghĩa là

$$F(\mathbf{H}) \;=\; \text{Merge}(\text{head}_1, \cdots, \text{head}_\tau)\mathbf{W}^{\text{head}}, \tag{8.7}$$

trong đó Merge$(\cdot)$ biểu thị phép ghép nối các đầu vào của nó, và $\mathbf{W}^{\text{head}} \in \mathbb{R}^{d\times d}$ là một ma trận tham số. head_j là đầu ra của chú ý QKV trên một không gian con của biểu diễn:

$$\text{head}_j \;=\; \text{Att}_{\text{qkv}}(\mathbf{Q}^{[j]}, \mathbf{K}^{[j]}, \mathbf{V}^{[j]}). \tag{8.8}$$

Ở đây, $\mathbf{Q}^{[j]}$, $\mathbf{K}^{[j]}$ và $\mathbf{V}^{[j]}$ là các truy vấn, khóa và giá trị được chiếu lên không gian con thứ j thông qua các phép biến đổi tuyến tính

$$\mathbf{Q}^{[j]} = \mathbf{H}\mathbf{W}_j^q \tag{8.9}$$

$$\mathbf{K}^{[j]} = \mathbf{H}\mathbf{W}_j^k \tag{8.10}$$

$$\mathbf{V}^{[j]} = \mathbf{H}\mathbf{W}_j^v, \tag{8.11}$$

trong đó $\mathbf{W}_j^q$, $\mathbf{W}_j^k$ và $\mathbf{W}_j^v \in \mathbb{R}^{d\times \frac{d}{\tau}}$ là các ma trận tham số của các phép biến đổi.

(Nhãn trong hình: *Language Model* = mô hình ngôn ngữ; *L Blocks* = L khối; *Post-norm or Pre-norm* = chuẩn hóa sau hoặc chuẩn hóa trước; *Self-attention* = tự chú ý.)

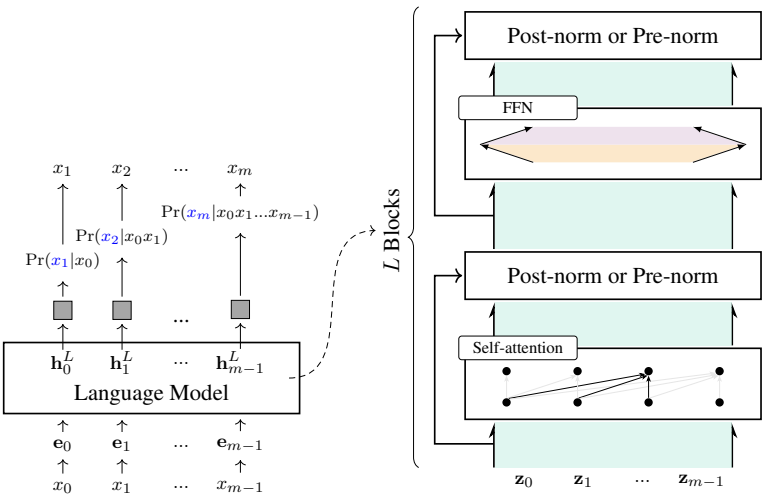
Giả sử ta có L khối Transformer. Một tầng Softmax được xây dựng phía trên đầu ra của khối cuối cùng. Tầng Softmax xuất ra một chuỗi gồm m phân phối trên bộ từ vựng, như sau

$$\begin{bmatrix} \Pr(\cdot|x_0) \\ \Pr(\cdot|x_0, x_1) \\ \vdots \\ \Pr(\cdot|x_0, \cdots, x_{m-1}) \end{bmatrix} \;=\; \text{Softmax}(\mathbf{H}^L\mathbf{W}^o), \tag{8.12}$$

trong đó $\mathbf{H}^L$ là đầu ra của khối Transformer cuối cùng, và $\mathbf{W}^o \in \mathbb{R}^{d\times|V|}$ là ma trận tham số.

Chú giải của người dịch: Kích thước các ma trận trong Phương trình (8.12)

$\mathbf{H}^L$ có m hàng (mỗi hàng ứng với một vị trí) và d cột; $\mathbf{W}^o$ có d hàng và $|V|$ cột ($|V|$ là số token trong bộ từ vựng). Tích $\mathbf{H}^L\mathbf{W}^o$ vì thế có kích thước $m \times |V|$: mỗi hàng là một dãy $|V|$ "điểm số", mỗi điểm cho một token có thể xuất hiện tiếp theo. Softmax áp dụng cho từng hàng biến các điểm số này thành xác suất có tổng bằng 1. Chẳng hạn với $|V| = 50\,000$, mỗi vị trí cho



Hình 8.1: Kiến trúc bộ giải mã Transformer dùng cho mô hình hóa ngôn ngữ. Thành phần trung tâm là L khối Transformer xếp chồng, mỗi khối gồm một tầng con tự chú ý và một tầng con FFN. Để ngăn mô hình truy cập ngữ cảnh bên phải, một biến che được đưa vào cơ chế tự chú ý. Tầng đầu ra dùng hàm Softmax để sinh một phân phối xác suất cho token tiếp theo, khi biết chuỗi các token đứng trước. Trong quá trình lập luận, mô hình lấy token vừa dự đoán để dự đoán token tiếp theo, lặp lại quá trình này cho đến khi đạt tới cuối chuỗi. $\{\mathbf{z}_0, \cdots, \mathbf{z}_{m-1}\}$ ký hiệu các đầu vào của một khối Transformer, và $\{\mathbf{h}_0^L, \cdots, \mathbf{h}_{m-1}^L\}$ ký hiệu các đầu ra của khối Transformer cuối cùng.

ra một bảng 50 000 xác suất, và token tiếp theo được chọn dựa trên bảng này.

Hình 8.1 cho thấy kiến trúc Transformer dùng cho mô hình hóa ngôn ngữ. Suy luận với mô hình này tuân theo một quá trình tự hồi quy. Mỗi lần, mô hình ngôn ngữ nhận tiền tố $x_0, \cdots, x_{i-1}$ làm đầu vào và dự đoán một token x_i làm cực đại xác suất $\Pr(x_i|x_0, \cdots, x_{i-1})$. Điều quan trọng cần lưu ý là, dù khác nhau ở các chi tiết hiện thực, nhiều LLM có chung kiến trúc được mô tả ở trên. Các mô hình này được gọi là *lớn* vì cả độ sâu lẫn độ rộng của chúng đều đáng kể. Bảng 8.2 cho thấy kích thước mô hình của một vài LLM, cùng với cấu hình mô hình của chúng.

LLM	Số tham số	Độ sâu L	Độ rộng d	Số đầu (Q/KV)
GPT-1 [Radford và cộng sự, 2018]	0.117B	12	768	12/12
GPT-2 [Radford và cộng sự, 2019]	1.5B	48	1,600	25/25
GPT-3 [Brown và cộng sự, 2020]	175B	96	12,288	96/96
Llama2 [Touvron và cộng sự, 2023b]	7B	32	4,096	32/32
	13B	40	5,120	40/40
	70B	80	8,192	64/64
Llama3/3.1 [Dubey và cộng sự, 2024]	8B	32	4,096	32/8
	70B	80	8,192	64/8
	405B	126	16,384	128/8
Gemma2 [Team và cộng sự, 2024]	2B	26	2,304	8/4
	9B	42	3,584	16/8
	37B	46	4,608	32/16
Qwen2.5 [Yang và cộng sự, 2024]	0.5B	24	896	14/2
	7B	28	3,584	28/4
	72B	80	8,192	64/8
DeepSeek-V3 [Liu và cộng sự, 2024a]	671B	61	7,168	128/128
Falcon [Penedo và cộng sự, 2023]	7B	32	4,544	71/71
	40B	60	8,192	128/128
	180B	80	14,848	232/232
Mistral [Jiang và cộng sự, 2023a]	7B	32	4,096	32/32

Bảng 8.2: So sánh một số LLM về kích thước mô hình, độ sâu mô hình, độ rộng mô hình và số đầu chú ý (a/b nghĩa là có a đầu cho truy vấn và b đầu dùng chung cho cả khóa và giá trị).

Chú giải của người dịch: Đọc Bảng 8.2

Chữ "B" là viết tắt của *billion* (tỷ): 175B nghĩa là 175 tỷ tham số. Dấu phẩy trong các số như 12,288 là dấu phân cách hàng nghìn theo kiểu tiếng Anh (tức 12 288). "Độ sâu" L là số khối Transformer xếp chồng, "độ rộng" d là số chiều của vectơ biểu diễn mỗi token. Khi số đầu khóa/giá trị (KV) nhỏ hơn số đầu truy vấn (như 64/8), nhiều đầu truy vấn dùng chung một bộ khóa–giá trị để tiết kiệm bộ nhớ; kỹ thuật này sẽ được bàn ở các phần sau.

8.1.2 Huấn luyện LLM

Bây giờ giả sử ta có một tập huấn luyện $\mathcal{D}$ gồm K chuỗi. Log-hợp lý của mỗi chuỗi $\mathbf{x} = x_0 \cdots x_m$ trong $\mathcal{D}$ có thể được tính bằng một mô hình ngôn ngữ

$$\mathcal{J}_\theta(\mathbf{x}) \;=\; \sum_{i=1}^{m} \log \Pr_\theta(x_i | x_0, \cdots, x_{i-1}), \tag{8.13}$$

trong đó chỉ số dưới θ trong $\mathcal{L}_\theta(\cdot)$ và $\Pr_\theta(\cdot)$ ký hiệu các tham số của mô hình ngôn ngữ. Khi đó, mục tiêu của huấn luyện hợp lý cực đại (*maximum likelihood*) được định nghĩa là

$$\hat{\theta} \;=\; \arg\max_\theta \sum_{\mathbf{x}\in\mathcal{D}} \mathcal{J}_\theta(\mathbf{x}). \tag{8.14}$$

Chú giải của người dịch: Huấn luyện hợp lý cực đại nói nôm na

θ là tập hợp tất cả các tham số (hàng tỷ con số) của mô hình. $\mathcal{J}_\theta(\mathbf{x})$ đo xem mô hình với bộ tham số θ "tin" vào câu thật $\mathbf{x}$ đến mức nào: mô hình càng gán xác suất cao cho đúng các token thật sự xuất hiện thì giá trị này càng lớn (càng gần 0, vì logarit của một số trong $(0, 1]$ là số không dương). Huấn luyện là đi tìm $\hat{\theta}$ sao cho tổng này trên toàn bộ dữ liệu là lớn nhất, tức là mô hình "đoán đúng từ tiếp theo" giỏi nhất có thể trên dữ liệu huấn luyện.

Việc huấn luyện các mô hình ngôn ngữ dựa trên Transformer với mục tiêu trên thường được xem là một quá trình tối ưu hóa chuẩn cho mạng nơ-ron. Điều này có thể thực hiện bằng các thuật toán hạ gradient (*gradient descent*), vốn được hỗ trợ rộng rãi bởi các bộ công cụ học sâu có sẵn. Có phần đáng ngạc nhiên là hiệu năng mô hình vẫn tiếp tục được cải thiện khi các mô hình ngôn ngữ tiến hóa thành những kiến trúc đòi hỏi nhiều

tính toán hơn và được huấn luyện trên những tập dữ liệu lớn hơn [Kaplan và cộng sự, 2020]. Những thành công này đã thúc đẩy các nhà nghiên cứu NLP liên tục mở rộng cả dữ liệu huấn luyện lẫn kích thước mô hình để xây dựng những mô hình ngôn ngữ ngày càng mạnh.

Tuy nhiên, khi mô hình ngôn ngữ trở nên lớn hơn, ta gặp phải những thách thức huấn luyện mới, làm thay đổi đáng kể bài toán so với việc huấn luyện các mô hình tương đối nhỏ. Một trong những thách thức đó xuất phát từ nhu cầu cần có các hệ thống phân tán quy mô lớn để quản lý dữ liệu, tham số mô hình và các quy trình huấn luyện. Việc phát triển và duy trì những hệ thống như vậy đòi hỏi nỗ lực đáng kể về cả kỹ thuật phần mềm lẫn phần cứng, bên cạnh chuyên môn về học sâu. Một vấn đề liên quan là việc mở rộng quy mô huấn luyện đòi hỏi tài nguyên tính toán khổng lồ để hoàn thành quá trình trong một khoảng thời gian hợp lý. Chẳng hạn, huấn luyện từ đầu một LLM có hàng chục tỷ tham số thường cần hàng trăm hoặc hàng nghìn GPU. Yêu cầu này làm tăng mạnh chi phí huấn luyện, nhất là khi tính đến rất nhiều vòng thử nghiệm cần thiết trong quá trình phát triển mô hình. Hơn nữa, xét từ góc độ tối ưu hóa, quá trình huấn luyện có thể trở nên rất bất ổn định khi tăng độ sâu mạng và số lượng tham số. Do đó, thường cần sửa đổi kiến trúc để LLM thích ứng với việc huấn luyện quy mô lớn. Trong Mục 8.2, chúng ta sẽ bàn chi tiết hơn về những vấn đề này.

8.1.3 Tinh chỉnh và ứng dụng LLM

Sau khi đã tiền huấn luyện một LLM, ta có thể áp dụng nó để thực hiện nhiều nhiệm vụ NLP khác nhau. Theo truyền thống, mô hình ngôn ngữ được dùng như thành phần của những hệ thống khác. Chẳng hạn, chúng từng được dùng rộng rãi để chấm điểm các bản dịch trong các hệ thống dịch máy thống kê. Ngược lại, trong AI tạo sinh, LLM được xem là những hệ thống hoàn chỉnh và được dùng để giải quyết các bài toán NLP bằng cách tận dụng bản chất sinh của chúng. Một cách tiếp cận phổ biến là mô tả nhiệm vụ cần làm bằng văn bản rồi *prompt* LLM sinh văn bản dựa trên mô tả này. Đây là một nhiệm vụ sinh văn bản tiêu chuẩn, trong đó mô hình viết tiếp hoặc hoàn thành văn bản bắt đầu từ một ngữ cảnh cho trước.

Một cách hình thức hơn, gọi $\mathbf{x} = x_0 \cdots x_m$ là một chuỗi token biểu diễn ngữ cảnh do người dùng cung cấp, và $\mathbf{y} = y_1 \cdots y_n$ là một chuỗi

token nối tiếp ngữ cảnh đó. Quá trình suy luận của một LLM có thể được định nghĩa là tìm chuỗi $\mathbf{y}$ có khả năng cao nhất khi biết $\mathbf{x}$:

$$\begin{aligned}\hat{\mathbf{y}} &= \arg\max_{\mathbf{y}} \log \Pr(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{\mathbf{y}} \sum_{i=1}^{n} \log \Pr(y_i|x_0, \cdots, x_m, y_1, \cdots, y_{i-1}). \end{aligned} \tag{8.15}$$

Ở đây, $\sum_{i=1}^{n} \log \Pr(y_i|x_0, \cdots, x_m, y_1, \cdots, y_{i-1})$ về mặt toán học tương đương với vế phải của Phương trình (8.2). Nó mô hình hóa log-xác suất của việc dự đoán các token bắt đầu từ vị trí $m+1$, thay vì từ vị trí 0. Trong suốt chương này và các chương sau, chúng ta sẽ dùng các biến riêng $\mathbf{x}$ và $\mathbf{y}$ để phân biệt đầu vào và đầu ra của một LLM, mặc dù chúng có thể được xem là các chuỗi con thuộc cùng một chuỗi liên tục. Khi dùng ký hiệu này, ta có thể thấy phương trình trên rất giống với các công thức dùng trong những mô hình sinh văn bản khác của NLP, chẳng hạn các kiến trúc dịch máy nơ-ron.

Để minh họa cách áp dụng LLM, hãy xét bài toán xác định một câu cho trước có đúng ngữ pháp hay không. Ta có thể định nghĩa một khuôn mẫu (*template*) như sau

> {∗sentence∗}
> Câu hỏi: Câu này có đúng ngữ pháp không?
> Trả lời: ___

Ở đây ___ biểu diễn phần văn bản ta muốn sinh ra. {∗sentence∗} là một biến chỗ trống (*placeholder*) sẽ được thay bằng câu thực tế do người dùng cung cấp. Ví dụ, giả sử ta có câu "*John seems happy today.*" (John hôm nay có vẻ vui). Ta có thể thay {∗sentence∗} trong khuôn mẫu bằng câu này để tạo đầu vào cho mô hình ngôn ngữ

> John seems happy today.
> Câu hỏi: Câu này có đúng ngữ pháp không?
> Trả lời: ___

Để thực hiện nhiệm vụ, mô hình ngôn ngữ được cho ngữ cảnh $\mathbf{x}$ = "John seems happy today .\n Câu hỏi : Câu này có đúng ngữ pháp

không?\n Trả lời :"[4]. Sau đó, dựa trên ngữ cảnh, nó sinh ra phần văn bản tiếp theo làm câu trả lời. Chẳng hạn, mô hình ngôn ngữ có thể xuất ra "Có" (tức là $\mathbf{y}$ = "Có") nếu đây là văn bản có xác suất cao nhất khi biết ngữ cảnh này.

Chú giải của người dịch: Vì sao một mô hình "đoán từ tiếp theo" lại trả lời được câu hỏi?

Mô hình không có "nút" nào dành riêng cho việc kiểm tra ngữ pháp. Nó chỉ làm đúng một việc: viết tiếp văn bản sao cho hợp lý nhất. Nhưng khi văn bản kết thúc bằng "Câu hỏi: … Trả lời:", cách viết tiếp hợp lý nhất chính là một câu trả lời đúng cho câu hỏi đó — vì trong hàng tỷ trang văn bản mô hình đã đọc, sau những câu hỏi như vậy thường là câu trả lời đúng. Như thế, bằng cách khéo léo "đóng khung" nhiệm vụ thành một đoạn văn cần viết tiếp, ta biến mọi bài toán thành bài toán sinh văn bản.

Tương tự, ta có thể định nghĩa thêm các khuôn mẫu để giải quyết những nhiệm vụ khác. Ví dụ, ta có thể dịch một câu tiếng Anh sang tiếng Trung bằng khuôn mẫu sau

```
{*sentence*}
Câu hỏi: Bản dịch tiếng Trung của câu tiếng Anh này là gì?
Trả lời: ___
```

hoặc dùng một khuôn mẫu dạng chỉ dẫn

```
{*sentence*}
Hãy dịch câu này từ tiếng Anh sang tiếng Trung.

___
```

hoặc dùng một khuôn mẫu dạng mã.

```
[src-lang] = English [tgt-lang] = Chinese [input] = {*sentence*}
[output] = ___
```

(Trong khuôn mẫu dạng mã, `src-lang` là ngôn ngữ nguồn, `tgt-lang` là ngôn ngữ đích, `input/output` là đầu vào/đầu ra.)

Các khuôn mẫu trên cung cấp một phương pháp đơn giản mà hiệu quả

[4] \n là một ký tự đặc biệt dùng để xuống dòng.

để "prompt" một LLM duy nhất thực hiện nhiều nhiệm vụ khác nhau mà không cần thay đổi cấu trúc của mô hình. Tuy nhiên, cách làm này đòi hỏi LLM phải nhận biết được và làm theo các chỉ dẫn hay câu hỏi. Một cách để đạt được điều đó là đưa các mẫu huấn luyện gồm chỉ dẫn và câu trả lời tương ứng vào tập dữ liệu tiền huấn luyện. Dù phương pháp này đơn giản, việc xây dựng và huấn luyện LLM từ đầu rất tốn kém về tính toán. Hơn nữa, để dữ liệu làm theo chỉ dẫn phát huy hiệu quả trong tiền huấn luyện, cần một lượng lớn dữ liệu như vậy, nhưng việc thu thập dữ liệu có nhãn quy mô lớn cho mọi nhiệm vụ quan tâm là rất khó.

Phương pháp thứ hai, vốn đã trở thành chuẩn mực trên thực tế trong các nghiên cứu gần đây, là điều chỉnh LLM thông qua tinh chỉnh (*fine-tuning*). Nhờ đó, năng lực dự đoán token học được trong giai đoạn tiền huấn luyện có thể được tổng quát hóa để hoàn thành các nhiệm vụ mới. Ý tưởng đằng sau tinh chỉnh là một phần tri thức tổng quát về ngôn ngữ đã được thu nhận trong quá trình tiền huấn luyện, nhưng ta cần một cơ chế để *kích hoạt* tri thức này nhằm áp dụng nó cho nhiệm vụ mới. Để làm được điều đó, ta có thể tinh chỉnh nhẹ các tham số mô hình bằng dữ liệu làm theo chỉ dẫn. Cách tiếp cận này được gọi là **tinh chỉnh theo chỉ dẫn** (*instruction fine-tuning*).

Một mẫu tinh chỉnh theo chỉ dẫn, được biểu diễn bằng một chuỗi token, có thể được xem là một bộ gồm một đầu vào và đầu ra mong muốn. Ở đây, đầu vào bao gồm các chỉ dẫn, thông tin hệ thống (hay tiền tố hệ thống), và bất kỳ thông tin nào khác do người dùng cung cấp[5]. Để minh họa, hãy xét các ví dụ sau (chữ màu xanh = đầu vào và chữ gạch dưới = đầu ra).

Đọc câu sau và trả lời xem nó có đúng ngữ pháp không.
LLMs are powerful models but are expensive to build.
Có

LLMs are powerful models but are expensive to build.
Câu này có hợp lý về mặt ngữ pháp không?
Phương án trả lời
- Có

[5]Thông tin hệ thống là một chuỗi token được thêm vào đầu một đầu vào nhằm định hướng hành vi của LLM, chẳng hạn: *bạn là một trợ lý hữu ích và không được xuất ra nội dung độc hại*.

- Không
Có

Có hay Không? Có = câu sau đúng ngữ pháp. Không = câu sau có lỗi ngữ pháp.
LLMs are powerful models but are expensive to build.
Có

(Câu tiếng Anh được giữ nguyên vì đó là đối tượng cần kiểm tra ngữ pháp; nghĩa: "LLM là những mô hình mạnh nhưng tốn kém để xây dựng.")

Tất cả các mẫu này mô tả cùng một nhiệm vụ phân loại nhị phân (*binary classification*), nhưng với những chỉ dẫn khác nhau. Để tăng tính đa dạng của dữ liệu tinh chỉnh và giúp LLM tổng quát hóa, ta có thể định nghĩa thêm nhiều nhiệm vụ bằng chỉ dẫn.

Hãy viết một truyện ngắn về ba nhân vật.
Nhân vật 1: Một cậu bé tên là Andy.
Nhân vật 2: Chú chó của Andy.
Nhân vật 3: Bạn của Andy, Jane.
Giữa lòng một thành phố nhộn nhịp, Andy, chú chó trung thành Rocket và cô bạn Jane đã khám phá ra một khu vườn bí mật dưới ánh hoàng hôn. Được bao quanh bởi những hàng cây cổ thụ và tiếng gió thì thầm, họ chia sẻ cho nhau những ước mơ và bí mật, gắn kết một tình bạn không gì phá vỡ được. Khi màn đêm buông xuống, họ cùng hứa dưới bầu trời sao rằng sẽ luôn tìm thấy điều kỳ diệu trong những điều bình dị. Cùng nhau, họ nhận ra rằng cuộc phiêu lưu thực sự nằm ở những khoảnh khắc bên bạn bè, và mỗi ngày đều là một kho báu đang chờ được khám phá.

Hãy chỉ cho tôi công thức làm kem.
Nguyên liệu:
2 cốc kem tươi béo; 1 cốc sữa; 3/4 cốc đường; 2 thìa cà phê tinh chất vani; Một nhúm muối.
Các bước:
1. Đánh đều tất cả nguyên liệu cho đến khi đường tan.
2. Để hỗn hợp trong tủ lạnh 2 giờ cho lạnh.
3. Cấp đông trong một khay nông, khuấy 30 phút một lần, 3–4 lần.
4. Cấp đông thêm 2 giờ cho đông cứng.

5. Thưởng thức món kem tự làm của bạn.

Nếu bạn mua 5 quả táo và mỗi quả giá $1.20, thì tổng cộng bạn tiêu bao nhiêu tiền?

$6.00

Hãy viết một chương trình Python để tính tổng bình phương của các số sau. 1 , 2 , 10 , -9 , 78

```
numbers = [1,2,10,-9,78]
sum_of_squares = sum(x**2 for x in numbers)
print(sum_of_squares)
```

Để có được năng lực làm theo chỉ dẫn, cần một lượng dữ liệu tinh chỉnh nhất định. Dữ liệu này có thể gồm những chỉ dẫn đa dạng và các câu trả lời khả dĩ. Người ta nhận thấy rằng tăng số lượng nhiệm vụ tinh chỉnh có lợi cho việc cải thiện hiệu năng của LLM [Chung và cộng sự, 2022]. Lưu ý rằng mặc dù càng nhiều dữ liệu tinh chỉnh càng tốt, lượng dữ liệu này thường nhỏ hơn dữ liệu tiền huấn luyện nhiều bậc độ lớn. Chẳng hạn, LLM có thể được tinh chỉnh với hàng chục hoặc hàng trăm nghìn mẫu, thậm chí ít hơn nếu các mẫu có chất lượng cao [Zhou và cộng sự, 2023a; Chen và cộng sự, 2023b], trong khi tiền huấn luyện những mô hình như vậy có thể cần hàng tỷ hoặc hàng nghìn tỷ token, dẫn đến nhu cầu tính toán lớn hơn đáng kể và thời gian huấn luyện dài hơn [Touvron và cộng sự, 2023a].

Cũng cần lưu ý rằng ta không nên kỳ vọng dữ liệu tinh chỉnh bao quát mọi nhiệm vụ hạ nguồn mà ta định áp dụng LLM. Một cách hiểu phổ biến về cơ chế hoạt động của cách tiếp cận *tiền huấn luyện + tinh chỉnh* là: LLM đã thu nhận được tri thức để hiểu chỉ dẫn và sinh câu trả lời trong giai đoạn tiền huấn luyện. Tuy nhiên, những năng lực này chưa được kích hoạt hoàn toàn cho đến khi ta đưa vào một hình thức giám sát nào đó. Hành vi làm theo chỉ dẫn tổng quát xuất hiện khi ta tinh chỉnh mô hình với một lượng tương đối nhỏ dữ liệu có nhãn. Nhờ vậy, ta có thể đạt được một mức độ **học zero-shot** (*zero-shot learning*): các mô hình đã tinh chỉnh có thể xử lý những nhiệm vụ mới mà chúng chưa từng được huấn luyện hay tinh chỉnh một cách tường minh [Sanh và cộng sự, 2022; Wei và cộng sự, 2022a]. Năng lực học zero-shot này phân biệt LLM sinh với các mô

hình tiền huấn luyện trước đây như BERT, vốn chủ yếu được tinh chỉnh cho các nhiệm vụ cụ thể.

Chú giải của người dịch: Tiền huấn luyện và tinh chỉnh: một phép so sánh

Có thể hình dung tiền huấn luyện giống như nhiều năm đọc sách: mô hình đọc hàng nghìn tỷ token và tích lũy hiểu biết rộng về ngôn ngữ và thế giới, nhưng chưa biết cách "làm bài theo đề". Tinh chỉnh theo chỉ dẫn giống như một khóa luyện ngắn với vài chục nghìn bài mẫu "đề bài — lời giải": không dạy thêm nhiều kiến thức mới, mà dạy cách dùng kiến thức sẵn có để trả lời đúng yêu cầu. "Zero-shot" nghĩa là sau khóa luyện đó, mô hình làm được cả những dạng bài chưa từng xuất hiện trong bài mẫu.

Khi đã tuyển chọn xong một tập dữ liệu chỉ dẫn, bản thân quá trình tinh chỉnh tương đối đơn giản. Nó có thể được xem là việc huấn luyện mạng nơ-ron tiêu chuẩn, tương tự tiền huấn luyện, nhưng được thực hiện trên một tập huấn luyện nhỏ hơn nhiều. Gọi $\mathcal{D}_{\text{tune}}$ là tập dữ liệu tinh chỉnh và $\hat{\theta}$ là các tham số mô hình đã được tối ưu qua tiền huấn luyện. Ta có thể sửa Phương trình (8.14) để có được mục tiêu của tinh chỉnh

$$\tilde{\theta} \;=\; \arg\max_{\hat{\theta}^+} \sum_{\text{sample}\in\mathcal{D}_{\text{tune}}} \mathcal{J}_{\hat{\theta}^+}(\text{sample}). \tag{8.16}$$

Ở đây $\tilde{\theta}$ ký hiệu các tham số tối ưu. Ký hiệu $\hat{\theta}^+$ có nghĩa là quá trình tinh chỉnh bắt đầu từ các tham số đã tiền huấn luyện $\hat{\theta}$.

Với mỗi sample $\in \mathcal{D}_{\text{tune}}$ (mỗi *mẫu*), ta chia nó thành một đoạn đầu vào $\mathbf{x}_{\text{sample}}$ và một đoạn đầu ra $\mathbf{y}_{\text{sample}}$, tức là

$$\text{sample} \;=\; [\mathbf{x}_{\text{sample}}, \mathbf{y}_{\text{sample}}]. \tag{8.17}$$

Sau đó ta định nghĩa hàm log-hợp lý là

$$\mathcal{J}_{\hat{\theta}^+}(\text{sample}) \;=\; \log \Pr_{\hat{\theta}^+}(\mathbf{y}_{\text{sample}} | \mathbf{x}_{\text{sample}}). \tag{8.18}$$

Nói cách khác, ta tính hàm mất mát trên chuỗi con $\mathbf{y}_{\text{sample}}$ thay vì trên toàn bộ chuỗi. Khi hiện thực lan truyền ngược (*back-propagation*) cho phương trình này trong thực tế, chuỗi $[\mathbf{x}_{\text{sample}}, \mathbf{y}_{\text{sample}}]$ được xây dựng trong lượt truyền xuôi (*forward pass*) như thường lệ. Tuy nhiên, trong lượt truyền ngược (*backward pass*), gradient lỗi chỉ được lan truyền ngược qua những

phần của mạng tương ứng với $\mathbf{y}_{\text{sample}}$, còn phần còn lại của mạng giữ nguyên. Ví dụ, xét chuỗi

$$\underbrace{\langle s \rangle \text{ Bình phương số này . 2 .}}_{\text{Ngữ cảnh (Đầu vào)}} \quad \underbrace{\text{Kết quả là 4 .}}_{\text{Dự đoán (Đầu ra)}}$$

Hàm mất mát chỉ được tính và lan truyền ngược cho Kết quả là 4 ..

Chú giải của người dịch: Vì sao chỉ tính mất mát trên phần đầu ra?

Phần đầu vào (câu hỏi, chỉ dẫn) là do người dùng viết; ta không cần mô hình học cách *viết ra câu hỏi*, mà chỉ cần nó học cách *trả lời*. Vì thế, trong ví dụ trên, mô hình vẫn "đọc" cả chuỗi, nhưng chỉ bị "chấm điểm" ở các token *Kết quả là 4 .*: nếu nó gán xác suất thấp cho các token này thì bị phạt và tham số được điều chỉnh; còn các token của câu hỏi không đóng góp vào hàm mất mát.

Tinh chỉnh theo chỉ dẫn cũng đòi hỏi khối lượng công việc kỹ thuật đáng kể. Để đạt kết quả thỏa đáng, người ta có thể phải thử nghiệm với các thiết lập khác nhau của tốc độ học (*learning rate*), kích thước lô (*batch*), số bước tinh chỉnh, v.v. Điều này thường đòi hỏi nhiều lượt tinh chỉnh và đánh giá. Chi phí và công sức thử nghiệm của tinh chỉnh vẫn rất đáng kể và không nên bị xem nhẹ, dù chúng thấp hơn nhiều so với giai đoạn tiền huấn luyện.

Mặc dù ở đây ta tập trung vào tinh chỉnh theo chỉ dẫn như một ví dụ minh họa, các kỹ thuật tinh chỉnh đóng vai trò quan trọng trong việc phát triển nhiều loại LLM và được dùng rộng rãi hơn thế. Ví dụ bao gồm tinh chỉnh LLM thành chatbot (*chatbot*) bằng dữ liệu hội thoại, và điều chỉnh các mô hình này để xử lý những chuỗi rất dài. Việc tinh chỉnh được ứng dụng rộng rãi đã khiến các nhà nghiên cứu cải tiến những kỹ thuật này, chẳng hạn thiết kế các thuật toán tinh chỉnh hiệu quả hơn. Dù nghiên cứu về tinh chỉnh rất phong phú, trong mục này chúng tôi chỉ trình bày khái quát các bước then chốt. Ta sẽ thấy những thảo luận chi tiết hơn về chủ đề này trong các chương sau.

8.1.4 Căn chỉnh LLM với thế giới

Tinh chỉnh theo chỉ dẫn cung cấp một cách đơn giản để điều chỉnh LLM cho những nhiệm vụ có thể được định nghĩa rõ ràng. Nhìn rộng ra, điều này có thể được xem là một bài toán **căn chỉnh** (*alignment*). Ở đây, căn chỉnh là quá trình dẫn dắt LLM hành xử theo những cách phù hợp với các giá trị và mục tiêu của con người. Sự dẫn dắt này có thể đến từ dữ liệu có nhãn, phản hồi của con người (*human feedback*), hoặc bất kỳ hình thức nào khác của sở thích (ưu tiên) của con người (*human preferences*). Chẳng hạn, ta muốn LLM không chỉ làm theo chỉ dẫn một cách chính xác, mà còn không thiên lệch, trung thực và vô hại. Vì vậy, ta cần giám sát mô hình hướng tới các giá trị và kỳ vọng của con người. Một ví dụ điển hình là khi ta hỏi một LLM cách chế tạo vũ khí, nó có thể đưa ra danh sách các bước then chốt để làm việc đó nếu không được căn chỉnh cẩn thận. Tuy nhiên, một mô hình có trách nhiệm cần nhận ra và tránh đáp ứng những yêu cầu về thông tin có hại hoặc bất hợp pháp. Trong trường hợp này, căn chỉnh là yếu tố then chốt để bảo đảm LLM hành động có trách nhiệm và tuân thủ các nguyên tắc đạo đức.

Một khái niệm liên quan chặt chẽ đến căn chỉnh là an toàn AI (*AI safety*). Một mục tiêu tối thượng của nghiên cứu AI là xây dựng những hệ thống thông minh an toàn và có ích cho xã hội. Để đạt được điều này, ta phải bảo đảm các hệ thống đó luôn vững vàng, bảo mật và kiểm soát được trong nhiều điều kiện thực tế khác nhau, kể cả những trường hợp bị lạm dụng hay bị tấn công đối kháng (*adversarial attacks*). Với LLM, mức độ an toàn có thể được nâng cao đáng kể nhờ căn chỉnh đúng đắn theo sự hướng dẫn của con người, thông qua dữ liệu do con người gán nhãn và các tương tác động với người dùng trong quá trình triển khai.

Căn chỉnh là việc khó vì các giá trị và kỳ vọng của con người rất đa dạng và luôn thay đổi. Đôi khi, rất khó mô tả chính xác con người muốn gì, trừ khi ta thấy được câu trả lời của LLM cho các yêu cầu của người dùng. Điều này khiến căn chỉnh không còn là bài toán điều chỉnh LLM trên những nhiệm vụ định sẵn, mà là một bài toán lớn hơn: huấn luyện chúng thông qua tương tác với thế giới thực.

Do những lo ngại về việc kiểm soát các hệ thống AI, nghiên cứu về vấn đề căn chỉnh LLM đã bùng nổ. Thông thường, hai bước căn chỉnh được áp dụng sau khi LLM được tiền huấn luyện trên dữ liệu không nhãn

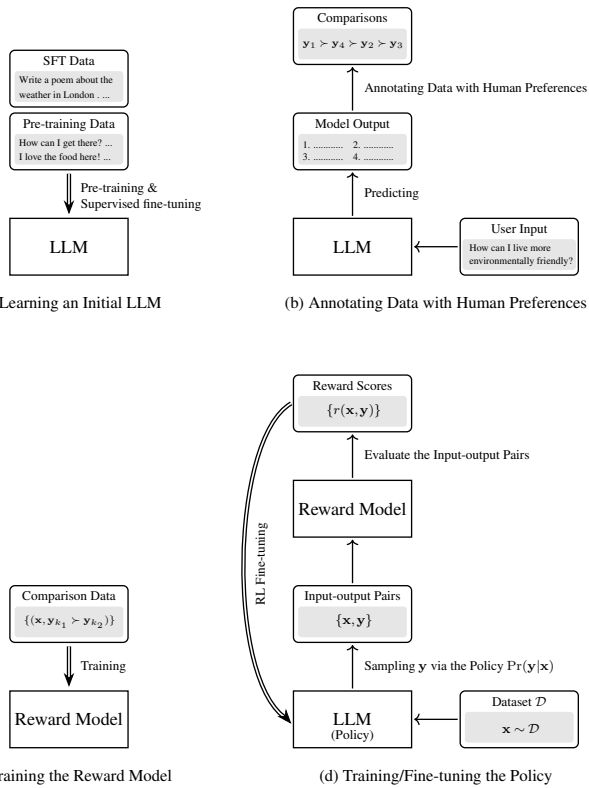
quy mô lớn.

- **Tinh chỉnh có giám sát** (**SFT**, *Supervised Fine-tuning*). Bước này tiếp tục huấn luyện LLM đã tiền huấn luyện trên dữ liệu có nhãn mới, hướng tới nhiệm vụ. Một kỹ thuật SFT thường dùng là tinh chỉnh theo chỉ dẫn. Như đã mô tả ở tiểu mục trước, bằng cách học từ dữ liệu được chú thích theo cặp chỉ dẫn–câu trả lời, LLM có thể được căn chỉnh với các hành vi mong muốn trong việc làm theo chỉ dẫn, nhờ đó có khả năng thực hiện nhiều nhiệm vụ được mô tả bằng chỉ dẫn. Tinh chỉnh có giám sát có thể được xem là đi theo mô thức *tiền huấn luyện + tinh chỉnh*, và là một phương pháp tương đối đơn giản để điều chỉnh LLM.

- **Học từ phản hồi của con người** (*Learning from Human Feedback*). Sau tiền huấn luyện và tinh chỉnh có giám sát, LLM có thể đáp ứng yêu cầu của người dùng khi được prompt phù hợp. Tuy nhiên, mô hình vẫn có thể sinh ra nội dung sai sự thật, thiên lệch hoặc có hại. Để căn chỉnh LLM tốt hơn với các giá trị của con người, một cách hiệu quả là học trực tiếp từ phản hồi của con người. Chẳng hạn, cho một chỉ dẫn và câu trả lời do mô hình sinh ra, các chuyên gia được yêu cầu đánh giá xem câu trả lời phù hợp đến mức nào với sở thích của họ và với các nguyên tắc an toàn. Phản hồi này sau đó được dùng để tiếp tục huấn luyện LLM nhằm đạt được sự căn chỉnh tốt hơn.

(Nhãn trong hình: (a) *Learning an Initial LLM* = học một LLM ban đầu, *Pre-training Data / SFT Data* = dữ liệu tiền huấn luyện / dữ liệu SFT; (b) *Annotating Data with Human Preferences* = gán nhãn dữ liệu theo sở thích của con người, *User Input* = đầu vào của người dùng ("Làm thế nào để tôi sống thân thiện với môi trường hơn?"), *Model Output* = đầu ra mô hình, *Predicting* = dự đoán, *Comparisons* = các so sánh; (c) *Training the Reward Model* = huấn luyện mô hình phần thưởng, *Comparison Data* = dữ liệu so sánh; (d) *Training/Fine-tuning the Policy* = huấn luyện/tinh chỉnh chính sách, *Sampling* **y** *via the Policy* = lấy mẫu **y** theo chính sách, *Input-output Pairs* = các cặp đầu vào–đầu ra, *Evaluate the Input-output Pairs* = đánh giá các cặp đầu vào–đầu ra, *Reward Scores* = điểm phần thưởng, *RL Fine-tuning* = tinh chỉnh bằng học tăng cường.)

Một phương pháp điển hình để học từ phản hồi của con người là



(a) Learning an Initial LLM

(b) Annotating Data with Human Preferences

(c) Training the Reward Model

(d) Training/Fine-tuning the Policy

Hình 8.2: Tổng quan về RLHF. Có 4 bước then chốt: a) huấn luyện một LLM ban đầu (tức chính sách) bằng tiền huấn luyện và tinh chỉnh có giám sát; b) thu thập dữ liệu sở thích của con người bằng cách xếp hạng các đầu ra của LLM; c) huấn luyện một mô hình phần thưởng bằng kết quả xếp hạng; d) tinh chỉnh chính sách bằng học tăng cường dựa trên mô hình phần thưởng. Các mũi tên nét đôi biểu thị huấn luyện hoặc tinh chỉnh.

xem nó như một bài toán học tăng cường (*reinforcement learning, RL*), được gọi là **học tăng cường từ phản hồi của con người** (*reinforcement learning from human feedback*) (**RLHF**) [Ouyang và cộng sự, 2022]. Phương pháp RLHF ban đầu được đề xuất để giải quyết các bài toán ra quyết định tuần tự tổng quát [Christiano và cộng sự, 2017], và sau đó được sử dụng thành công trong việc phát triển các mô hình thuộc dòng GPT [Stiennon và cộng sự, 2020]. Là một cách tiếp cận học tăng cường, mục tiêu của RLHF là học một chính sách (*policy*) bằng cách cực đại hóa một phần thưởng nào đó nhận được từ môi trường. Cụ thể, RLHF gồm hai thành phần chính:

- **Tác tử** (*Agent*). Tác tử, còn được gọi là tác tử LM, chính là LLM mà ta muốn tối ưu. Tác tử này hoạt động bằng cách tương tác với môi trường của nó: nó nhận văn bản từ môi trường và tạo ra các câu trả lời bằng văn bản. Chính sách của tác tử này là phân phối xác suất có điều kiện do LLM định nghĩa, ký hiệu là $\Pr(\mathbf{y}|\mathbf{x})$.

- **Mô hình phần thưởng** (*Reward Model*). Mô hình phần thưởng đóng vai trò đại diện thay cho môi trường con người. Mỗi khi tác tử tạo ra một chuỗi đầu ra, mô hình phần thưởng đánh giá nó và gán cho nó một điểm số (tức là phần thưởng (*reward*)). Điểm số này cho tác tử biết một cách tường minh về chất lượng và mức độ căn chỉnh của đầu ra.

Chú giải của người dịch: Học tăng cường qua hình ảnh huấn luyện thú cưng

Học tăng cường giống cách ta dạy một chú chó: ta không chỉ cho nó từng động tác đúng, mà để nó tự thử, rồi thưởng khi nó làm tốt. Dần dần nó làm nhiều hơn những hành vi được thưởng. Ở đây, "chú chó" là LLM (tác tử), "hành vi" là câu trả lời nó viết ra, còn "người thưởng" là mô hình phần thưởng — một mô hình được huấn luyện để chấm điểm giống như con người sẽ chấm. *Chính sách* chỉ đơn giản là "quy tắc hành xử" của tác tử: với LLM, đó là phân phối xác suất $\Pr(\mathbf{y}|\mathbf{x})$ quyết định nó trả lời $\mathbf{y}$ thế nào cho mỗi câu hỏi $\mathbf{x}$.

Trong RLHF, có hai giai đoạn học riêng biệt được thực hiện: 1) học mô hình phần thưởng, tức là huấn luyện mô hình phần thưởng bằng phản hồi của con người về các đầu ra của tác tử, và 2) học chính sách, tức là dùng các thuật toán học tăng cường để tối ưu chính sách dưới sự dẫn dắt

của mô hình phần thưởng đã huấn luyện. Các bước then chốt của RLHF có thể được tóm tắt như sau:

- Xây dựng một chính sách ban đầu bằng tiền huấn luyện và tinh chỉnh theo chỉ dẫn.
- Dùng chính sách để sinh nhiều đầu ra cho mỗi đầu vào, rồi thu thập phản hồi của con người về các đầu ra này (ví dụ: so sánh giữa các đầu ra).
- Học một mô hình phần thưởng từ phản hồi của con người.
- Tinh chỉnh chính sách dưới sự giám sát của mô hình phần thưởng.

Hình 8.2 trình bày tổng quan về RLHF. Vì mục này chỉ là phần giới thiệu ngắn về các khái niệm của LLM, chúng tôi sẽ không bàn chi tiết các kỹ thuật RLHF. Thay vào đó, chúng tôi minh họa các ý tưởng cơ bản đằng sau RLHF bằng một ví dụ đơn giản.

Giả sử ta đã huấn luyện một LLM bằng tiền huấn luyện và tinh chỉnh theo chỉ dẫn. LLM này được triển khai để đáp ứng yêu cầu của người dùng. Chẳng hạn, một người dùng có thể nhập

> Làm thế nào để tôi có một lối sống thân thiện với môi trường hơn?

Ta dùng LLM để sinh 4 đầu ra khác nhau (ký hiệu là $\{\mathbf{y}_1, \cdots, \mathbf{y}_4\}$) bằng cách lấy mẫu từ phân phối đầu ra

> Đầu ra 1 ($\mathbf{y}_1$): Hãy cân nhắc chuyển sang dùng xe điện hoặc xe đạp thay cho ô tô truyền thống để giảm phát thải carbon và bảo vệ hành tinh của chúng ta.
>
> Đầu ra 2 ($\mathbf{y}_2$): Hãy theo đuổi lối sống tối giản. Sở hữu ít đồ đạc hơn để giảm tiêu dùng và giảm tác động môi trường của việc sản xuất và thải bỏ.
>
> Đầu ra 3 ($\mathbf{y}_3$): Hãy sống tách khỏi lưới điện. Tự sản xuất năng lượng tái tạo và hứng nước mưa để hoàn toàn tự cung tự cấp và giảm phụ thuộc vào các nguồn tài nguyên không tái tạo.
>
> Đầu ra 4 ($\mathbf{y}_4$): Hãy ủng hộ nông sản địa phương để giảm lượng khí thải carbon do vận chuyển thực phẩm, đồng thời được thưởng thức đồ ăn tươi ngon, lành mạnh.

Sau đó ta yêu cầu những người gán nhãn đánh giá các đầu ra này. Một cách đơn giản là gán một điểm đánh giá cho từng đầu ra. Khi đó, bài toán học mô hình phần thưởng có thể được phát biểu như một nhiệm vụ huấn luyện mô hình hồi quy (*regression*). Nhưng việc cho điểm bằng số đối với các đầu ra của LLM không phải là nhiệm vụ dễ dàng cho người gán nhãn. Thường rất khó thiết kế một tiêu chuẩn gán nhãn mà mọi người gán nhãn đều đồng ý và dễ dàng tuân theo. Một phương pháp thay thế, phổ biến hơn trong việc phát triển LLM, là *xếp hạng* các đầu ra này. Ví dụ, một cách xếp hạng khả dĩ cho các đầu ra trên là

$$\mathbf{y}_1 \succ \mathbf{y}_4 \succ \mathbf{y}_2 \succ \mathbf{y}_3$$

Chú giải của người dịch: Ký hiệu $\succ$

$\mathbf{y}_a \succ \mathbf{y}_b$ đọc là "$\mathbf{y}_a$ được ưa thích hơn $\mathbf{y}_b$". Dãy trên nghĩa là người gán nhãn thấy đầu ra 1 tốt nhất, rồi đến 4, rồi 2, kém nhất là 3 (lời khuyên "sống tách khỏi lưới điện" khó thực hiện với đa số người). So sánh "cái nào hơn" dễ và nhất quán hơn nhiều so với việc cho điểm tuyệt đối (7 hay 8 điểm?), giống như giám khảo dễ chọn bài hay hơn trong hai bài văn hơn là chấm chính xác từng bài. Từ một xếp hạng 4 đầu ra, ta có thể rút ra $\binom{4}{2} = 6$ cặp so sánh để huấn luyện.

Sau đó, một mô hình phần thưởng được huấn luyện bằng kết quả xếp hạng này. Nói chung, mô hình phần thưởng trong RLHF là một mô hình ngôn ngữ có cùng kiến trúc với LLM đích, nhưng có kích thước nhỏ hơn. Cho đầu vào $\mathbf{x}$ và đầu ra $\mathbf{y}_k$, ta nối chúng lại để tạo thành chuỗi $\text{seq}_k = [\mathbf{x}, \mathbf{y}_k]$. Chuỗi này được xử lý từ trái sang phải bằng giải mã cưỡng bức (*forced decoding*). Vì trong mô hình hóa ngôn ngữ mỗi vị trí chỉ truy cập được ngữ cảnh bên trái của nó, đầu ra của tầng Transformer trên cùng tại vị trí đầu tiên không thể dùng làm biểu diễn của chuỗi. Thay vào đó, một ký hiệu đặc biệt (ví dụ $\langle/s\rangle$) được thêm vào cuối chuỗi, và đầu ra tương ứng của chồng các tầng Transformer được xem là biểu diễn của toàn bộ chuỗi. Một tầng đầu ra, chẳng hạn một tầng biến đổi tuyến tính, được xây dựng phía trên biểu diễn này để sinh ra phần thưởng, ký hiệu là $R(\text{seq}_k)$ hoặc $R(\mathbf{x}, \mathbf{y}_k)$.

Ta huấn luyện mô hình phần thưởng này bằng hàm mất mát xếp hạng (*ranking loss*). Chẳng hạn, một hàm mất mát xếp hạng theo cặp có thể

được viết dưới dạng

$$\text{Loss}_\omega(\mathcal{D}_r) = -\mathbb{E}_{(\mathbf{x},\mathbf{y}_{k_1},\mathbf{y}_{k_2})\sim\mathcal{D}_r} \log\big(\text{Sigmoid}(R_\omega(\mathbf{x},\mathbf{y}_{k_1}) - R_\omega(\mathbf{x},\mathbf{y}_{k_2}))\big), \tag{8.19}$$

trong đó ω biểu diễn các tham số của mô hình phần thưởng, và $\mathcal{D}_r$ biểu diễn một tập các bộ gồm một đầu vào và một cặp đầu ra. $(\mathbf{x},\mathbf{y}_{k_1},\mathbf{y}_{k_2}) \sim \mathcal{D}_r$ là một phép lấy mẫu, rút một mẫu $(\mathbf{x},\mathbf{y}_{k_1},\mathbf{y}_{k_2})$ từ $\mathcal{D}_r$ với một xác suất nào đó. Ví dụ, giả sử trước tiên ta rút một đầu vào mô hình $\mathbf{x}$ theo phân phối đều (*uniform distribution*), rồi rút một cặp đầu ra mô hình với xác suất $\mathbf{y}_{k_1} \succ \mathbf{y}_{k_2}$ khi biết $\mathbf{x}$ (ký hiệu là $\Pr(\mathbf{y}_{k_1} \succ \mathbf{y}_{k_2}|\mathbf{x})$). Hàm mất mát tương ứng là

$$\begin{aligned}
&\text{Loss}_\omega(\mathcal{D}_r)\\
&= -\sum \Pr(\mathbf{x}) \cdot \Pr(\mathbf{y}_{k_1} \succ \mathbf{y}_{k_2}|\mathbf{x}) \cdot\\
&\quad \log\big(\text{Sigmoid}(R_\omega(\mathbf{x},\mathbf{y}_{k_1}) - R_\omega(\mathbf{x},\mathbf{y}_{k_2}))\big)\\
&= -\frac{1}{K}\sum \Pr(\mathbf{y}_{k_1} \succ \mathbf{y}_{k_2}|\mathbf{x}) \cdot\\
&\quad \log\big(\text{Sigmoid}(R_\omega(\mathbf{x},\mathbf{y}_{k_1}) - R_\omega(\mathbf{x},\mathbf{y}_{k_2}))\big),
\end{aligned} \tag{8.20}$$

trong đó K biểu diễn số lượng đầu vào mô hình tham gia vào việc lấy mẫu. Dù dạng của các hàm này trông có vẻ phức tạp, ý tưởng của chúng rất đơn giản: ta phạt mô hình nếu thứ hạng dự đoán của hai đầu ra khác với thứ hạng do con người gán nhãn. Ngược lại, mô hình được "thưởng" nếu thứ hạng dự đoán khớp với thứ hạng do con người gán nhãn.

Chú giải của người dịch: Hiểu Phương trình (8.19) bằng số cụ thể

Ký hiệu $\mathbb{E}$ là *kỳ vọng*: hiểu nôm na là giá trị trung bình khi lấy mẫu nhiều lần. Hàm $\text{Sigmoid}(z) = \frac{1}{1+e^{-z}}$ biến mọi số thực thành một số trong $(0,1)$, có thể đọc là "xác suất mô hình tin rằng $\mathbf{y}_{k_1}$ tốt hơn $\mathbf{y}_{k_2}$". Giả sử con người nói $\mathbf{y}_{k_1}$ tốt hơn.

- Nếu mô hình chấm $R(\mathbf{x},\mathbf{y}_{k_1}) = 3$, $R(\mathbf{x},\mathbf{y}_{k_2}) = 1$: hiệu là 2, $\text{Sigmoid}(2) \approx 0.88$, mất mát $-\ln 0.88 \approx 0.13$ (nhỏ: mô hình đúng).
- Nếu chấm ngược lại, 1 và 3: hiệu là -2, $\text{Sigmoid}(-2) \approx 0.12$, mất mát $-\ln 0.12 \approx 2.13$ (lớn: mô hình bị phạt nặng).

Lưu ý rằng chỉ *hiệu* của hai điểm số là quan trọng, nên mô hình học cách cho

điểm sao cho câu trả lời được ưa thích luôn có điểm cao hơn.

Ta có thể huấn luyện mô hình phần thưởng bằng cách cực tiểu hóa hàm mất mát xếp hạng ở trên

$$\hat{\omega} \;=\; \arg\min_{\omega} \text{Loss}_{\omega}(\mathcal{D}_r). \tag{8.21}$$

Mô hình thu được $R_{\hat{\omega}}(\cdot)$ có thể được dùng để đánh giá bất kỳ cặp đầu vào và đầu ra nào. Lưu ý rằng mặc dù mô hình phần thưởng được huấn luyện bằng một mục tiêu dựa trên xếp hạng, nó lại được dùng để *chấm điểm*. Điều này cho phép nó cung cấp các tín hiệu giám sát liên tục, rất có lợi cho việc huấn luyện những mô hình khác.

Bây giờ ta chuyển sang bài toán học chính sách. Một mục tiêu thường được dùng là cực đại hóa phần thưởng trên một tập các cặp đầu vào–đầu ra. Theo một dạng tương tự Phương trình (8.16), ta có được một mục tiêu huấn luyện đơn giản cho việc tinh chỉnh bằng học tăng cường

$$\tilde{\theta} \;=\; \arg\max_{\hat{\theta}^+} \mathbb{E}_{(\mathbf{x},\mathbf{y}_{\hat{\theta}^+})\sim\mathcal{D}_{\text{rlft}}} R_{\hat{\omega}}(\mathbf{x},\mathbf{y}_{\hat{\theta}^+}), \tag{8.22}$$

trong đó các tham số tối ưu $\tilde{\theta}$ có được bằng cách tinh chỉnh các tham số đã tiền huấn luyện $\hat{\theta}$. $\mathcal{D}_{\text{rlft}}$ là tập dữ liệu tinh chỉnh bằng học tăng cường. Với mỗi mẫu $(\mathbf{x}, \mathbf{y}_{\hat{\theta}^+})$, $\mathbf{x}$ được lấy mẫu từ một tập dữ liệu chuỗi đầu vào đã chuẩn bị sẵn, còn $\mathbf{y}_{\hat{\theta}^+}$ được lấy mẫu từ phân phối $\Pr_{\hat{\theta}^+}(\mathbf{y}|\mathbf{x})$ do chính sách đưa ra.

Trong thực tế, các thuật toán học tăng cường tiên tiến hơn, chẳng hạn **tối ưu chính sách cận kề** (*proximal policy optimization*) (**PPO**), thường được dùng để huấn luyện ổn định hơn cũng như đạt hiệu năng tốt hơn. Chúng tôi để dành phần thảo luận chi tiết về các thuật toán học tăng cường cho những phần sau của cuốn sách, nơi RLHF được dùng rộng rãi cho việc căn chỉnh.

Một câu hỏi thú vị nảy sinh ở đây: tại sao không coi việc học từ sở thích của con người là một bài toán học có giám sát (*supervised learning*) tiêu chuẩn? Câu hỏi này liên quan mật thiết đến thảo luận ở trên về độ khó của việc gán nhãn dữ liệu. Thường thì việc mô tả các giá trị và mục tiêu của con người đã là thách thức, và việc con người tự viết ra những đầu ra được căn chỉnh tốt lại càng khó hơn. Thay vào đó, việc gán nhãn sở

thích cho một danh sách đầu ra mô hình cho trước là một nhiệm vụ đơn giản hơn. Bằng cách đó, ta có thể tạo ra một mô hình hiểu được sở thích của con người, rồi dùng nó làm mô hình phần thưởng để huấn luyện các chính sách. Từ góc độ học máy, RLHF đặc biệt hữu ích trong những tình huống mà hành vi mong muốn của tác tử khó *minh họa* nhưng lại dễ được con người *nhận ra*. Một ưu điểm khác của RLHF là khả năng khám phá không gian mẫu. Nhờ sử dụng các kỹ thuật lấy mẫu, các mô hình được huấn luyện bằng học tăng cường có thể vượt ra ngoài tập dữ liệu đã gán nhãn để khám phá thêm những mẫu khác. Khả năng khám phá này cho phép RLHF tìm ra những chính sách có thể có lợi mà không hiển nhiên nếu chỉ dựa vào dữ liệu có nhãn.

8.1.5 Prompt LLM

Cho đến giờ, ta đã thấy rằng LLM có thể được dùng để thực hiện nhiều nhiệm vụ khác nhau bằng cách cung cấp cho chúng các prompt phù hợp. Không có giới hạn nào đối với các prompt này; chúng có thể chứa bất kỳ thông tin nào ta muốn cung cấp cho LLM, chẳng hạn chỉ dẫn bằng ngôn ngữ tự nhiên và ngữ cảnh hội thoại. Vì cách tiếp cận này không đòi hỏi thêm việc huấn luyện hay tinh chỉnh nào, việc điều chỉnh LLM trở nên rất hiệu quả một khi chúng đã được phát triển. Điều này làm thay đổi căn bản mô thức của NLP: ta không còn cần phát triển các hệ thống riêng cho từng nhiệm vụ mà có thể dùng một LLM duy nhất, được huấn luyện tốt, để thực hiện các nhiệm vụ khác nhau chỉ bằng cách prompt nó. Từ đó, một khía cạnh hấp dẫn của LLM xuất hiện: người dùng có thể dễ dàng tạo ra các hệ thống tùy biến bằng cách tự thiết kế prompt cho LLM. Do vai trò quan trọng của prompting đối với LLM, **thiết kế prompt** (*prompt engineering*) đã trở thành một lĩnh vực nghiên cứu rất sôi động trong NLP.

Thuật ngữ *prompt* được dùng theo nhiều cách khác nhau trong các tài liệu. Trong chương này, thuật ngữ này chỉ *toàn bộ* đầu vào của LLM, và vì vậy ta dùng các thuật ngữ *prompt* và *đầu vào mô hình* thay thế cho nhau. Trước khi bàn thêm về prompting, hãy cùng xem một vài ví dụ trong đó prompt phức tạp hơn những prompt đã trình bày ở các tiểu mục trước. Lưu ý rằng tiểu mục này không nhằm hướng dẫn viết prompt chất lượng cao, mà nhằm làm nổi bật một số vấn đề thú vị khi prompt LLM.

Một trong những cách dùng LLM phổ biến là giao cho chúng một

"vai" khi sinh câu trả lời. Chẳng hạn, LLM có thể đóng vai một nhà tâm lý học khi trả lời câu hỏi.

Hãy giải thích trì hoãn sự thỏa mãn là gì.

Lưu ý, bạn là một nhà nghiên cứu có nền tảng sâu về tâm lý học. Lĩnh vực nghiên cứu của bạn là tâm lý học phát triển ở trẻ em, tập trung vào các khía cạnh tự kiểm soát và trí thông minh. Bạn rất am hiểu khái niệm trì hoãn sự thỏa mãn và đã áp dụng nó vào nghiên cứu của mình. Bạn đã công bố các bài báo về trì hoãn sự thỏa mãn và đã tiến hành các thí nghiệm liên quan.

Trì hoãn sự thỏa mãn là quá trình cưỡng lại một phần thưởng trước mắt để chờ nhận một phần thưởng giá trị hơn trong tương lai. Đây là một khái niệm quan trọng trong tâm lý học phát triển, đặc biệt liên quan đến cách trẻ em học tính tự chủ và phát triển các năng lực nhận thức. Nghiên cứu cho thấy khả năng trì hoãn sự thỏa mãn gắn liền với những kết quả tích cực về thành tích học tập, năng lực xã hội và sức khỏe cảm xúc. Các nghiên cứu và thí nghiệm trong lĩnh vực này nhấn mạnh tầm quan trọng của việc rèn luyện kỹ năng này từ sớm để hỗ trợ thành công lâu dài và khả năng phục hồi.

Một ví dụ khác là dùng LLM để phát hiện và sửa lỗi, chẳng hạn lỗi cú pháp hay lỗi ngữ nghĩa trong văn bản. Với một LLM được huấn luyện trên cả dữ liệu mã nguồn lẫn ngôn ngữ tự nhiên, ta có thể dùng nó để gỡ lỗi chương trình[6].

Hãy sửa các lỗi trong chương trình C này.

```
#include <stdio.h>
int main() {
   printg("Hello, World!")
   return 0;
}
```

1. Tên hàm 'printg' không đúng. Nó phải là 'printf'.
2. Thiếu dấu chấm phẩy ở cuối lời gọi hàm 'printf'.

Đây là phiên bản đã sửa của chương trình:

```
#include <stdio.h>
int main() {
    printf("Hello, World!");
    return 0;
```

[6]Trong ví dụ này, mã nguồn không được tách token để dễ đọc hơn.

```
}
```

Ngoài ra, LLM có thể được tinh chỉnh để đóng vai một người bạn trò chuyện[7].

Xin chào! Tôi là một trợ lý. Tôi có thể giúp gì cho bạn?
Đội nào đã vô địch FIFA World Cup 2022?
Argentina đã vô địch FIFA World Cup 2022.
Giải đấu được tổ chức ở đâu?
FIFA World Cup 2022 được tổ chức tại Qatar.
Argentina đã vô địch World Cup bao nhiêu lần?
Argentina đã vô địch FIFA World Cup ba lần.
…

Chú giải của người dịch: Hội thoại nhiều lượt thực chất là một prompt dài

Mô hình không "nhớ" cuộc trò chuyện như con người. Ở mỗi lượt, toàn bộ các câu hỏi và câu trả lời trước đó được ghép lại thành một prompt duy nhất, rồi mô hình viết tiếp phần trả lời. Vì vậy, khi được hỏi "Giải đấu được tổ chức ở đâu?", mô hình hiểu "giải đấu" là World Cup 2022 nhờ đọc thấy câu hỏi trước nằm ngay trong prompt. Cuộc trò chuyện càng dài thì prompt càng dài (đây chính là ý của chú thích chân trang 7).

Các ví dụ này và những ví dụ trước đã cho thấy rằng có thể sinh ra câu trả lời phù hợp thông qua các prompt chứa chỉ dẫn và câu hỏi rõ ràng. Tuy nhiên, khi việc giải quyết vấn đề đòi hỏi những tri thức không được nêu rõ, LLM có thể mắc lỗi, ngay cả khi chỉ dẫn rõ ràng và chính xác. Một nhóm nhiệm vụ đầy thách thức đối với LLM là lập luận số học (*arithmetic reasoning*) và lập luận thường thức (*commonsense reasoning*). Chẳng hạn, ta có thể yêu cầu một LLM giải các bài toán tiểu học được trình bày bằng ngôn ngữ tự nhiên.

[7] Để tinh chỉnh một LLM cho hội thoại nhiều lượt, cần xét đến lịch sử hội thoại trong ngữ cảnh khi dự đoán câu trả lời ở lượt hội thoại hiện tại. Điều này khiến prompt thực tế dùng để sinh câu trả lời tương đối dài hơn so với prompt dùng trong hội thoại một lượt.

Jack có 7 quả táo. Cậu ăn 2 quả vào bữa tối, nhưng sau đó mẹ cho cậu thêm 5 quả nữa. Ngày hôm sau, Jack cho bạn mình là John 3 quả. Cuối cùng Jack còn lại bao nhiêu quả táo?

Đáp án là 10.

Đáp án đúng phải là 7, vì vậy đầu ra của mô hình là sai.

Một cách giải quyết những vấn đề như vậy là đưa việc học vào trong prompt, được gọi là **học trong ngữ cảnh** (*in-context learning*) (**ICL**). Ý tưởng của ICL là minh họa cách giải bài toán ngay trong prompt, và để các dự đoán dựa vào những minh họa này. Dưới đây là một ví dụ trong đó một bài toán tương tự cùng với đáp án tương ứng được đưa vào prompt (màu xanh lá = các minh họa).

Tom có 12 viên bi. Cậu thắng thêm 7 viên trong một trò chơi với bạn nhưng ngày hôm sau lại thua mất 5 viên. Anh trai cậu tặng thêm cho cậu 3 viên bi làm quà. Bây giờ Tom có bao nhiêu viên bi?

Đáp án là 17.

Jack có 7 quả táo. Cậu ăn 2 quả vào bữa tối, nhưng sau đó mẹ cho cậu thêm 5 quả nữa. Ngày hôm sau, Jack cho bạn mình là John 3 quả. Cuối cùng Jack còn lại bao nhiêu quả táo?

Đáp án là 12.

Nhưng lần này LLM vẫn mắc lỗi. Một lý do có thể là việc giải toán không chỉ liên quan đến ánh xạ từ bài toán sang đáp án, mà ở mức độ lớn hơn còn liên quan đến quá trình suy diễn logic gồm nhiều bước ẩn bên dưới. Một phương pháp để cải thiện năng lực lập luận của LLM là **prompt chuỗi suy nghĩ** (*chain-of-thought prompting*) (**prompt CoT**) [Wei và cộng sự, 2022c]. Trong prompt CoT, ta phân rã các bài toán lập luận phức tạp thành nhiều bước trung gian để giải quyết vấn đề. Các bước này được minh họa trong prompt để LLM có thể được dẫn dắt học cách bắt chước quá trình tư duy ẩn bên dưới khi giải bài toán. Prompt CoT được nhận thấy là rất hữu ích để đạt kết quả tốt trên các nhiệm vụ khó, chẳng hạn bộ đánh giá chuẩn lập luận toán học GSM8K.

Hãy xét ví dụ minh họa ở trên: *Tom có 12 viên bi. Cậu thắng thêm 7 viên trong một trò chơi với bạn nhưng ngày hôm sau lại thua mất 5 viên. Anh trai cậu tặng thêm cho cậu 3 viên bi làm quà. Bây giờ Tom có bao*

nhiêu viên bi? Các bước lập luận logic tương ứng có thể được cấu trúc như sau:

Bước 1: *Ban đầu Tom có 12 viên bi.*

Bước 2: *Cậu thắng thêm 7 viên trong một trò chơi, nên ta cộng chúng vào số bi ban đầu: 12 viên + 7 viên = 19 viên.*

Bước 3: *Ngày hôm sau, Tom thua mất 5 viên, nên ta trừ chúng khỏi tổng hiện tại: 19 viên - 5 viên = 14 viên.*

Bước 4: *Anh trai cậu cho thêm 3 viên, nên ta cộng chúng vào tổng hiện tại: 14 viên + 3 viên = 17 viên.*

Bước 5: *Sau tất cả các sự việc này, Tom có 17 viên bi.*

Ta có thể thêm các bước lập luận này vào prompt, và nhận được đáp án đúng.

Tom có 12 viên bi. Cậu thắng thêm 7 viên trong một trò chơi với bạn nhưng ngày hôm sau lại thua mất 5 viên. Anh trai cậu tặng thêm cho cậu 3 viên bi làm quà. Bây giờ Tom có bao nhiêu viên bi?

Ban đầu Tom có 12 viên bi. Sau khi thắng một trò chơi, cậu có thêm 7 viên, nâng tổng số lên 19 viên. Ngày hôm sau cậu thua mất 5 viên, khiến tổng số giảm xuống còn 14 viên. Anh trai tặng cậu thêm 3 viên, nâng tổng số lên 17 viên. Vì vậy, bây giờ Tom có 17 viên bi. Vậy đáp án là 17.

Jack có 7 quả táo. Cậu ăn 2 quả vào bữa tối, nhưng sau đó mẹ cho cậu thêm 5 quả nữa. Ngày hôm sau, Jack cho bạn mình là John 3 quả. Cuối cùng Jack còn lại bao nhiêu quả táo?

Ban đầu Jack có 7 quả táo. Cậu ăn 2 quả vào bữa tối, nên ta lấy 7 trừ 2, còn lại 5 quả. Mẹ cho cậu thêm 5 quả, cộng 5 vào tổng hiện tại. Bây giờ cậu có 5 + 5 = 10 quả. Ngày hôm sau, Jack cho bạn mình là John 3 quả. Lấy 10 trừ 3, Jack còn lại 7 quả. Vì vậy, Jack còn lại 7 quả táo. Vậy đáp án là 7.

Cách tiếp cận này còn được gọi là **prompt CoT one-shot** (*one-shot CoT prompting*). Tương tự, ta có thể thêm nhiều minh họa hơn để có cách tiếp cận **prompt CoT few-shot** (*few-shot CoT prompting*).

Hơn nữa, một biến thể đặc biệt gọi là **CoT zero-shot** (*zero-shot CoT*) đưa vào các cụm từ kích hoạt nhằm khuyến khích LLM suy nghĩ từng bước, mà không cần cung cấp tường minh bất kỳ ví dụ lập luận trung gian

nào trong prompt [Kojima và cộng sự, 2022]. Điều thú vị là chỉ cần thêm câu "*Hãy suy nghĩ từng bước.*" (*Let's think step by step.*) vào cuối prompt thường đã đủ để khiến LLM sinh ra các bước lập luận bên trong và đi đến đáp án cuối cùng đúng.

Jack có 7 quả táo. Cậu ăn 2 quả vào bữa tối, nhưng sau đó mẹ cho cậu thêm 5 quả nữa. Ngày hôm sau, Jack cho bạn mình là John 3 quả. Cuối cùng Jack còn lại bao nhiêu quả táo?
Hãy suy nghĩ từng bước.

1. Số lượng ban đầu: Jack bắt đầu với 7 quả táo.
2. Sau bữa tối: Cậu ăn 2 quả, nên còn lại $7 - 2 = 5$ quả.
3. Mẹ cho thêm: Mẹ cho cậu thêm 5 quả, nên bây giờ cậu có $5 + 5 = 10$ quả.
4. Cho John: Ngày hôm sau, Jack cho bạn mình là John 3 quả, nên còn lại $10 - 3 = 7$ quả.

Cuối cùng, Jack còn lại 7 quả táo.

Chú giải của người dịch: Vì sao "suy nghĩ từng bước" lại giúp ích?

Mô hình sinh văn bản từng token một, và mỗi token mới được tính dựa trên mọi token đã viết trước đó. Khi buộc phải trả lời ngay một con số, mô hình phải "tính nhẩm" toàn bộ bài toán chỉ trong một bước dự đoán, nên dễ sai (như đáp án 10 hay 12 ở trên). Khi viết ra các bước trung gian ($7 - 2 = 5$, rồi $5 + 5 = 10$, rồi $10 - 3 = 7$), mỗi phép tính nhỏ đã nằm sẵn trong ngữ cảnh và trở thành "giấy nháp" cho bước sau — giống như khi giải toán, ta trình bày từng dòng thay vì chỉ ghi đáp số.

Học zero-shot, one-shot và few-shot là những khái niệm phổ biến trong lĩnh vực học trong ngữ cảnh của LLM và không chỉ giới hạn ở prompt CoT. Nói rộng ra, bất kỳ cách prompt nào chỉ gồm những chỉ dẫn đơn giản mà không có minh họa nào đều có thể được xem là một dạng học zero-shot. Năng lực học zero-shot này xuất hiện khi LLM được tiền huấn luyện và/hoặc tinh chỉnh. Ngoài ra, các phương pháp học one-shot và few-shot thường được cân nhắc hơn khi LLM không có được năng lực học zero-shot tương ứng. Vì vậy, các phương pháp này quan trọng đối với học trong ngữ cảnh khi giải quyết những nhiệm vụ mới. Ví dụ bao gồm những phương pháp thực hiện nhiều nhiệm vụ NLP khác nhau bằng cách minh họa các mẫu được trình bày theo định dạng của nhiệm vụ. Hãy xem các ví

dụ sau về phân loại cảm xúc của câu và dịch cụm từ bằng học few-shot.

Cho các đoạn văn bản sau, hãy phân loại cảm xúc của chúng là Tích cực, Tiêu cực hoặc Trung tính.

Ví dụ 1: "Tôi đã có một ngày tuyệt vời ở công viên!"
Cảm xúc: Tích cực

Ví dụ 2: "Dịch vụ ở nhà hàng thật tệ."
Cảm xúc: Tiêu cực

Ví dụ 3: "Tôi nghĩ hôm nay trời sẽ mưa."
Cảm xúc: Trung tính

Văn bản: "Bộ phim này là một chuyến du hành tuyệt vời qua trí tưởng tượng."
Cảm xúc: Tích cực

Hãy dịch các cụm từ tiếng Trung sau sang tiếng Anh.

Ví dụ 1: "你好"
Bản dịch: "Hello"

Ví dụ 2: "谢谢你"
Bản dịch: "Thank you"

Cụm từ cần dịch: "早上好"
Bản dịch: "Good Morning"

(Nghĩa: 你好 = "Xin chào", 谢谢你 = "Cảm ơn bạn", 早上好 = "Chào buổi sáng".)

Ở trên, chúng tôi đã trình bày các ví dụ để minh họa những năng lực học trong ngữ cảnh cơ bản khi prompt LLM. Tuy nhiên, mục này không bao gồm các kỹ thuật prompt nâng cao hơn để giữ nội dung ngắn gọn và súc tích. Có thể tìm thêm các thảo luận về prompting trong Chương 9.

8.2 Huấn luyện ở quy mô lớn

Bước đầu tiên trong việc phát triển LLM là huấn luyện các mô hình này trên một lượng dữ liệu lớn. Bản thân nhiệm vụ huấn luyện thì quen thuộc: mục tiêu là cực đại hóa log-hợp lý (*log-likelihood*), điều có thể đạt được bằng hạ gradient (*gradient descent*). Tuy nhiên, khi ta tăng cả kích thước mô hình lẫn lượng dữ liệu, bài toán tối ưu trở nên rất khó khăn. Chẳng hạn,

các mô hình lớn thường khiến quá trình huấn luyện kém ổn định. Trong mục này, chúng ta bàn về một số vấn đề vốn có của việc huấn luyện LLM ở quy mô lớn, bao gồm chuẩn bị dữ liệu, điều chỉnh mô hình và huấn luyện phân tán. Chúng ta cũng bàn về các định luật co giãn (*scaling laws*) của LLM, giúp ta hiểu hiệu suất và hiệu quả huấn luyện của chúng.

8.2.1 Chuẩn bị dữ liệu

Tầm quan trọng của dữ liệu trong NLP không bao giờ là quá lời. Khi các mạng nơ-ron ngày càng lớn hơn được phát triển, nhu cầu về dữ liệu cũng không ngừng tăng. Ví dụ, việc phát triển LLM có thể đòi hỏi hàng nghìn tỷ token để tiền huấn luyện (*pre-training*) (xem Bảng 8.3), lớn hơn nhiều bậc so với lượng dữ liệu dùng để huấn luyện các mô hình NLP truyền thống. Nói chung, ta có thể muốn thu thập càng nhiều dữ liệu huấn luyện càng tốt. Tuy nhiên, tập dữ liệu huấn luyện lớn hơn không có nghĩa là kết quả huấn luyện tốt hơn, và việc phát triển LLM đặt ra những vấn đề mới trong việc tạo lập hay thu thập các tập dữ liệu này.

Vấn đề thứ nhất là *chất lượng* dữ liệu. Dữ liệu chất lượng cao từ lâu đã được xem là yếu tố then chốt để huấn luyện các hệ thống NLP dựa trên dữ liệu. Việc đưa thẳng văn bản thô từ nhiều nguồn khác nhau vào mô hình nhìn chung là không nên. Chẳng hạn, một phần đáng kể dữ liệu dùng để huấn luyện các LLM gần đây có nguồn gốc từ việc thu thập dữ liệu web tự động (*web scraping*), vốn thường chứa lỗi và nội dung không phù hợp, như ngôn ngữ độc hại và các sự kiện bịa đặt. Ngoài ra, internet ngày càng tràn ngập nội dung do máy sinh ra vì AI được sử dụng rộng rãi, gây thêm khó khăn cho việc lọc các kho ngữ liệu (*corpora*) thu thập từ web. Các nhà nghiên cứu đã chứng minh rằng huấn luyện LLM trên dữ liệu chưa được lọc có thể gây hại [Raffel và cộng sự, 2020]. Việc cải thiện chất lượng dữ liệu thường bao gồm việc tích hợp các bước lọc và làm sạch nghiêm ngặt vào quy trình xử lý dữ liệu. Ví dụ, Penedo và cộng sự [2023] cho thấy rằng bằng cách áp dụng một loạt kỹ thuật xử lý dựa trên quy tắc kinh nghiệm (*heuristic*), có tới 90% dữ liệu thu thập từ web của họ có thể bị loại bỏ trước khi huấn luyện LLM. Bên cạnh văn bản thu thập từ web ở quy mô lớn, dữ liệu huấn luyện LLM thường bao gồm sách, bài báo khoa học, nội dung do người dùng tạo ra trên mạng xã hội, và nhiều kho ngữ liệu đa dạng khác. Phần lớn các LLM mới nhất được huấn luyện trên những tập

dữ liệu kết hợp như vậy, và điều này được nhận thấy là quan trọng đối với hiệu năng mạnh mẽ của các mô hình thu được.

Bảng 8.3: Số token tiền huấn luyện được công bố và các loại dữ liệu chính của một số LLM tiêu biểu. T biểu thị một nghìn tỷ (*trillion*) token. Số token phụ thuộc vào bộ tách token (*tokenizer*) và có thể bao gồm cả việc lấy mẫu lặp lại dữ liệu huấn luyện.

LLM	Số token	Dữ liệu
GPT-3-175B [Brown và cộng sự, 2020]	0.3T	Trang web, sách, Wikipedia
Falcon-180B [Almazrouei và cộng sự, 2023]	3.5T	Trang web, sách, hội thoại, mã nguồn, bài viết kỹ thuật
PaLM-540B [Chowdhery và cộng sự, 2022]	0.78T	Trang web, sách, hội thoại, mã nguồn, Wikipedia, tin tức
Gemma-7B [Gemma Team, 2024]	6T	Trang web, toán học, mã nguồn
Llama3.1-405B [Dubey và cộng sự, 2024]	15.6T	Văn bản tổng quát, toán học, dữ liệu lập luận, mã nguồn, văn bản đa ngôn ngữ
DeepSeek-V3 [Liu và cộng sự, 2024a]	14.8T	Văn bản đa ngôn ngữ, toán học, mã nguồn
Qwen3-235B-A22B [Yang và cộng sự, 2025]	36T	Văn bản đa ngôn ngữ, sách, mã nguồn, dữ liệu lập luận, dữ liệu tổng hợp

Chú giải của người dịch: Một nghìn tỷ token lớn cỡ nào?

Một token xấp xỉ một từ hoặc một mảnh từ. Một cuốn tiểu thuyết dày khoảng 100 000 từ, tức cỡ 10^5 token. Vậy $15.6\text{T} = 1.56 \times 10^{13}$ token của Llama3.1 tương đương khoảng $1.56 \times 10^{13}/10^5 \approx 1.5 \times 10^8$, tức hơn 150 triệu cuốn tiểu thuyết! Một người đọc mỗi ngày một cuốn cũng phải mất hơn 400 000 năm mới đọc hết. Đó là lý do dữ liệu phải lấy chủ yếu từ web và phải được lọc bằng máy.

Vấn đề thứ hai là *tính đa dạng* của dữ liệu. Ta muốn dữ liệu huấn luyện bao phủ càng nhiều loại dữ liệu càng tốt, để các mô hình được huấn luyện có thể dễ dàng thích nghi với nhiều nhiệm vụ hạ nguồn (*downstream tasks*) khác nhau. Người ta đã thừa nhận rộng rãi rằng cả chất lượng lẫn độ đa dạng của dữ liệu huấn luyện đều đóng vai trò rất quan trọng đối với LLM. Một ví dụ thú vị là việc đưa mã nguồn lập trình vào dữ liệu huấn

luyện được nhận thấy là có lợi cho LLM. Lợi ích này không chỉ thể hiện ở việc nâng cao khả năng lập trình của LLM, mà còn ở việc cải thiện khả năng lập luận (*reasoning*) cho các bài toán phức tạp, đặc biệt là những bài toán đòi hỏi kỹ thuật prompt theo chuỗi suy nghĩ (*chain of thought, CoT*). Khái niệm "đa dạng" cũng có thể được mở rộng để bao gồm cả sự đa dạng về ngôn ngữ. Ví dụ, nhiều LLM được huấn luyện trên dữ liệu đa ngôn ngữ, và nhờ đó ta có thể xử lý nhiều ngôn ngữ chỉ bằng một mô hình duy nhất. Mặc dù cách tiếp cận này cho thấy năng lực mạnh mẽ trong các nhiệm vụ đa ngôn ngữ và xuyên ngôn ngữ (*cross-lingual*), hiệu năng của nó trên từng ngôn ngữ cụ thể phụ thuộc phần lớn vào khối lượng và chất lượng dữ liệu của ngôn ngữ đó. Trong một số trường hợp, nó đã được chứng minh là cho kết quả kém đối với các ngôn ngữ ít tài nguyên (*low-resource languages*).

Vấn đề thứ ba là *thiên lệch* (*bias*) trong dữ liệu huấn luyện. Đây không phải là vấn đề riêng của LLM mà tồn tại trong nhiều hệ thống NLP. Một ví dụ phổ biến là thiên lệch giới tính (*gender bias*), khi LLM thể hiện những ưu tiên hay định kiến không có cơ sở. Điều này một phần có thể quy cho sự mất cân bằng lớp (*class imbalance*) trong các kho ngữ liệu huấn luyện. Ví dụ, từ *nurse* (y tá) gắn với phụ nữ một cách không cân xứng trong các văn bản lịch sử. Để khử thiên lệch trong dữ liệu, một cách làm phổ biến là cân bằng tỷ lệ đại diện của các nhóm nhân khẩu học khác nhau, chẳng hạn giới tính, dân tộc và phương ngữ. Thiên lệch dữ liệu gắn liền về bản chất với vấn đề đa dạng đã nêu ở trên. Vì nhiều LLM được huấn luyện và căn chỉnh (*aligned*) bằng dữ liệu lấy tiếng Anh làm trung tâm, chúng vốn nghiêng về các giá trị văn hóa và quan điểm phổ biến trong cộng đồng nói tiếng Anh. Tăng cường sự đa dạng về ngôn ngữ và văn hóa trong dữ liệu huấn luyện có thể phần nào giảm bớt những thiên lệch này.

Một vấn đề khác trong việc thu thập dữ liệu quy mô lớn là *quyền riêng tư*. Nếu LLM được huấn luyện trên dữ liệu từ những nguồn rộng khắp, điều này có thể dẫn tới rủi ro làm lộ thông tin nhạy cảm, như sở hữu trí tuệ và dữ liệu cá nhân. Điều này đặc biệt đáng lo ngại khi xét đến khả năng của LLM trong việc biểu diễn các khuôn mẫu từ dữ liệu mà chúng được huấn luyện, việc có thể vô tình kéo theo sự ghi nhớ và tái tạo lại những chi tiết cụ thể. Một cách đơn giản để bảo vệ quyền riêng tư là xóa bỏ hoặc ẩn danh hóa (*anonymize*) thông tin nhạy cảm. Ví dụ, có thể áp dụng các kỹ thuật ẩn danh hóa để loại bỏ thông tin định danh cá nhân khỏi dữ liệu huấn

luyện, nhằm ngăn LLM học từ những dữ liệu đó. Tuy nhiên, trong thực tế, việc xóa hay che hết mọi dữ liệu nhạy cảm là rất khó. Vì vậy, nhiều LLM, đặc biệt là những mô hình được đưa ra phục vụ công chúng, thường hoạt động cùng các hệ thống có khả năng phát hiện nguy cơ lộ dữ liệu nhạy cảm, hoặc được tinh chỉnh để từ chối một số yêu cầu có thể dẫn đến rò rỉ thông tin.

8.2.2 Điều chỉnh mô hình

Huấn luyện LLM là việc khó. Một vấn đề thường gặp là quá trình tối ưu ngày càng trở nên kém ổn định khi kích thước mô hình tăng lên. Chẳng hạn, ta cần chọn một tốc độ học (*learning rate*) nhỏ để huấn luyện ổn định bằng hạ gradient, nhưng điều này lại làm tăng mạnh thời gian huấn luyện. Đôi khi, ngay cả khi cấu hình huấn luyện đã được thiết kế cẩn thận, quá trình huấn luyện vẫn có thể phân kỳ (*diverge*) tại một số thời điểm trong quá trình tối ưu. Việc huấn luyện LLM nhìn chung chịu ảnh hưởng của nhiều yếu tố, như cách khởi tạo tham số, cách chia lô và chính quy hóa. Ở đây, chúng ta tập trung vào những điều chỉnh và cải tiến phổ biến đối với kiến trúc Transformer chuẩn, vốn quan trọng cho việc huấn luyện LLM ổn định.

Chú giải của người dịch: "Phân kỳ" trong huấn luyện

Hạ gradient giống như người đi xuống thung lũng trong sương mù: mỗi bước, ta dò độ dốc tại chỗ rồi bước một bước theo hướng đi xuống; *tốc độ học* là độ dài bước chân. Bước quá dài thì có thể nhảy vọt sang sườn bên kia và cao hơn chỗ cũ; lặp lại vài lần như vậy, giá trị hàm mất mát tăng vọt (thậm chí thành vô cùng). Hiện tượng này gọi là huấn luyện *phân kỳ*. Bước quá ngắn thì an toàn nhưng phải đi rất nhiều bước, tức tốn rất nhiều thời gian. Mô hình càng lớn thì "địa hình" càng gồ ghề, nên càng dễ phân kỳ.

1. Chuẩn hóa tầng với kết nối tắt

Chuẩn hóa tầng (*layer normalization*) được dùng để ổn định quá trình huấn luyện các mạng nơ-ron sâu. Đó là quá trình trừ đi giá trị trung bình và chia cho độ lệch chuẩn. Bằng cách chuẩn hóa đầu ra của tầng theo cách này, ta có thể giảm thiểu hiệu quả hiện tượng dịch chuyển hiệp biến nội tại (*internal covariate shift*) và tăng cường độ ổn định khi huấn luyện.

Trong Transformer, chuẩn hóa tầng thường được dùng cùng với kết nối tắt (*residual connections*). Như đã mô tả ở Mục 8.1.1, một tầng con có thể dựa trên kiến trúc chuẩn hóa sau (*post-norm*), trong đó chuẩn hóa tầng được thực hiện ngay sau một khối residual, hoặc kiến trúc chuẩn hóa trước (*pre-norm*), trong đó chuẩn hóa tầng được thực hiện bên trong khối residual. Mặc dù cả hai kiến trúc đều được dùng rộng rãi trong các hệ thống dựa trên Transformer [Wang và cộng sự, 2019a], kiến trúc chuẩn hóa trước đã chứng tỏ là đặc biệt hữu ích khi huấn luyện các Transformer sâu. Vì thế, phần lớn LLM dựa trên kiến trúc chuẩn hóa trước, được biểu diễn là output $= F(\text{LNorm}(\text{input})) + \text{input}$.

Một dạng hàm chuẩn hóa tầng được dùng rộng rãi là

$$\text{LNorm}(\mathbf{h}) \;=\; \boldsymbol{\alpha} \odot \frac{\mathbf{h} - \mu}{\sigma + \epsilon} + \boldsymbol{\beta}, \tag{8.23}$$

trong đó $\mathbf{h}$ là một vectơ thực d chiều, μ là trung bình của tất cả các phần tử của $\mathbf{h}$, và σ là độ lệch chuẩn tương ứng. ϵ được đưa vào để đảm bảo ổn định số (*numerical stability*). $\boldsymbol{\alpha} \in \mathbb{R}^d$ và $\boldsymbol{\beta} \in \mathbb{R}^d$ lần lượt là các số hạng hệ số khuếch đại (*gain*) và độ lệch (*bias*).

Một biến thể của chuẩn hóa tầng, gọi là chuẩn hóa tầng theo căn trung bình bình phương (*root mean square (RMS) layer normalization*), chỉ co giãn lại vectơ đầu vào mà không dời tâm của nó [Zhang và Sennrich, 2019]. Hàm chuẩn hóa tầng RMS được cho bởi

$$\text{RMSNorm}(\mathbf{h}) \;=\; \boldsymbol{\alpha} \odot \frac{\mathbf{h}}{\sigma_{\text{rms}} + \epsilon}, \tag{8.24}$$

trong đó σ_{rms} là căn trung bình bình phương của $\mathbf{h}$, tức là $\sigma_{\text{rms}} = (\frac{1}{d}\sum_{k=1}^{d} h_k^2)^{\frac{1}{2}}$. Hàm chuẩn hóa tầng này được dùng trong các LLM như dòng Llama.

Chú giải của người dịch: Tính tay LNorm và RMSNorm

Lấy $\mathbf{h} = [1 \;\; 2 \;\; 3 \;\; 6]$ ($d = 4$), bỏ qua ϵ và cho $\boldsymbol{\alpha} = [1\ 1\ 1\ 1]$, $\boldsymbol{\beta} = [0\ 0\ 0\ 0]$.

- **LNorm:** trung bình $\mu = (1+2+3+6)/4 = 3$; phương sai $= ((-2)^2+(-1)^2+0^2+3^2)/4 = 14/4 = 3.5$, nên $\sigma = \sqrt{3.5} \approx 1.87$. Kết quả $\approx [-1.07 \;\; -0.53 \;\; 0 \;\; 1.60]$: các số được "kéo về" quanh 0 với độ phân tán cỡ 1.
- **RMSNorm:** $\sigma_{\text{rms}} = \sqrt{(1+4+9+36)/4} = \sqrt{12.5} \approx 3.54$. Kết

quả $\approx [0.28 \;\; 0.57 \;\; 0.85 \;\; 1.70]$: chỉ chia cho một số, không trừ trung bình, nên các số vẫn dương.

Ký hiệu $\odot$ là nhân *theo từng phần tử*: $[a_1 \;\; a_2] \odot [b_1 \;\; b_2] = [a_1 b_1 \quad a_2 b_2]$. RMSNorm bỏ bớt bước tính trung bình nên rẻ hơn một chút, mà trong thực tế hiệu quả gần như tương đương.

2. Hàm kích hoạt trong FFN

Trong Transformer, các tầng con FFN được thiết kế để đưa tính phi tuyến vào việc học biểu diễn (*representation learning*), và đã được nhận thấy là hữu ích trong việc ngăn các biểu diễn học được bởi cơ chế tự chú ý (*self-attention*) bị suy biến (*degeneration*)[8] [Dong và cộng sự, 2021]. Một dạng chuẩn của FFN dùng trong các tầng con này có thể được biểu diễn là

$$\text{FFN}(\mathbf{h}) \;=\; \sigma(\mathbf{h}\mathbf{W}_h + \mathbf{b}_h)\mathbf{W}_f + \mathbf{b}_f, \tag{8.25}$$

trong đó $\mathbf{W}_h \in \mathbb{R}^{d \times d_h}$, $\mathbf{b}_h \in \mathbb{R}^{d_h}$, $\mathbf{W}_f \in \mathbb{R}^{d_h \times d}$ và $\mathbf{b}_f \in \mathbb{R}^d$ là các tham số, còn d_h là kích thước ẩn (*hidden size*). Hàm $\sigma(\cdot)$ là hàm kích hoạt (*activation function*) của tầng ẩn. Một lựa chọn phổ biến cho $\sigma(\cdot)$ là **đơn vị tuyến tính chỉnh lưu** (*rectified linear unit, ReLU*), cho bởi

$$\sigma_{\text{relu}}(\mathbf{h}) \;=\; \max(\mathbf{0}, \mathbf{h}). \tag{8.26}$$

Trong các cài đặt thực tế, tăng d_h là có lợi, và vì vậy trong LLM giá trị này thường được đặt lớn hơn d rất nhiều. Tuy nhiên, kích thước ẩn quá lớn gây khó khăn cho cả việc huấn luyện lẫn triển khai. Trong những trường hợp như vậy, việc chọn hàm kích hoạt đóng vai trò quan trọng hơn trong các FFN rộng.

Một số lựa chọn thay thế hiệu quả cho ReLU đã được đề xuất trong các LLM hiện đại. Một trong những hàm được dùng rộng rãi nhất là **đơn vị tuyến tính sai số Gauss** (*gaussian error linear unit, GeLU*), có thể được xem là một xấp xỉ trơn của ReLU. Thay vì áp dụng một ngưỡng cứng dựa trên dấu của đầu vào, hàm GeLU đánh trọng số cho mỗi đầu vào bằng hàm phân phối tích lũy (*cumulative distribution function*) của phân phối chuẩn tắc (*standard normal distribution*). Cụ thể, hàm GeLU được định

[8] Ở đây, suy biến chỉ hiện tượng hạng của ma trận biểu diễn bị giảm đáng kể sau khi xử lý.

nghĩa theo từng phần tử là

$$\sigma_{\text{gelu}}(\mathbf{h}) \;=\; \mathbf{h} \odot \Phi(\mathbf{h}), \tag{8.27}$$

trong đó $\Phi(\cdot)$ ký hiệu hàm phân phối tích lũy của phân phối chuẩn tắc $\mathcal{N}(\mathbf{0}, \mathbf{1})$. Hàm GeLU đã được dùng trong một số LLM, như BERT, GPT-3 và BLOOM.

Chú giải của người dịch: ReLU và GeLU bằng số

ReLU giữ nguyên số dương và biến số âm thành 0: $\max(0,\, 2) \;=\; 2$, $\max(0,\, -1.5) = 0$. Đồ thị của nó có một "góc gãy" tại 0. $\Phi(x)$ là xác suất để một đại lượng ngẫu nhiên có phân phối chuẩn tắc (hình chuông, tâm 0, độ lệch chuẩn 1) nhận giá trị $\leq x$; nó tăng mượt từ 0 đến 1, với $\Phi(0) = 0.5$, $\Phi(1) \approx 0.84$, $\Phi(-1) \approx 0.16$. Do đó GeLU cho $1 \cdot 0.84 = 0.84$ tại $x = 1$, $-1 \cdot 0.16 = -0.16$ tại $x = -1$, và gần như bằng x khi x lớn, gần 0 khi x rất âm. GeLU vì vậy giống ReLU nhưng "bo tròn" chỗ góc gãy.

Một họ hàm kích hoạt phổ biến khác là họ **đơn vị tuyến tính có cổng** (*gated linear unit, GLU*). Dạng cơ bản của GLU là

$$\sigma_{\text{glu}}(\mathbf{h}) \;=\; \sigma(\mathbf{h}\mathbf{W}_1 + \mathbf{b}_1) \odot (\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2), \tag{8.28}$$

trong đó $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d \times d_h}$ và $\mathbf{b}_1, \mathbf{b}_2 \in \mathbb{R}^{d_h}$ là các tham số mô hình. Lưu ý rằng cơ chế GLU thực chất thay thế bước chiếu tuyến tính đơn lẻ cùng bước kích hoạt trong một FFN chuẩn, ánh xạ đầu vào $\mathbf{h}$ trực tiếp sang chiều ẩn mở rộng d_h. Những lựa chọn khác nhau cho hàm kích hoạt nền $\sigma(\cdot)$ cho ra các biến thể khác nhau của GLU. Chẳng hạn, nếu $\sigma(\cdot)$ được định nghĩa là hàm GELU, ta thu được hàm GeGLU

$$\sigma_{\text{geglu}}(\mathbf{h}) \;=\; \sigma_{\text{gelu}}(\mathbf{h}\mathbf{W}_1 + \mathbf{b}_1) \odot (\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2). \tag{8.29}$$

Hàm kích hoạt này đã được áp dụng thành công trong các LLM như Gemma.

Ngoài ra, nếu ta định nghĩa $\sigma(\cdot)$ là hàm Swish, $\sigma_{\text{swish}}(\mathbf{h}) \;=\; \mathbf{h} \odot \text{Sigmoid}(c\mathbf{h})$ [Ramachandran và cộng sự, 2017], ta thu được hàm SwiGLU

$$\sigma_{\text{swiglu}}(\mathbf{h}) \;=\; \sigma_{\text{swish}}(\mathbf{h}\mathbf{W}_1 + \mathbf{b}_1) \odot (\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2). \tag{8.30}$$

Cả hai dòng mô hình PaLM và Llama đều dựa vào hàm SwiGLU cho các tầng FFN của chúng. Để có thảo luận đầy đủ hơn về các biến thể GLU

và tác động của chúng, bạn đọc có thể tham khảo công trình của Shazeer [2020].

Chú giải của người dịch: "Cổng" trong GLU nghĩa là gì?

Hãy hình dung hai nhánh song song cùng nhận $\mathbf{h}$. Nhánh thứ hai $\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2$ tạo ra "thông tin"; nhánh thứ nhất qua hàm kích hoạt σ tạo ra các số đóng vai trò "van" (với Sigmoid thì nằm trong khoảng $(0, 1)$). Phép $\odot$ nhân từng thông tin với van tương ứng: van gần 0 thì chặn, gần 1 thì cho qua. Ví dụ, thông tin $[4 \quad -2]$ và van $[0.9 \quad 0.1]$ cho ra $[3.6 \quad -0.2]$: thành phần đầu gần như được giữ, thành phần sau gần như bị chặn. Mô hình *tự học* nên mở/đóng van nào tùy theo đầu vào.

3. Loại bỏ các số hạng độ lệch

Một thiết kế mô hình phổ biến khác là loại bỏ các số hạng độ lệch khỏi các phép biến đổi affine (*affine transformations*) dùng trong LLM. Sự đơn giản hóa này có thể được áp dụng cho chuẩn hóa tầng, các phép chiếu đầu vào cho cơ chế chú ý QKV, và các tầng con FFN. Ví dụ, ta có thể sửa Phương trình (8.25) để có một FFN không có số hạng độ lệch

$$\text{FFN}(\mathbf{h}) \;=\; \sigma(\mathbf{h}\mathbf{W}_h)\mathbf{W}_f. \tag{8.31}$$

Chowdhery và cộng sự [2022] báo cáo rằng việc loại bỏ các số hạng độ lệch giúp cải thiện độ ổn định khi huấn luyện LLM. Phương pháp này đã được dùng trong một số LLM, như Llama và Gemma.

4. Các vấn đề khác

Nhiều LLM cũng có những điều chỉnh đối với mô hình nhúng vị trí. Ví dụ, ta có thể thay mã hóa vị trí hình sin (*sinusoidal positional encodings*) bằng nhúng vị trí xoay (*rotary position embeddings*). Các mô hình này sẽ được bàn ở Mục 8.3.

Cần lưu ý rằng mặc dù việc điều chỉnh mô hình là phổ biến trong phát triển LLM, độ ổn định khi huấn luyện có thể được cải thiện theo nhiều cách khác nhau. Chẳng hạn, việc tăng kích thước lô (*batch size*) khi quá trình huấn luyện tiến triển đã được nhận thấy là hữu ích với một số LLM. Nhìn chung, để huấn luyện LLM quy mô lớn một cách ổn định và hiệu quả, cần có những thiết lập được thiết kế cẩn thận, như lịch điều chỉnh tốc độ học (*learning rate scheduling*), lựa chọn bộ tối ưu (*optimizer*), chiến

lược huấn luyện song song và các phương pháp huấn luyện độ chính xác hỗn hợp (*mixed-precision training*). Một số khía cạnh trong đó đòi hỏi rất nhiều công sức kỹ thuật, và vì vậy ta thường cần chạy huấn luyện nhiều lần mới có được LLM vừa ý.

8.2.3 Huấn luyện phân tán

Huấn luyện LLM đòi hỏi tài nguyên tính toán rất lớn. Một cách phổ biến để nâng cao hiệu suất huấn luyện là sử dụng các hệ thống phân tán quy mô lớn. May mắn là cùng với sự trỗi dậy của mạng nơ-ron trong AI, phần mềm và phần cứng cho học sâu cũng đã được phát triển, giúp việc cài đặt LLM và thực hiện tính toán trở nên dễ dàng hơn. Ví dụ, ngày nay người ta có thể dễ dàng tinh chỉnh một LLM bằng các khung phần mềm (*software frameworks*) học sâu và một máy tính có nhiều GPU. Tuy nhiên, việc mở rộng quy mô huấn luyện LLM vẫn còn nhiều thách thức, và đòi hỏi nỗ lực đáng kể để phát triển các hệ thống phần cứng và phần mềm cho huấn luyện phân tán ổn định và hiệu quả.

Một yếu tố quan trọng cần xem xét trong huấn luyện phân tán là tính song song (*parallelism*). Có một số dạng song song: song song dữ liệu, song song mô hình, song song tensor và song song đường ống. Mặc dù có nhiều cách khác nhau để phân bổ tính toán lên các thiết bị, các phương pháp song song này đều dựa trên một ý tưởng chung: bài toán huấn luyện có thể được chia thành những nhiệm vụ nhỏ hơn và thực hiện đồng thời. Vấn đề song song trong huấn luyện LLM đã được nghiên cứu sâu rộng [Narayanan và cộng sự, 2021; Fedus và cộng sự, 2022b]. Ở đây ta phác thảo các khái niệm cơ bản.

- **Song song dữ liệu (*Data Parallelism*)**. Đây là một trong những phương pháp song song được dùng rộng rãi nhất để huấn luyện mạng nơ-ron. Để minh họa, xét trường hợp đơn giản nhất, khi quy tắc delta (*delta rule*) chuẩn được dùng trong hạ gradient

$$\theta_{t+1} \;=\; \theta_t - \eta \cdot \frac{\partial L_{\theta_t}(\mathcal{D}_{\text{mini}})}{\partial \theta_t}, \tag{8.32}$$

  trong đó các tham số mới θ_{t+1} thu được bằng cách cập nhật các tham số gần nhất θ_t một bước nhỏ η theo hướng ngược với gradient của hàm mất mát. $\frac{\partial L_{\theta_t}(\mathcal{D}_{\text{mini}})}{\partial \theta_t}$ là gradient của hàm mất mát theo các

tham số θ_t, và được tính trên một lô nhỏ (*minibatch*) các mẫu huấn luyện $\mathcal{D}_{\text{mini}}$. Trong song song dữ liệu, ta chia $\mathcal{D}_{\text{mini}}$ thành N lô nhỏ hơn, ký hiệu là $\{\mathcal{D}^1, \cdots, \mathcal{D}^N\}$. Sau đó, ta phân phối các lô này cho N máy thực thi (*workers*), mỗi máy nhận một lô tương ứng. Khi dữ liệu đã được phân phối, các máy thực thi này có thể làm việc cùng lúc. Gradient của toàn bộ lô nhỏ thu được bằng cách tổng hợp các gradient do các máy thực thi tính ra, như sau

$$\frac{\partial L_{\theta_t}(\mathcal{D}_{\text{mini}})}{\partial \theta_t} = \underbrace{\frac{\partial L_{\theta_t}(\mathcal{D}^1)}{\partial \theta_t}}_{\text{máy 1}} + \underbrace{\frac{\partial L_{\theta_t}(\mathcal{D}^2)}{\partial \theta_t}}_{\text{máy 2}} + \cdots + \underbrace{\frac{\partial L_{\theta_t}(\mathcal{D}^N)}{\partial \theta_t}}_{\text{máy } N}. \tag{8.33}$$

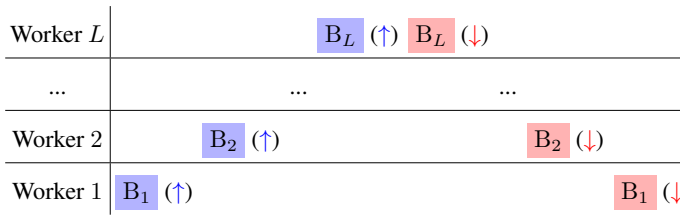
Trong điều kiện lý tưởng, khi các máy thực thi phối hợp tốt và chi phí truyền thông (*communication overhead*) nhỏ, song song dữ liệu có thể giúp tăng tốc huấn luyện gần N lần.

Chú giải của người dịch: Vì sao có thể cộng các gradient?

Hàm mất mát trên một lô là *tổng* mất mát của từng mẫu trong lô, mà đạo hàm của một tổng bằng tổng các đạo hàm (như ở lớp 11: $(f+g)' = f' + g'$). Vậy nếu lô có 1 000 câu, ta có thể giao 250 câu cho mỗi máy trong 4 máy; mỗi máy tính đạo hàm trên phần của mình, rồi cộng 4 kết quả lại là ra đúng đạo hàm trên cả 1 000 câu. Giống như bốn bạn chia nhau chấm 1 000 bài kiểm tra rồi cộng tổng điểm: kết quả không đổi, chỉ nhanh gấp gần 4 lần. (Nếu mất mát định nghĩa là *trung bình* thay vì tổng thì chỉ cần chia thêm cho một hằng số.)

- **Song song mô hình (*Model Parallelism*)**. Mặc dù song song dữ liệu đơn giản và hiệu quả, nó đòi hỏi mỗi máy thực thi phải chạy toàn bộ LLM và thực hiện trọn vẹn lượt truyền xuôi và lượt truyền ngược. Khi LLM ngày càng lớn, đôi khi không thể nạp và chạy một LLM trên một thiết bị duy nhất. Khi đó, ta có thể tách LLM thành các thành phần nhỏ hơn và chạy chúng trên các thiết bị khác nhau. Một cách đơn giản để làm việc này là gom các tầng liên tiếp trong chồng tầng thành nhóm và giao mỗi nhóm cho một máy thực thi, gọi là song song mô hình đơn giản (*naive model parallelism*). Các máy thực thi hoạt động tuần tự theo thứ tự các tầng trong chồng, nghĩa là trong lượt truyền xuôi ta xử lý đầu vào từ các tầng thấp lên các tầng cao, còn trong lượt truyền ngược ta lan truyền gradient

của lỗi từ các tầng cao xuống các tầng thấp. Chẳng hạn, xét một bộ giải mã Transformer gồm L khối xếp chồng. Để phân bổ khối lượng tính toán, mỗi khối được giao cho một máy thực thi. Hình minh họa dưới đây cho thấy một lần chạy lượt truyền xuôi và lượt truyền ngược của mô hình này.



Ở đây B_l ký hiệu phép tính của khối l, còn các ký hiệu ↑ và ↓ lần lượt chỉ lượt truyền xuôi và lượt truyền ngược (*Worker l* là máy thực thi thứ l). Lưu ý rằng phương pháp song song này buộc các máy thực thi chạy tuần tự, nên một máy phải đợi máy trước nó hoàn thành công việc. Điều này khiến các thiết bị phải nằm chờ phần lớn thời gian. Trong các hệ thống thực tế, song song mô hình thường được dùng kết hợp với các cơ chế song song khác để tận dụng tối đa các thiết bị.

- **Song song tensor (*Tensor Parallelism*)**. Song song hóa cũng có thể được thực hiện bên trong một bước tính toán duy nhất. Một cách phổ biến là chia một ma trận tham số lớn thành các khối, nhân tensor đầu vào với từng khối một cách riêng rẽ, rồi ghép nối (*concatenate*) kết quả của các phép nhân này để tạo thành đầu ra. Ví dụ, xét phép nhân biểu diễn $\mathbf{h} \in \mathbb{R}^d$ với ma trận tham số $\mathbf{W}_h \in \mathbb{R}^{d \times d_h}$ trong một tầng con FFN (xem Phương trình (8.25)). Ta có thể cắt ma trận $\mathbf{W}_h \in \mathbb{R}^{d \times d_h}$ theo chiều dọc thành M ma trận con

$$\mathbf{W}_h = \begin{bmatrix} \mathbf{W}_h^1 & \mathbf{W}_h^2 & \cdots & \mathbf{W}_h^M \end{bmatrix}, \tag{8.34}$$

trong đó mỗi ma trận con $\mathbf{W}_h^k$ có kích thước $d \times \frac{d_h}{M}$. Phép nhân $\mathbf{h}$ với $\mathbf{W}_h$ có thể được viết là

$$\mathbf{h}\mathbf{W}_h = \mathbf{h} \begin{bmatrix} \mathbf{W}_h^1 & \mathbf{W}_h^2 & \cdots & \mathbf{W}_h^M \end{bmatrix}$$

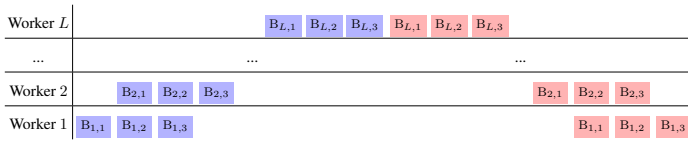
$$= \begin{bmatrix} \mathbf{h}\mathbf{W}_h^1 & \mathbf{h}\mathbf{W}_h^2 & \cdots & \mathbf{h}\mathbf{W}_h^M \end{bmatrix}. \tag{8.35}$$

Ta có thể thực hiện các phép nhân ma trận $\{\mathbf{h}\mathbf{W}_h^1, \mathbf{h}\mathbf{W}_h^2, \cdots, \mathbf{h}\mathbf{W}_h^M\}$ riêng rẽ trên M thiết bị. Nhờ đó, ta phân bổ một phép nhân ma trận lớn lên nhiều thiết bị, mỗi thiết bị có thể chỉ có bộ nhớ tương đối nhỏ. Từ góc độ thiết kế GPU hiện đại, song song tensor trên các GPU mang lại một cách tiếp cận hai cấp, dựa trên chia ô (*tile-based*), cho tính toán song song. Trước hết, ở cấp cao hơn, ta phân rã một phép nhân ma trận thành các phép nhân ma trận con vừa khít bộ nhớ của GPU. Sau đó, ở cấp thấp hơn, ta thực hiện các phép nhân ma trận con này trên GPU bằng các thuật toán song song chia ô được tối ưu riêng cho GPU.

Chú giải của người dịch: Ví dụ nhỏ về cắt ma trận theo cột

Nhân vectơ hàng $\mathbf{h} = [1 \quad 2]$ với ma trận 2×4: mỗi phần tử kết quả chỉ phụ thuộc vào *một cột* của ma trận (phần tử thứ j = h_1·(số hàng 1, cột j) + h_2·(số hàng 2, cột j)). Vì vậy, nếu cắt ma trận thành hai nửa trái (cột 1–2) và phải (cột 3–4), máy A nhân $\mathbf{h}$ với nửa trái được 2 số đầu, máy B nhân với nửa phải được 2 số cuối; đặt cạnh nhau là ra đúng kết quả đầy đủ. Mỗi máy chỉ cần giữ một nửa ma trận trong bộ nhớ.

- **Song song đường ống (*Pipeline Parallelism*)**. Ở phần song song mô hình bên trên, chúng ta đã mô tả một cách đơn giản để phân bổ các nhóm thành phần của mô hình lên nhiều thiết bị. Nhưng phương pháp này kém hiệu quả vì trong quá trình xử lý, tại mỗi thời điểm chỉ có một thiết bị hoạt động. Song song đường ống giải quyết vấn đề này bằng cách cho các phép tính trên các thiết bị khác nhau chồng lấp lên nhau [Harlap và cộng sự, 2018; Huang và cộng sự, 2019]. Để làm vậy, một lô mẫu được chia thành nhiều lô vi mô (*micro-batches*), rồi các lô vi mô này được mỗi máy thực thi xử lý như bình thường. Ngay khi một lô vi mô được một máy xử lý xong và chuyển sang máy kế tiếp, lô vi mô tiếp theo lập tức thế chỗ trên chính máy đó. Nói cách khác, ta tạo ra một đường ống, trong đó các bước tính toán khác nhau có thể chồng lấp nếu nhiều công việc được đưa vào đường ống. Hình dưới đây minh họa song song đường ống khi xử lý 3 lô vi mô.



Ở đây $B_{l,k}$ biểu diễn việc máy thực thi thứ l xử lý lô vi mô thứ k. Lý tưởng nhất, ta muốn tăng tối đa số lô vi mô, và nhờ đó giảm tối thiểu thời gian nằm chờ của các máy thực thi. Tuy nhiên, trong thực tế, dùng các lô vi mô nhỏ thường làm giảm mức độ tận dụng GPU và tăng chi phí chuyển đổi tác vụ. Điều này đến lượt nó có thể làm giảm thông lượng (*throughput*) tổng thể của hệ thống.

Chú giải của người dịch: Đường ống giống dây chuyền lắp ráp

Hình dung một xưởng làm bánh có 3 công đoạn: nhào bột, nướng, đóng gói, mỗi công đoạn mất 1 giờ. Nếu làm từng mẻ một cách tuần tự (song song mô hình đơn giản), 3 mẻ mất $3 \times 3 = 9$ giờ và mỗi lúc chỉ có một người làm việc. Theo dây chuyền: khi mẻ 1 chuyển sang lò nướng thì mẻ 2 bắt đầu nhào bột ngay, … Tổng thời gian chỉ còn $3+2 = 5$ giờ. Càng chia nhiều mẻ nhỏ thì thời gian chờ ở đầu và cuối dây chuyền càng ít đáng kể, nhưng mẻ quá nhỏ thì mỗi lần chuyển mẻ lại tốn công, đúng như sách nói về lô vi mô.

Mục tiêu tối thượng của xử lý song song là đạt được mức tăng hiệu suất tuyến tính, nghĩa là số mẫu có thể được xử lý trong một đơn vị thời gian tăng tuyến tính theo số thiết bị. Tuy nhiên, huấn luyện phân tán rất phức tạp và chịu ảnh hưởng của nhiều yếu tố ngoài phương pháp song song mà ta chọn. Một vấn đề thường gắn với các hệ thống phân tán là chi phí truyền thông. Ta có thể xem một hệ thống phân tán như một nhóm các nút (*nodes*) được kết nối mạng với nhau. Mỗi nút có thể thực hiện tính toán cục bộ hoặc truyền dữ liệu cho các nút khác. Nếu số nút như vậy rất lớn, việc phân phát và thu gom dữ liệu giữa chúng sẽ rất tốn kém. Đôi khi, thời gian tiết kiệm được nhờ song song hóa lại bị triệt tiêu bởi chi phí truyền thông của một mạng lớn. Một vấn đề khác với các hệ thống phân tán quy mô lớn là việc đồng bộ hóa (*synchronization*) các nút gây thêm chi phí. Như thường lệ, một số nút có thể mất nhiều thời gian hơn để hoàn

thành nhiệm vụ, khiến các nút khác phải đợi những nút chậm nhất. Mặc dù có thể dùng huấn luyện bất đồng bộ (*asynchronous training*) để xử lý sự không đồng đều về tài nguyên tính toán, điều này có thể dẫn tới gradient lỗi thời (*stale gradients*) và không đảm bảo hội tụ. Hơn nữa, khi càng có thêm nhiều nút trong mạng, khả năng có nút bị sập trong quá trình huấn luyện càng cao. Khi đó, ta cần bảo đảm toàn bộ hệ thống có khả năng chịu lỗi (*fault-tolerant*). Trong nhiều tình huống thực tế, để tăng khả năng mở rộng (*scalability*), người ta cần tính đến thêm nhiều vấn đề, bao gồm thiết kế kiến trúc, việc cho truyền dữ liệu chồng lấp với tính toán, cân bằng tải (*load balancing*), băng thông bộ nhớ (*memory bandwidth*), v.v.

Huấn luyện LLM tốn kém về tính toán đến mức, dù đã dùng huấn luyện phân tán, các nhà nghiên cứu và kỹ sư vẫn thường áp dụng nhiều phương pháp nén mô hình (*model compression*) và tăng tốc khác nhau để nâng cao hiệu suất huấn luyện [Weng, 2021]. Một ví dụ là huấn luyện độ chính xác hỗn hợp, trong đó dữ liệu độ chính xác thấp (như dữ liệu FP16 và FP8) được dùng để tính gradient trên mỗi nút, còn dữ liệu độ chính xác cao hơn (ví dụ FP32/FP64) được dùng để cập nhật mô hình [Micikevicius và cộng sự, 2018]. Một thao tác then chốt trong cách tiếp cận này là tích lũy gradient (*gradient accumulation*), trong đó các gradient cần được cộng dồn và đồng bộ giữa các nút. Tuy nhiên, do phép cộng số dấu phẩy động không có tính kết hợp, điều này có thể dẫn tới những sai khác số học nhỏ trong các gradient tích lũy trên các nút khác nhau, có thể ảnh hưởng tới sự hội tụ của mô hình và hiệu năng cuối cùng. Vấn đề này càng rõ rệt khi có nhiều nút tham gia huấn luyện phân tán, đặc biệt khi các phép tính số học độ chính xác thấp có thể gặp các vấn đề tràn trên (*overflow*) và tràn dưới (*underflow*), cũng như sự không nhất quán giữa các thiết bị phần cứng khác nhau. Do đó, việc thiết kế hệ thống phân tán cần xét đến các vấn đề tính toán số này để bảo đảm kết quả và sự hội tụ thỏa đáng.

Chú giải của người dịch: Phép cộng máy tính không có tính kết hợp

Máy tính lưu số thực với một số lượng chữ số có nghĩa giới hạn (FP16 chỉ khoảng 3–4 chữ số thập phân, FP32 khoảng 7). Giả sử máy chỉ giữ 3 chữ số có nghĩa. Khi đó $(1000 + 0.4) + 0.4$: bước đầu 1000.4 bị làm tròn thành 1000, rồi $1000 + 0.4$ lại thành 1000. Nhưng $1000 + (0.4 + 0.4) = 1000 + 0.8 = 1000.8 \to 1001$. Hai cách nhóm cho hai kết quả khác nhau! Vì vậy, khi các

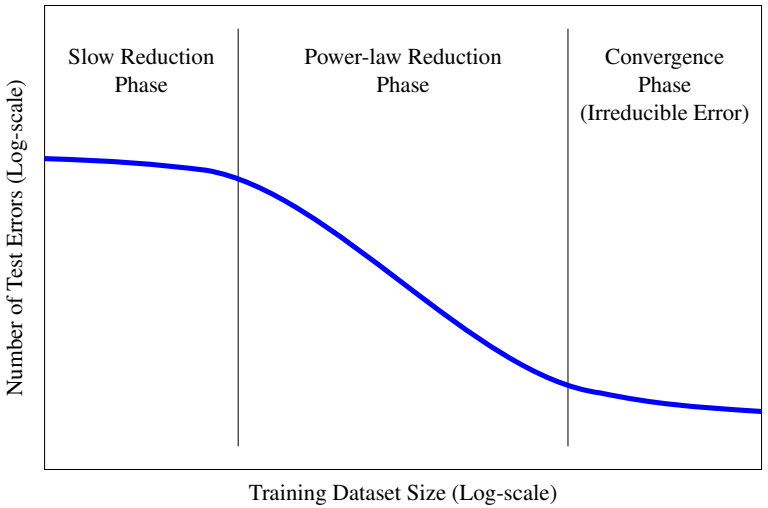
nút cộng gradient theo những thứ tự khác nhau, kết quả có thể lệch nhau chút ít. "Tràn trên" là khi số quá lớn vượt khả năng biểu diễn (FP16 tối đa khoảng 65 504), "tràn dưới" là khi số quá nhỏ bị làm tròn thành 0.

8.2.4 Định luật co giãn

Thành công của LLM cho thấy việc huấn luyện các mô hình ngôn ngữ lớn hơn với nhiều tài nguyên hơn có thể giúp cải thiện hiệu năng mô hình. Các nhà nghiên cứu đã mô tả hiện tượng này là **định luật co giãn** (*scaling laws*) của LLM. Cụ thể hơn, định luật co giãn mô tả mối quan hệ giữa hiệu năng của LLM và các yếu tố tham gia vào quá trình huấn luyện, như kích thước mô hình, lượng tính toán dùng để huấn luyện và lượng dữ liệu huấn luyện. Chẳng hạn, Hestness và cộng sự [2017] chỉ ra rằng hiệu năng của các mạng nơ-ron sâu là một hàm lũy thừa (*power-law function*) của kích thước dữ liệu huấn luyện. Ban đầu, khi lượng dữ liệu huấn luyện còn tương đối ít, hiệu năng của mô hình được cải thiện chậm. Sau đó, khi dùng thêm nhiều dữ liệu huấn luyện, mô hình bước vào giai đoạn hiệu năng tăng nhanh, và đường cong hiệu năng đi theo xu hướng hàm lũy thừa. Cuối cùng, mức cải thiện hiệu năng lại trở nên chậm, và thêm dữ liệu không đem lại lợi ích đáng kể. Hình 8.3 cho thấy một ví dụ về những đường cong như vậy.

Trong NLP, một quan điểm truyền thống cho rằng lợi ích về hiệu năng sẽ biến mất tại một thời điểm nào đó khi quy mô huấn luyện tăng lên. Tuy nhiên, các kết quả gần đây cho thấy rằng nếu xét vấn đề ở quy mô lớn hơn, việc mở rộng quy mô huấn luyện vẫn là một phương pháp rất hiệu quả để có được những LLM mạnh hơn. Ví dụ, cả LLM mã nguồn đóng lẫn mã nguồn mở đều có thể hưởng lợi từ việc có thêm dữ liệu, dù đã dùng tới hàng nghìn tỷ token để huấn luyện.

Khi quy mô huấn luyện mô hình tăng lên, LLM thể hiện những năng lực mới, được gọi là **năng lực đột sinh** (*emergent abilities*) của LLM. Chẳng hạn, Wei và cộng sự [2022b] đã nghiên cứu các tính chất co giãn của LLM với những kích thước mô hình và lượng tài nguyên tính toán khác nhau. Công trình của họ cho thấy một số năng lực xuất hiện khi ta tăng kích thước mô hình tới một mức nhất định. Việc quan sát thấy các năng lực đột sinh đã chứng minh vai trò của huấn luyện quy mô lớn trong việc nâng cao hiệu năng của LLM, và ở một mức độ nào đó, cũng thôi



Hình 8.3: Minh họa định luật co giãn của lỗi kiểm tra theo một biến được quan tâm (ví dụ: kích thước tập dữ liệu huấn luyện) [Hestness và cộng sự, 2017]. Đường cong của định luật co giãn có thể được chia thành ba giai đoạn. Ban đầu, lỗi kiểm tra giảm chậm khi dùng thêm dữ liệu huấn luyện, nhưng giai đoạn này chỉ kéo dài trong thời gian ngắn. Ở giai đoạn thứ hai, lỗi kiểm tra giảm mạnh, và đường cong trở thành một đường cong lũy thừa. Sau đó, mức giảm lỗi lại chậm dần ở giai đoạn thứ ba. Lưu ý rằng có những lỗi không thể khử được, bất kể lượng dữ liệu huấn luyện là bao nhiêu. (Nhãn trong hình: *Slow Reduction Phase* – giai đoạn giảm chậm; *Power-law Reduction Phase* – giai đoạn giảm theo hàm lũy thừa; *Convergence Phase (Irreducible Error)* – giai đoạn hội tụ (lỗi không thể khử); trục tung: số lỗi trên tập kiểm tra (thang log); trục hoành: kích thước tập dữ liệu huấn luyện (thang log).)

thúc các nhà nghiên cứu không ngừng thử huấn luyện các mô hình lớn hơn. Khi các LM lớn hơn và mạnh hơn tiếp tục xuất hiện, hiểu biết của chúng ta về định luật co giãn ngày càng chín muồi. Hiểu biết này giúp các nhà nghiên cứu dự đoán hiệu năng của LLM trong quá trình huấn luyện và ước tính lượng tài nguyên tính toán cần thiết để đạt được một mức hiệu năng cho trước.

Để hiểu hiệu năng mô hình thay đổi thế nào theo các yếu tố khác nhau trong huấn luyện, người ta thường biểu diễn hiệu năng mô hình như một hàm của các yếu tố đó. Ví dụ, trong trường hợp đơn giản nhất, ta có thể biểu diễn mất mát hoặc lỗi của một LLM như một hàm của một biến được quan tâm duy nhất. Tuy nhiên, không có định luật co giãn phổ quát nào mô tả được mối quan hệ này. Thay vào đó, người ta đề xuất những hàm khác nhau để khớp với đường cong học (*learning curves*) của LLM.

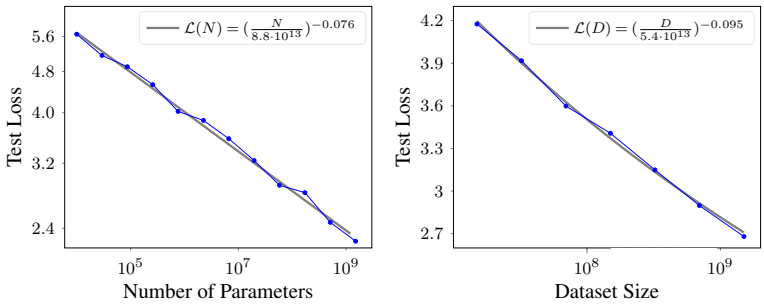
Gọi x là biến được quan tâm (chẳng hạn số tham số của mô hình) và $\mathcal{L}(x)$ là mất mát của mô hình ứng với x (chẳng hạn mất mát entropy chéo trên dữ liệu kiểm tra). Dạng đơn giản nhất của $\mathcal{L}(x)$ là một luật lũy thừa (*power law*)

$$\mathcal{L}(x) = ax^b, \tag{8.36}$$

trong đó a và b là các tham số được ước lượng bằng thực nghiệm. Dù đơn giản, hàm này đã giải thích thành công hành vi co giãn của các mô hình ngôn ngữ và các hệ thống dịch máy theo kích thước mô hình (ký hiệu là N) và kích thước tập dữ liệu huấn luyện (ký hiệu là D) [Gordon và cộng sự, 2021; Hestness và cộng sự, 2017]. Ví dụ, Kaplan và cộng sự [2020] nhận thấy rằng, sau một giai đoạn chuyển tiếp ban đầu, hiệu năng mô hình ngôn ngữ của họ được cải thiện theo luật lũy thừa của N hoặc của D, và biểu diễn các mối quan hệ này bằng $\mathcal{L}(N) = \left(\frac{N}{8.8\times10^{13}}\right)^{-0.076}$ và $\mathcal{L}(D) = \left(\frac{D}{5.4\times10^{13}}\right)^{-0.095}$ (xem Hình 8.4).

Chú giải của người dịch: Luật lũy thừa và thang logarit

Lấy logarit hai vế của $\mathcal{L}(x) = ax^b$: $\log \mathcal{L} = \log a + b \log x$. Đây là một hàm *bậc nhất* theo $\log x$, nên khi vẽ trên hai trục thang log (như Hình 8.4) ta được một *đường thẳng* có hệ số góc b (âm, vì mất mát giảm). Với $\mathcal{L}(N) = (N/(8.8 \times 10^{13}))^{-0.076}$: tăng số tham số lên 10 lần thì mất mát nhân với $10^{-0.076} \approx 0.84$, tức giảm khoảng 16%. Muốn giảm thêm 16% nữa lại phải tăng mô hình thêm 10 lần – lợi ích có thật nhưng ngày càng đắt.



Hình 8.4: Mất mát trên tập kiểm tra (*Test Loss*) theo kích thước mô hình (N, *Number of Parameters* – số tham số) và kích thước tập dữ liệu huấn luyện (D, *Dataset Size*) (các điểm dữ liệu được vẽ chỉ nhằm minh họa). Ta vẽ mất mát kiểm tra như một hàm của N, được định nghĩa là $\mathcal{L}(N) = \left(\frac{N}{8.8 \times 10^{13}}\right)^{-0.076}$, và như một hàm của D, được định nghĩa là $\mathcal{L}(D) = \left(\frac{D}{5.4 \times 10^{13}}\right)^{-0.095}$ [Kaplan và cộng sự, 2020].

Một cải tiến cho định luật co giãn này là thêm một số hạng **lỗi không thể khử** (*irreducible error*) vào luật lũy thừa. Khi đó dạng của $\mathcal{L}(x)$ là

$$\mathcal{L}(x) \;=\; ax^b + \epsilon_\infty, \tag{8.37}$$

trong đó ϵ_∞ là lỗi không thể khử, đại diện cho phần lỗi do các biến chưa biết gây ra, vẫn tồn tại ngay cả khi $x \to \infty$. Phương trình (8.37) là một trong những dạng được dùng rộng rãi nhất để mô hình hóa định luật co giãn của LLM. Ví dụ, Rosenfeld và cộng sự [2020] đã xây dựng một định luật co giãn bao gồm cả việc co giãn mô hình lẫn co giãn tập dữ liệu, như sau

$$\mathcal{L}(N, D) \;=\; aN^b + cD^d + \epsilon_\infty. \tag{8.38}$$

Một ví dụ cho dạng công thức này là định luật co giãn Chinchilla (*Chinchilla scaling law*). Định luật này phát biểu rằng mất mát kiểm tra trên mỗi token bằng tổng của các hàm lũy thừa nghịch của N và D, cộng thêm một số hạng lỗi không thể khử. Hoffmann và cộng sự [2022] biểu diễn định luật co giãn này là

$$\mathcal{L}(N, D) \;=\; \underbrace{\frac{406.4}{N^{0.34}}}_{\text{co giãn mô hình}} + \underbrace{\frac{410.7}{D^{0.28}}}_{\text{co giãn dữ liệu}} + \underbrace{1.69}_{\text{lỗi không thể khử}}. \tag{8.39}$$

Chú giải của người dịch: Thử số với định luật Chinchilla

Lấy mô hình $N = 10^{10}$ tham số (10 tỷ) huấn luyện trên $D = 2 \times 10^{11}$ token (200 tỷ). Ta có $N^{0.34} = 10^{3.4} \approx 2512$ nên số hạng đầu $\approx 406.4/2512 \approx 0.16$; $D^{0.28} = 10^{0.28 \times 11.3} \approx 10^{3.16} \approx 1445$ nên số hạng thứ hai $\approx 410.7/1445 \approx 0.28$. Tổng mất mát $\approx 0.16 + 0.28 + 1.69 = 2.13$. Dù N và D lớn đến đâu, mất mát cũng không thể xuống dưới 1.69: đó là phần "không thể đoán trước" vốn có của ngôn ngữ (cùng một ngữ cảnh, người viết thật vẫn có thể viết tiếp theo nhiều cách khác nhau).

Tất cả các định luật co giãn kể trên đều dựa trên các hàm đơn điệu. Vì vậy chúng không thể bao quát những hàm có điểm uốn (*inflection points*), chẳng hạn các đường cong suy giảm kép (*double descent*). Để giải quyết điều này, các nhà nghiên cứu đã khám phá những hàm phức tạp hơn để khớp với đường cong học. Có thể tìm thấy ví dụ về những hàm như vậy trong công trình của Alabdulmohsin và cộng sự [2022] và Caballero và cộng sự [2023].

Ý nghĩa của định luật co giãn nằm ở chỗ nó đưa ra định hướng cho nghiên cứu LLM: nếu ta vẫn còn ở vùng đường cong lũy thừa, việc dùng nhiều tài nguyên hơn để huấn luyện các mô hình lớn hơn là một hướng đi rất hứa hẹn. Dù kết quả này buộc các nhóm nghiên cứu và công ty lớn đầu tư mạnh vào tài nguyên tính toán để huấn luyện các mô hình lớn hơn – điều rất tốn kém – định luật co giãn vẫn liên tục đẩy xa giới hạn của AI. Mặt khác, hiểu biết về định luật co giãn giúp các nhà nghiên cứu đưa ra quyết định có cơ sở khi huấn luyện LLM. Ví dụ, với lượng tài nguyên tính toán sẵn có, ta có thể ước tính được hiệu năng cuối cùng của một LLM.

Một lưu ý cuối cùng về định luật co giãn trong mục này: đối với LLM, mất mát kiểm tra thấp hơn không phải lúc nào cũng có nghĩa là hiệu năng tốt hơn trên mọi nhiệm vụ hạ nguồn. Để thích nghi LLM, có một số bước như tinh chỉnh và kỹ thuật prompt có thể ảnh hưởng đến kết quả cuối cùng. Do đó, trong thực tế, định luật co giãn cho các nhiệm vụ hạ nguồn khác nhau có thể khác nhau.

8.3 Mô hình hóa chuỗi dài

Chúng ta đã thấy rằng trong huấn luyện quy mô lớn, có thể phát triển các mô hình ngôn ngữ mạnh hơn bằng cách tăng quy mô cả dữ liệu lẫn tài nguyên tính toán. Tuy nhiên, việc co giãn cũng có thể diễn ra theo những chiều khác. Chẳng hạn, trong nhiều ứng dụng hiện đại, LLM thường cần được điều chỉnh để xử lý các chuỗi rất dài. Một tình huống phổ biến là tiền huấn luyện LLM trên các văn bản có độ dài thông thường, rồi sau đó áp dụng nó cho những chuỗi dài hơn rất nhiều so với những gì đã thấy khi tiền huấn luyện. Ở đây, ta dùng $\Pr(\mathbf{y}|\mathbf{x})$ để ký hiệu xác suất có điều kiện của việc sinh văn bản, trong đó $\mathbf{x}$ là ngữ cảnh và $\mathbf{y}$ là văn bản được sinh ra. Nói chung, các bài toán mô hình hóa chuỗi dài có thể được chia thành ba loại:

- **Sinh văn bản với ngữ cảnh dài** (*Long-context text generation*) (tức $\mathbf{x}$ là một chuỗi dài). Ví dụ, sinh một bản tóm tắt ngắn cho một tài liệu dài.

- **Sinh văn bản dài** (*Long-text generation*) (tức $\mathbf{y}$ là một chuỗi dài). Ví dụ, sinh một câu chuyện hoàn chỉnh từ vài từ khóa.

- **Sinh văn bản dài với ngữ cảnh dài** (*Long-context and long-text generation*) (tức cả $\mathbf{x}$ và $\mathbf{y}$ đều là chuỗi dài). Ví dụ, dịch một tài liệu dài từ tiếng Trung sang tiếng Anh.

Gần đây, các nhà nghiên cứu NLP ngày càng quan tâm đến việc áp dụng và đánh giá LLM trên các nhiệm vụ có văn bản đầu vào cực kỳ dài. Hãy tưởng tượng một LLM đọc một tệp mã nguồn C++ gồm hàng chục nghìn dòng, rồi phác thảo chức năng của chương trình ứng với tệp đó. Những mô hình có khả năng xử lý ngữ cảnh văn bản lớn như vậy thường được gọi là **LLM ngữ cảnh dài** (*long-context LLMs*). Trong mục này, chúng ta sẽ giới hạn ở LLM ngữ cảnh dài, nhưng các phương pháp được bàn ở đây cũng có thể áp dụng cho các bài toán khác.

Với Transformer, việc xử lý chuỗi dài rất tốn kém về tính toán, vì chi phí tính toán của cơ chế tự chú ý tăng theo bình phương độ dài chuỗi. Điều này khiến việc huấn luyện và triển khai các mô hình như vậy cho đầu vào rất dài là không khả thi. Có hai hướng nghiên cứu đã cố gắng điều chỉnh Transformer cho mô hình hóa ngôn ngữ ngữ cảnh dài.

- Hướng thứ nhất tìm tòi các phương pháp huấn luyện và kiến trúc mô hình hiệu quả để học các mô hình tự chú ý từ dữ liệu chuỗi dài.
- Hướng thứ hai điều chỉnh các LLM đã được tiền huấn luyện để xử lý chuỗi dài mà chỉ cần tinh chỉnh ít hoặc không cần tinh chỉnh.

Ở đây, chúng ta sẽ bàn ngắn gọn về hướng thứ nhất vì Chương 6 đã trình bày kỹ nhiều phương pháp thuộc hướng này. Chúng ta sẽ tập trung vào hướng thứ hai, nhấn mạnh các phương pháp phổ biến trong các LLM gần đây. Chúng ta cũng sẽ bàn về điểm mạnh và hạn chế của các mô hình chuỗi dài này.

Chú giải của người dịch: "Tăng theo bình phương" nghĩa là gì?

Trong tự chú ý, mỗi token phải "nhìn" vào mọi token khác, nên với chuỗi m token ta cần cỡ $m \times m = m^2$ điểm chú ý. Chuỗi 1 000 token cần 10^6 điểm; chuỗi 100 000 token (dài gấp 100 lần) cần 10^{10} điểm – gấp $100^2 = 10\,000$ lần. Giống như số cái bắt tay khi mọi người trong phòng đều bắt tay nhau: số người tăng gấp đôi thì số cái bắt tay tăng gần gấp bốn.

8.3.1 Tối ưu hóa phần cứng và tính toán song song

Chúng ta bắt đầu bằng việc xem xét các cải tiến cho mô hình Transformer chuẩn từ góc độ tính toán hiệu năng cao (*high-performance computing*). Phần lớn các cải tiến này, dù không được thiết kế riêng cho LLM, đã được áp dụng rộng rãi cho nhiều mô hình học sâu khác nhau [Kim và cộng sự, 2023]. Một cách tiếp cận thường dùng là sử dụng các định dạng dữ liệu độ chính xác giảm. Ví dụ, thay vì dùng phép tính dấu phẩy động (*floating-point*) 32 bit chuẩn (FP32), các mô hình có thể dùng số dấu phẩy động 16 bit (FP16/BF16), hay thậm chí lượng tử hóa (*quantization*) thành số nguyên 8 bit (INT8). Dùng các kiểu dữ liệu độ chính xác thấp này có thể cải thiện hiệu suất tính toán và thông lượng bộ nhớ, để có thể xử lý các chuỗi dài hơn dễ dàng hơn. Một cách tiếp cận khác là cải tiến Transformer bằng các kỹ thuật nhận biết phần cứng (*hardware-aware*). Ví dụ, trên các GPU hiện đại, hiệu suất của Transformer có thể được cải thiện bằng các cài đặt hàm tự chú ý có nhận biết vào/ra (*IO-aware*) [Dao và cộng sự, 2022; Kwon và cộng sự, 2023].

Một cách khác để xử lý chuỗi dài là thông qua song song chuỗi (*sequence parallelism*) [Li và cộng sự, 2023b; Korthikanti và cộng sự, 2023]. Cụ thể, xét bài toán tổng quát: cho truy vấn $\mathbf{q}_i$ tại vị trí i chú ý tới các khóa $\mathbf{K}$ và các giá trị $\mathbf{V}$. Ta có thể chia $\mathbf{K}$ theo chiều chuỗi (tức theo hàng) và thu được một tập ma trận con $\{\mathbf{K}^{[1]}, \cdots, \mathbf{K}^{[n_u]}\}$, mỗi ma trận ứng với một đoạn của chuỗi. Tương tự, ta có thể thu được các ma trận con của $\mathbf{V}$, ký hiệu là $\{\mathbf{V}^{[1]}, \cdots, \mathbf{V}^{[n_u]}\}$. Sau đó, ta giao mỗi cặp $\mathbf{K}^{[u]}$ và $\mathbf{V}^{[u]}$ cho một nút tính toán (*compute node*) (ví dụ một GPU trong một cụm GPU). Các nút được giao việc có thể chạy song song, nhờ đó song song hóa phép toán chú ý.

Nhắc lại rằng đầu ra của mô hình tự chú ý có thể được viết là

$$\text{Att}_{\text{qkv}}(\mathbf{q}_i, \mathbf{K}, \mathbf{V}) \ = \ \sum_{j=0}^{m-1} \alpha_{i,j}\mathbf{v}_j, \tag{8.40}$$

trong đó $\alpha_{i,j}$ là trọng số chú ý giữa vị trí i và j. Trong Transformer, $\alpha_{i,j}$ thu được bằng cách chuẩn hóa tích vô hướng đã được co giãn giữa $\mathbf{q}_i$ và $\mathbf{k}_j$. Gọi $\beta_{i,j}$ là điểm chú ý giữa $\mathbf{q}_i$ và $\mathbf{k}_j$. Ta có

$$\beta_{i,j} \ = \ \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}} + \text{Mask}(i,j), \tag{8.41}$$

trong đó $\text{Mask}(i,j)$ là giá trị mặt nạ nhân quả (*causal mask*) cho cặp vị trí (i,j). Khi đó, ta định nghĩa trọng số chú ý $\alpha_{i,j}$ là

$$\begin{aligned} \alpha_{i,j} &= \text{Softmax}(\beta_{i,j}) \\ &= \frac{\exp(\beta_{i,j})}{\sum_{j'} \exp(\beta_{i,j'})}. \end{aligned} \tag{8.42}$$

Trên mỗi nút tính toán, ta cần thực hiện các phương trình này. Với các khóa và giá trị được giao cho nút này, việc tính tử số ở vế phải của Phương trình (8.42) (tức $\exp(\beta_{i,j})$) là đơn giản, vì mọi thông tin cần thiết đều nằm trên nút. Tuy nhiên, việc tính mẫu số ở vế phải của Phương trình (8.42) đòi hỏi một tổng của $\exp(\beta_{i,j'})$ trên mọi j', điều này cần truyền dữ liệu qua lại với các nút khác. Để minh họa, giả sử $\mathbf{v}_j$ và $\mathbf{k}_j$ được đặt trên nút u. Ta có thể viết lại Phương trình (8.42) thành

$\alpha_{i,j}$

$$= \frac{\overbrace{\exp(\beta_{i,j})}^{\text{nút } u}}{\underbrace{\sum_{\mathbf{k}_{j'} \in \mathbf{K}^{[1]}} \exp(\beta_{i,j'})}_{\text{nút } 1} + \cdots + \underbrace{\sum_{\mathbf{k}_{j'} \in \mathbf{K}^{[u]}} \exp(\beta_{i,j'})}_{\text{nút } u} + \cdots + \underbrace{\sum_{\mathbf{k}_{j'} \in \mathbf{K}^{[n_u]}} \exp(\beta_{i,j'})}_{\text{nút } n_u}}, \tag{8.43}$$

trong đó ký hiệu $\mathbf{k}_{j'} \in \mathbf{K}^{[u]}$ cho biết $\mathbf{k}_{j'}$ là một vectơ hàng của $\mathbf{K}^{[u]}$. Trong một cài đặt trực tiếp, trước hết ta thực hiện các phép tính tổng $\{\sum_{\mathbf{k}_{j'} \in \mathbf{K}^{[u]}} \exp(\beta_{i,j'})\}$ một cách riêng rẽ trên các nút tương ứng. Sau đó, ta thu thập các kết quả tổng này từ các nút khác nhau để kết hợp thành một kết quả cuối cùng. Điều này tương ứng với một phép toán tập thể (*collective operation*) trong bối cảnh xử lý song song. Có nhiều cách cài đặt hiệu quả cho những phép toán như vậy, chẳng hạn các phép toán *all-reduce*. Do đó, tổng của mọi giá trị $\exp(\beta_{i,j})$ có thể được tính bằng các thủ tục đã được tối ưu trong các bộ công cụ truyền thông tập thể (*collective communication*).

Với các trọng số chú ý $\{\alpha_{i,j}\}$ đã có, ta tính kết quả chú ý bằng Phương trình (8.40). Bài toán có thể được biểu diễn lại là

$$\begin{aligned}&\text{Att}_{\text{qkv}}(\mathbf{q}_i, \mathbf{K}, \mathbf{V}) \\ &= \underbrace{\sum_{\mathbf{v}_{j'} \in \mathbf{V}^{[1]}} \alpha_{i,j'} \mathbf{v}_{j'}}_{\text{nút } 1} + \cdots + \underbrace{\sum_{\mathbf{v}_{j'} \in \mathbf{V}^{[u]}} \alpha_{i,j'} \mathbf{v}_{j'}}_{\text{nút } u} + \cdots + \underbrace{\sum_{\mathbf{v}_{j'} \in \mathbf{V}^{[n_u]}} \alpha_{i,j'} \mathbf{v}_{j'}}_{\text{nút } n_u}.\end{aligned} \tag{8.44}$$

Giống như Phương trình (8.43), Phương trình (8.44) có thể được cài đặt như một phép rút gọn (*reduction operation*) trong xử lý song song. Trước hết, ta thực hiện đồng thời các phép tính tổng có trọng số của các giá trị trên các nút khác nhau. Sau đó, ta thu thập kết quả từ các nút này bằng các phép toán tập thể.

Chú giải của người dịch: Song song chuỗi qua một ví dụ nhỏ

Giả sử chuỗi có 6 token, chia cho 2 nút: nút 1 giữ khóa/giá trị của token 0–2, nút 2 giữ token 3–5. Để tính trọng số chú ý, mẫu số của Softmax là $e^{\beta_0} + \cdots + e^{\beta_5}$. Nút 1 tự tính $S_1 = e^{\beta_0} + e^{\beta_1} + e^{\beta_2}$, nút 2 tự tính $S_2 = e^{\beta_3} + e^{\beta_4} + e^{\beta_5}$, rồi chỉ cần trao đổi *một con số* S_1, S_2 để mỗi nút có tổng

$S = S_1 + S_2$. Sau đó mỗi nút tính phần tổng có trọng số $\sum \alpha_{i,j}\mathbf{v}_j$ trên các token của mình, và cuối cùng cộng hai vectơ lại. "All-reduce" chính là thao tác "mọi nút góp phần của mình, rồi mọi nút đều nhận được tổng chung".

Lưu ý rằng mặc dù mục này chủ yếu tập trung vào mô hình hóa chuỗi dài, phần lớn động lực của song song chuỗi đến từ các phương pháp huấn luyện phân tán cho mạng sâu, như đã bàn ở Mục 8.2.3. Do đó, việc cài đặt các phương pháp này có thể được xây dựng trên cùng một thư viện xử lý song song.

8.3.2 Các kiến trúc hiệu quả

Một khó khăn khi áp dụng Transformer cho chuỗi dài là cơ chế tự chú ý chuẩn có độ phức tạp thời gian và bộ nhớ bậc hai theo độ dài chuỗi. Hơn nữa, việc sinh văn bản tự hồi quy (*autoregressive*) đòi hỏi duy trì một **bộ nhớ đệm khóa-giá trị** (*key-value cache*) (hay **bộ nhớ đệm KV** (*KV cache*)), có kích thước tăng dần khi càng nhiều token được xử lý. Mặc dù dung lượng bộ nhớ của bộ nhớ đệm KV chỉ tăng tuyến tính theo độ dài chuỗi, với những ngữ cảnh cực kỳ dài, yêu cầu bộ nhớ này trở nên quá lớn, khiến việc triển khai LLM chuẩn không còn thực tế. Kết quả là kiến trúc mô hình của các LLM ngữ cảnh dài nhìn chung rời xa Transformer chuẩn, chuyển sang phát triển các biến thể và phương án thay thế hiệu quả hơn.

Một cách tiếp cận là thay cơ chế tự chú ý chuẩn bằng chú ý thưa (*sparse attention*). Họ mô hình này xuất phát từ nhận xét rằng các phân phối chú ý trên chuỗi dài thường rất lệch. Do đó, chỉ một tập con nhỏ các token có ảnh hưởng đáng kể tới ngữ cảnh của một token bất kỳ. Vì vậy, ta có thể cắt tỉa phần lớn các trọng số chú ý, biểu diễn cơ chế này dưới dạng được nén mạnh. Để minh họa, xét công thức tự chú ý chuẩn:

$$\mathrm{Att}_{\mathrm{qkv}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \;=\; \alpha(\mathbf{Q}, \mathbf{K})\mathbf{V}, \tag{8.45}$$

trong đó ma trận trọng số chú ý $\alpha(\mathbf{Q}, \mathbf{K}) \in \mathbb{R}^{m \times m}$ thu được bởi

$$\alpha(\mathbf{Q}, \mathbf{K}) = \mathrm{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{Mask}\right)$$

$$= \begin{bmatrix} \alpha_{0,0} & 0 & 0 & \cdots & 0 \\ \alpha_{1,0} & \alpha_{1,1} & 0 & \cdots & 0 \\ \alpha_{2,0} & \alpha_{2,1} & \alpha_{2,2} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \alpha_{m-1,0} & \alpha_{m-1,1} & \alpha_{m-1,2} & \cdots & \alpha_{m-1,m-1} \end{bmatrix}. \tag{8.46}$$

Mỗi vectơ hàng $\begin{bmatrix} \alpha_{i,0} & \cdots & \alpha_{i,i} & 0 & \cdots & 0 \end{bmatrix}$ tương ứng với một phân phối trên các token, cho biết token thứ i chú ý tới các token khác như thế nào. Vì mô hình ngôn ngữ dự đoán token tiếp theo chỉ dựa trên ngữ cảnh bên trái, ta thường viết đầu ra của mô hình chú ý tại vị trí i là

$$\begin{aligned} \text{Att}_{\text{qkv}}(\mathbf{q}_i, \mathbf{K}_{\leq i}, \mathbf{V}_{\leq i}) &= \begin{bmatrix} \alpha_{i,0} & \cdots & \alpha_{i,i} \end{bmatrix} \begin{bmatrix} \mathbf{v}_0 \\ \vdots \\ \mathbf{v}_i \end{bmatrix} \\ &= \sum_{j=0}^{i} \alpha_{i,j} \mathbf{v}_j, \end{aligned} \tag{8.47}$$

trong đó $\mathbf{K}_{\leq i} = \begin{bmatrix} \mathbf{k}_0 \\ \vdots \\ \mathbf{k}_i \end{bmatrix}$ và $\mathbf{V}_{\leq i} = \begin{bmatrix} \mathbf{v}_0 \\ \vdots \\ \mathbf{v}_i \end{bmatrix}$ là các khóa và giá trị tính đến vị trí i.

Trong tự chú ý chuẩn, vectơ $\begin{bmatrix} \alpha_{i,0} & \cdots & \alpha_{i,i} \end{bmatrix}$ được giả định là dày đặc (*dense*), nghĩa là gần như mọi phần tử hợp lệ đều khác 0. Trong chú ý thưa, chỉ một tập con định trước của các phần tử này được tính, còn phần còn lại bị bỏ qua một cách tường minh (ép bằng 0). Gọi $G \subseteq \{0, \cdots, i\}$ là tập chỉ số của các phần tử hoạt động (khác 0). Khi đó, đầu ra của mô hình chú ý thưa tại vị trí i trở thành:

$$\text{Att}_{\text{sparse}}(\mathbf{q}_i, \mathbf{K}_{\leq i}, \mathbf{V}_{\leq i}) = \sum_{j \in G} \alpha'_{i,j} \mathbf{v}_j. \tag{8.48}$$

Ở đây $\{\alpha'_{i,j}\}$ được chuẩn hóa trên G. Vì thế giá trị của chúng khác với các trọng số chú ý ban đầu (thực tế ta có $\alpha'_{i,j} > \alpha_{i,j}$). Độ thưa của mô hình được quyết định bởi kích thước của G. Các mô hình chú ý thưa khác nhau ở cách định nghĩa G. Một cách đơn giản là định nghĩa G dựa trên các mẫu được thiết kế theo kinh nghiệm. Ví dụ, một mẫu được dùng rộng

rãi là cho G bao phủ một cửa sổ các token nằm gần vị trí i [Parmar và cộng sự, 2018].

Chú giải của người dịch: Vì sao $\alpha'_{i,j} > \alpha_{i,j}$?

Giả sử token i chú ý tới 4 token với trọng số $[0.1 \;\; 0.2 \;\; 0.3 \;\; 0.4]$ (tổng bằng 1). Nếu chú ý thưa chỉ giữ hai token cuối, $G = \{2, 3\}$, thì ta chuẩn hóa lại cho tổng bằng 1: $0.3/(0.3 + 0.4) \approx 0.43$ và $0.4/0.7 \approx 0.57$. Cả hai đều lớn hơn giá trị cũ vì "phần bánh" của các token bị bỏ đi được chia lại cho các token còn giữ. Đổi lại, mỗi token chỉ phải tính $|G|$ trọng số thay vì $i + 1$ trọng số.

Dù chú ý thưa giảm khối lượng tính toán nhờ dùng các phép toán thưa, những mô hình này vẫn có hạn chế đáng kể vì ta cần giữ toàn bộ bộ nhớ đệm KV (tức $\mathbf{K}_{\leq i}$ và $\mathbf{V}_{\leq i}$) trong quá trình suy luận. Nếu chuỗi rất dài, việc lưu bộ nhớ đệm này sẽ tốn rất nhiều bộ nhớ. Để giải quyết, ta có thể xét một lớp mô hình chú ý khác, trong đó bộ nhớ đệm KV không được lưu giữ một cách tường minh. Chú ý tuyến tính (*Linear attention*) là một cách tiếp cận như vậy [Katharopoulos và cộng sự, 2020]. Nó dùng một hàm nhân (*kernel function*) $\phi(\cdot)$ để ánh xạ mỗi truy vấn và khóa vào một không gian đặc trưng, lần lượt cho ra $\mathbf{q}'_i = \phi(\mathbf{q}_i)$ và $\mathbf{k}'_i = \phi(\mathbf{k}_i)$. Bằng cách thay phép chuẩn hóa Softmax bằng một công thức tuyến tính[9], dạng của mô hình chú ý thu được là

$$\begin{aligned} \mathrm{Att}_{\mathrm{qkv}}(\mathbf{q}_i, \mathbf{K}_{\leq i}, \mathbf{V}_{\leq i}) &\approx \mathrm{Att}_{\mathrm{linear}}(\mathbf{q}'_i, \mathbf{K}'_{\leq i}, \mathbf{V}_{\leq i}) \\ &= \frac{\mathbf{q}'_i \mu_i}{\mathbf{q}'_i \nu_i}, \end{aligned} \tag{8.49}$$

trong đó μ_i và ν_i là các biến được tính theo cách hồi quy

$$\mu_i = \mu_{i-1} + \mathbf{k}'^{\top}_i \mathbf{v}_i \tag{8.50}$$

$$\nu_i = \nu_{i-1} + \mathbf{k}'^{\top}_i. \tag{8.51}$$

μ_i và ν_i có thể được xem như các bản tóm tắt lịch sử tính đến vị trí i. Một lợi ích của mô hình này là ta không cần lưu tường minh mọi khóa và giá trị trong quá khứ. Thay vào đó, chỉ các biểu diễn mới nhất μ_i và ν_i được

[9] Trong không gian mới sau phép biến đổi này, phép chuẩn hóa Softmax có thể được chuyển thành phép chuẩn hóa co giãn đơn giản.

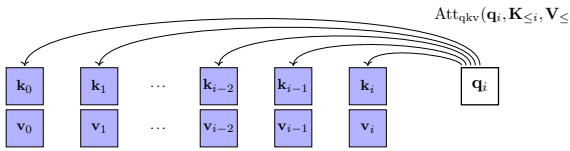
sử dụng. Vì vậy chi phí tính toán ở mỗi bước là một hằng số, và mô hình có thể dễ dàng mở rộng để xử lý chuỗi dài.

Chú giải của người dịch: Ý tưởng "tổng cộng dồn" của chú ý tuyến tính

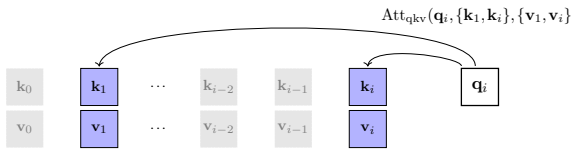
Phương trình (8.50–8.51) giống cách tính tổng tiền tiết kiệm: thay vì ghi lại từng khoản gửi rồi mỗi lần lại cộng hết từ đầu, ta chỉ giữ *số dư* và cộng thêm khoản mới: số dư$_i$ = số dư$_{i-1}$ + khoản gửi$_i$. Ở đây "số dư" là μ_i (một ma trận cố định kích thước) và ν_i (một vectơ). Mỗi token mới chỉ tốn một phép cộng vào hai đại lượng này, nên bộ nhớ và thời gian cho mỗi bước không phụ thuộc vào việc chuỗi đã dài bao nhiêu. Điều này làm được là vì đã bỏ hàm mũ của Softmax: tổng $\sum_j (\mathbf{q}_i' \mathbf{k}_j'^\top)\mathbf{v}_j$ có thể viết thành $\mathbf{q}_i' \big(\sum_j \mathbf{k}_j'^\top \mathbf{v}_j \big)$, tức đưa $\mathbf{q}_i'$ ra ngoài tổng.

Thực ra, cách tiếp cận tuần tự này đối với mô hình hóa chuỗi dài xuất hiện một cách tự nhiên khi ta nhìn từ góc độ các mô hình hồi quy. Những mô hình như vậy đọc một token (hoặc một số ít token) mỗi lần, cập nhật trạng thái hồi quy (*recurrent state*) bằng các đầu vào này, rồi loại bỏ chúng trước khi token tiếp theo đến. Đầu ra ở mỗi bước được sinh ra chỉ dựa trên trạng thái hồi quy, chứ không dựa trên tất cả các trạng thái trước đó. Dung lượng bộ nhớ được quyết định bởi trạng thái hồi quy, vốn có kích thước cố định. Các mô hình hồi quy có thể được dùng trong các tình huống học thời gian thực, khi dữ liệu đến theo dòng và dự đoán có thể được đưa ra tại bất kỳ bước thời gian nào. Trong NLP, áp dụng mô hình hồi quy cho mô hình hóa ngôn ngữ là một trong những nỗ lực thành công sớm nhất để học biểu diễn của chuỗi. Mặc dù Transformer đã được dùng làm kiến trúc nền tảng của LLM, các mô hình hồi quy vẫn là những mô hình mạnh, đặc biệt cho việc phát triển các LLM hiệu quả. Chẳng hạn, các mô hình hồi quy, nhất là các mô hình không gian trạng thái (*state space models*) hiện đại như Mamba [Gu và Dao, 2023], đã hồi sinh trong mô hình hóa ngôn ngữ và được xem xét lại như một phương án thay thế đầy hứa hẹn cho Transformer.

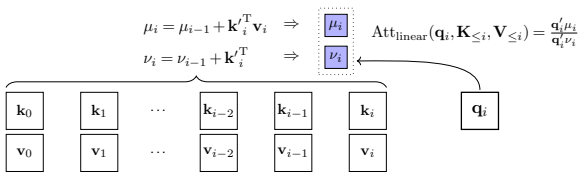
Hình 8.5 so sánh các mô hình được bàn trong tiểu mục này. Vì các mô hình này, cùng những mô hình khác không được nhắc tới ở đây, đã được bàn kỹ trong Chương 6 và trong các bài tổng quan liên quan [Tay và cộng sự, 2020b], chúng tôi bỏ qua phần thảo luận chi tiết hơn ở đây.



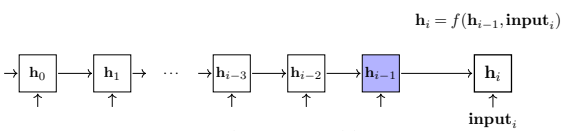
(a) Standard Self-attention



(b) Sparse Attention



(c) Linear Attention



(d) Recurrent Models

Hình 8.5: Minh họa cơ chế tự chú ý, chú ý thưa, chú ý tuyến tính và các mô hình hồi quy. Các hộp màu xanh lam là những trạng thái được lưu đệm để tạo ra đầu ra tại vị trí i. $f(\cdot)$ là một ô hồi quy (*recurrent cell*). (Các nhãn con: (a) tự chú ý chuẩn; (b) chú ý thưa; (c) chú ý tuyến tính; (d) các mô hình hồi quy.)

8.3.3 Bộ nhớ đệm và bộ nhớ

Các LLM dựa trên kiến trúc Transformer chuẩn sử dụng cơ chế chú ý toàn cục (dày đặc). Trong quá trình lập luận tự hồi quy, các mô hình này cần lưu trữ toàn bộ ngữ cảnh bên trái để dự đoán các token tương lai. Điều này đòi hỏi một bộ nhớ đệm KV (*KV cache*), trong đó các biểu diễn (tức là khóa và giá trị) của mọi token đã được sinh ra trước đó đều được giữ lại. Kết quả là dung lượng bộ nhớ mà bộ nhớ đệm này chiếm dụng tăng tuyến tính theo tiến trình suy luận. Trong các mục trước, chúng ta đã bàn về những phương pháp giảm bớt nút thắt ngữ cảnh này thông qua các thuật toán chú ý hiệu quả, như chú ý thưa và chú ý tuyến tính. Một hướng tiếp cận thay thế có liên quan là mã hóa và nén ngữ cảnh một cách tường minh thông qua một mô-đun bộ nhớ phụ trợ (*auxiliary memory module*).

Chú giải của người dịch: Vì sao bộ nhớ đệm KV "phình" ra?

Khi sinh văn bản, mô hình tạo từng token một. Để tạo token thứ $i+1$, truy vấn $\mathbf{q}_i$ phải "nhìn" lại khóa và giá trị của *tất cả* token trước đó. Thay vì tính lại chúng mỗi bước, ta lưu chúng lại (đó là bộ nhớ đệm KV). Nếu mỗi token cần lưu, chẳng hạn, 1 MB khóa–giá trị, thì một văn bản 1000 token cần 1 GB, còn 100 000 token cần tới 100 GB. Dung lượng tăng tỉ lệ thuận với độ dài, nên với ngữ cảnh rất dài, bộ nhớ trở thành "nút cổ chai". Ý tưởng của tiểu mục này: giữ cho bộ nhớ có kích thước *cố định*, bất kể văn bản dài bao nhiêu.

1. Bộ nhớ đệm KV kích thước cố định

Một cách tiếp cận trực tiếp là biểu diễn các khóa và giá trị bằng một mô hình bộ nhớ có kích thước cố định. Giả sử ta có một bộ nhớ Mem lưu giữ thông tin ngữ cảnh. Ta có thể viết phép toán chú ý tại vị trí i dưới dạng tổng quát

$$\mathrm{Att}(\mathbf{q}_i, \mathrm{Mem}) \;=\; \mathrm{Att}_{\mathrm{qkv}}(\mathbf{q}_i, \mathbf{K}_{\le i}, \mathbf{V}_{\le i}). \tag{8.52}$$

Trong mô hình này, Mem đơn giản chính là bộ nhớ đệm KV, tức là $\mathrm{Mem} = (\mathbf{K}_{\le i}, \mathbf{V}_{\le i})$. Do đó kích thước của Mem được quyết định bởi i. Nếu Mem được định nghĩa có kích thước cố định thì chi phí thực hiện $\mathrm{Att}(\mathbf{q}_i, \mathrm{Mem})$ cũng sẽ cố định. Có một số cách khác nhau để thiết kế Mem.

- Một trong những phương pháp đơn giản nhất là xét một cửa sổ có kích thước cố định gồm các khóa và giá trị trước đó. Khi đó Mem

được cho bởi

$$\text{Mem} = (\mathbf{K}_{[i-n_c+1,i]}, \mathbf{V}_{[i-n_c+1,i]}), \tag{8.53}$$

trong đó n_c là kích thước của cửa sổ. Các ký hiệu $\mathbf{K}_{[i-n_c+1,i]}$ và $\mathbf{V}_{[i-n_c+1,i]}$ chỉ các khóa và giá trị trên các vị trí từ $i - n_c + 1$ đến i.[10] Mô hình này có thể được xem là một dạng mô hình chú ý cục bộ (*local attention*).

- Ta cũng có thể định nghĩa Mem là một cặp vectơ tóm tắt, dẫn tới một biểu diễn lịch sử được nén nhiều hơn. Một cách đơn giản để tóm tắt các khóa và giá trị trước đó là dùng trung bình trượt (*moving average*) của các vectơ này. Ví dụ, Mem có thể được định nghĩa là trung bình trượt không trọng số của n_c khóa và giá trị gần nhất

$$\text{Mem} = \left(\frac{\sum_{j=i-n_c+1}^{i} \mathbf{k}_j}{n_c}, \frac{\sum_{j=i-n_c+1}^{i} \mathbf{v}_j}{n_c} \right). \tag{8.54}$$

Ngoài ra, ta có thể dùng trung bình trượt có trọng số

$$\text{Mem} = \left(\frac{\sum_{j=i-n_c+1}^{i} \beta_{j-i+n_c} \mathbf{k}_j}{\sum_{j=1}^{n_c} \beta_j}, \frac{\sum_{j=i-n_c+1}^{i} \beta_{j-i+n_c} \mathbf{v}_j}{\sum_{j=1}^{n_c} \beta_j} \right). \tag{8.55}$$

Ở đây $\{\beta_1, \cdots, \beta_{n_c}\}$ là các hệ số, có thể được học như tham số mô hình hoặc được xác định bằng các quy tắc kinh nghiệm (*heuristics*). Ví dụ, ta có thể đặt chúng là các hệ số tăng dần (tức là $\beta_1 < \beta_2 < \cdots < \beta_{n_c-1} < \beta_{n_c}$) để gán trọng số lớn hơn cho những vị trí gần i hơn. Ta có thể mở rộng trung bình trượt để bao gồm tất cả các vị trí cho tới i. Điều này dẫn tới trung bình tích lũy (*cumulative average*) của các khóa và giá trị, có dạng

$$\text{Mem} = \left(\frac{\sum_{j=0}^{i} \mathbf{k}_j}{i+1}, \frac{\sum_{j=0}^{i} \mathbf{v}_j}{i+1} \right). \tag{8.56}$$

[10] Một cách hình thức hơn, ta viết $\mathbf{K}_{[i-n_c+1,i]} = \begin{bmatrix} \mathbf{k}_{i-n_c+1} \\ \vdots \\ \mathbf{k}_i \end{bmatrix}$ và $\mathbf{V}_{[i-n_c+1,i]} = \begin{bmatrix} \mathbf{v}_{i-n_c+1} \\ \vdots \\ \mathbf{v}_i \end{bmatrix}$. Đôi khi, để ký hiệu gọn hơn, ta viết $\mathbf{K}_{[i-n_c+1,i]}$ thành $\{\mathbf{k}_{i-n_c+1}, \cdots, \mathbf{k}_i\}$ và $\mathbf{V}_{[i-n_c+1,i]}$ thành $\{\mathbf{v}_{i-n_c+1}, \cdots, \mathbf{v}_i\}$.

Nói chung, trung bình tích lũy có thể được viết dưới dạng đệ quy

$$\text{Mem}_i = \frac{(\mathbf{k}_i, \mathbf{v}_i) + i \cdot \text{Mem}_{i-1}}{i+1}, \tag{8.57}$$

trong đó Mem_i và Mem_{i-1} lần lượt là trung bình tích lũy tại vị trí hiện tại và vị trí trước đó. Ưu điểm của mô hình này là trong khi suy luận ta chỉ cần lưu một cặp khóa–giá trị duy nhất, thay vì lưu tất cả các cặp khóa–giá trị. Lưu ý rằng các mô hình bộ nhớ ở trên có liên hệ với các mô hình hồi quy, và nhiều kỹ thuật tiên tiến hơn đã được dùng để phát triển các phương án thay thế cho cơ chế tự chú ý trong Transformer [Ma và cộng sự, 2023].

Chú giải của người dịch: Kiểm tra công thức trung bình tích lũy bằng số

Lấy các "khóa" là các số đơn giản: $k_0 = 2$, $k_1 = 4$, $k_2 = 9$. Theo Phương trình (8.56), trung bình tích lũy tại $i = 2$ là $(2+4+9)/3 = 5$. Dùng dạng đệ quy (8.57): $\text{Mem}_0 = 2$; $\text{Mem}_1 = (4 + 1 \cdot 2)/2 = 3$; $\text{Mem}_2 = (9 + 2 \cdot 3)/3 = 5$. Kết quả trùng khớp! Điểm mấu chốt: ở mỗi bước ta chỉ cần nhớ *một* giá trị trung bình cũ (nhân với i để "khôi phục" tổng cũ), chứ không cần nhớ toàn bộ dãy $k_0, k_1, \ldots$. Đây cũng là cách bạn tính lại điểm trung bình môn khi có thêm một bài kiểm tra mà không cần xem lại mọi điểm cũ.

Trong trung bình trượt có trọng số (8.55), với $n_c = 3$ và $\beta_1 = 1, \beta_2 = 2, \beta_3 = 3$, token gần nhất ($j = i$) có trọng số $\beta_3 = 3$, token xa nhất trong cửa sổ có trọng số $\beta_1 = 1$; tổng được chia cho $1 + 2 + 3 = 6$ để các "tỉ lệ đóng góp" cộng lại bằng 1.

- Bộ nhớ Mem cũng có thể là một mạng nơ-ron. Tại mỗi bước, nó nhận cả đầu ra trước đó của bộ nhớ lẫn các trạng thái hiện tại của mô hình làm đầu vào, rồi tạo ra đầu ra mới của bộ nhớ. Mạng nơ-ron này có thể được biểu diễn dưới dạng hàm

$$\text{Mem} = \text{Update}(S_{\text{kv}}, \text{Mem}_{\text{pre}}). \tag{8.58}$$

Ở đây Mem và Mem_{pre} lần lượt là đầu ra của bộ nhớ tại bước hiện tại và bước trước đó. S_{kv} là một tập các cặp khóa–giá trị, biểu diễn các trạng thái gần đây của mô hình. Cách biểu diễn này mang tính tổng quát và cho phép ta phát triển nhiều mô hình bộ nhớ khác nhau bằng

cách chọn các cấu hình Update$(\cdot)$ và S_{kv} khác nhau. Ví dụ, nếu S_{kv} chỉ chứa cặp khóa–giá trị mới nhất $(\mathbf{k}_i, \mathbf{v}_i)$ và Update$(\cdot)$ được định nghĩa là một ô hồi quy, thì Phương trình (8.58) có thể được biểu diễn thành một mô hình giống RNN

$$\text{Mem} \;=\; f((\mathbf{k}_i, \mathbf{v}_i), \text{Mem}_{\text{pre}}), \tag{8.59}$$

trong đó $f(\cdot)$ là một ô hồi quy (*recurrent cell*). Phép hồi quy cũng có thể được áp dụng cho việc mô hình hóa ở mức đoạn để tăng hiệu quả. Một cách đơn giản là chia chuỗi thành các đoạn (*segment*) và coi S_{kv} là một đoạn. Áp dụng các mô hình hồi quy cho Update$(\cdot)$ sẽ cho ra các mô hình bộ nhớ hoạt động trên các đoạn. Một trường hợp đặc biệt là định nghĩa Update$(\cdot)$ là một hàm FIFO, thêm S_{kv} vào bộ nhớ và loại bỏ đoạn khóa–giá trị cũ nhất khỏi bộ nhớ, cho bởi

$$\text{Mem} \;=\; \text{FIFO}(S_{\text{kv}}, \text{Mem}_{\text{pre}}). \tag{8.60}$$

Xét một bộ nhớ gồm hai đoạn: một cho đoạn hiện tại và một cho đoạn trước đó. Trong phép toán chú ý, mỗi vị trí có thể truy cập các cặp khóa–giá trị lịch sử trong hai đoạn liên tiếp gần nhất. Tuy đơn giản về mặt khái niệm, bộ nhớ cục bộ dạng khối trượt (*sliding-chunk local memory*) này cùng các biến thể của nó đã được dùng rộng rãi trong các kiến trúc hồi quy mức đoạn [Dai và cộng sự, 2019; Hutchins và cộng sự, 2022; Bulatov và cộng sự, 2022].

Chú giải của người dịch: FIFO là gì?

FIFO (*First In, First Out* – vào trước, ra trước) là cách hoạt động của một *hàng đợi*, giống hàng người xếp hàng mua vé: ai đến trước được phục vụ (và rời hàng) trước. Với bộ nhớ có sức chứa cố định là 2 đoạn: khi đoạn mới S_3 đến, bộ nhớ đang chứa $\{S_1, S_2\}$ sẽ "đẩy" S_1 (cũ nhất) ra và trở thành $\{S_2, S_3\}$. Nhờ vậy kích thước bộ nhớ luôn không đổi.

- Các mô hình bộ nhớ trên có thể được mở rộng để dùng nhiều bộ nhớ. Một ví dụ cho cách tiếp cận này là Transformer nén (*compressive Transformer*) [Rae và cộng sự, 2019b]. Mô hình này dùng hai bộ nhớ riêng biệt có kích thước cố định: một để mô hình hóa ngữ cảnh cục bộ (ký hiệu Mem), và một để mô hình hóa và nén lịch sử dài

hạn (ký hiệu CMem). Bộ nhớ đệm KV trong mô hình này là sự kết hợp của Mem và CMem. Hàm chú ý có thể được viết là

$$\text{Att}_{\text{com}}(\mathbf{q}_i, \text{Mem}, \text{CMem}) \;=\; \text{Att}_{\text{qkv}}(\mathbf{q}_i, [\text{Mem}, \text{CMem}]), \quad (8.61)$$

trong đó $[\text{Mem}, \text{CMem}]$ là bộ nhớ gộp của Mem và CMem. Tương tự các mô hình mức đoạn chuẩn, Transformer nén xử lý chuỗi theo từng khối. Mỗi đoạn gồm n_s token liên tiếp, và S^k_{kv} ký hiệu các cặp khóa–giá trị thô sinh ra từ đoạn thứ k. Khi một đoạn mới đến, Mem được cập nhật qua một hàng đợi FIFO: nó nối thêm n_s cặp khóa–giá trị mới từ S^k_{kv} và đồng thời loại bỏ n_s cặp cũ nhất:

$$\text{Mem} \;=\; \text{FIFO}(S^k_{\text{kv}}, \text{Mem}_{\text{pre}}). \quad (8.62)$$

Thay vì vứt bỏ các cặp bị đẩy ra này, chúng được đưa qua một mạng nén (*compression network*). n_s cặp thô được nén thành một tập nhỏ hơn gồm $\frac{n_s}{c}$ cặp khóa–giá trị cô đọng. Sau đó CMem đóng vai trò hàng đợi FIFO thứ cấp, nối thêm $\frac{n_s}{c}$ cặp đã nén này và loại bỏ $\frac{n_s}{c}$ mục cũ nhất của nó để giữ kích thước cố định. Điều này được cho bởi

$$\text{CMem} \;=\; \text{FIFO}(C^k_{\text{kv}}, \text{CMem}_{\text{pre}}), \quad (8.63)$$

trong đó C^k_{kv} biểu diễn tập các cặp khóa–giá trị đã nén. Transformer nén ngầm giả định rằng ngữ cảnh cục bộ nên được biểu diễn tường minh với mức mất mát thông tin tối thiểu, còn ngữ cảnh xa có thể được nén nhiều hơn.

Chú giải của người dịch: Ví dụ số cho Transformer nén

Giả sử mỗi đoạn có $n_s = 4$ token và tỉ lệ nén $c = 2$. Khi đoạn mới đến, 4 cặp khóa–giá trị mới được đưa vào Mem, đồng thời 4 cặp cũ nhất bị đẩy ra. Thay vì xóa, 4 cặp này được "nén" (chẳng hạn lấy trung bình từng đôi một) thành $4/2 = 2$ cặp, rồi đưa vào CMem; CMem lại đẩy ra 2 cặp cũ nhất của nó. Giống như cách ta ghi nhớ: chuyện vừa xảy ra thì nhớ chi tiết, chuyện lâu rồi chỉ nhớ ý chính.

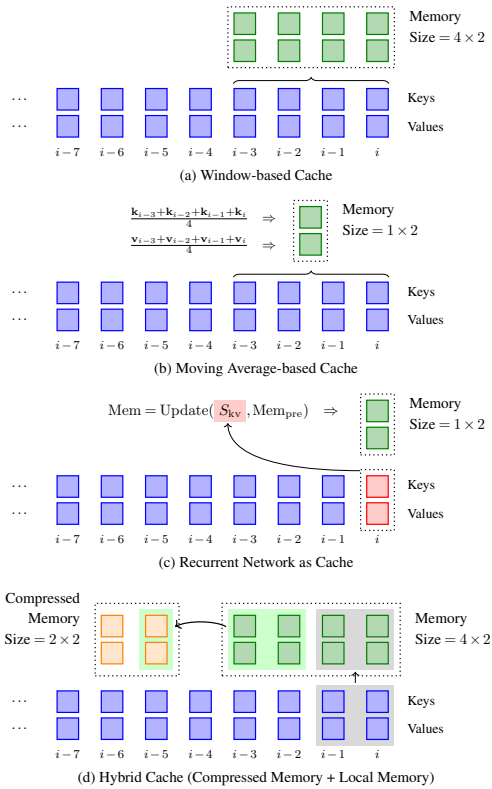
- Chúng ta đã thấy rằng cả ngữ cảnh toàn cục lẫn cục bộ đều hữu ích và có thể được mô hình hóa bằng các mô hình chú ý. Quan điểm này thúc đẩy việc mở rộng các mô hình chú ý để kết hợp cả bộ nhớ cục bộ lẫn bộ nhớ dài hạn [Ainslie và cộng sự, 2020; Zaheer và cộng

sự, 2020; Gupta và Berant, 2020]. Một cách đơn giản nhưng được dùng rộng rãi là đưa vài token đầu tiên của chuỗi vào phép chú ý, đóng vai trò các token toàn cục (*global tokens*). Cách này thường được áp dụng cùng với các mô hình chú ý thưa khác. Một ưu điểm của việc đưa vào các token toàn cục là chúng đóng vai trò một bể hút chú ý (*attention sink*), hấp thụ các điểm chú ý dư thừa, nhờ đó ổn định hiệu năng mô hình khi kích thước ngữ cảnh rất lớn [Xiao và cộng sự, 2024]. Tuy nhiên, một nhược điểm là việc dùng bộ nhớ toàn cục có kích thước cố định có thể gây mất mát thông tin. Khi xử lý các chuỗi dài, ta cần mở rộng bộ nhớ đệm KV để có đủ biểu diễn cho ngữ cảnh, nhưng điều này lại làm tăng chi phí tính toán.

Chú giải của người dịch: Bể hút chú ý

Trọng số chú ý được tính bằng Softmax nên luôn có tổng bằng 1: dù truy vấn không thực sự "cần" token nào, nó vẫn phải phân bổ hết 1 đơn vị chú ý. Thực nghiệm cho thấy mô hình thường "đổ" phần chú ý thừa vào vài token đầu chuỗi. Nếu ta cắt bỏ các token đầu này khỏi cửa sổ (như trong bộ nhớ đệm cửa sổ trượt thuần túy), phần chú ý thừa không còn chỗ "đổ" và chất lượng có thể giảm mạnh. Giữ lại vài token đầu như các token toàn cục giúp tránh hiện tượng đó.

Hình 8.6 minh họa các cách tiếp cận trên. Lưu ý rằng mặc dù ở đây ta tập trung vào việc tối ưu bộ nhớ đệm KV, vấn đề này có liên hệ chặt chẽ với những vấn đề đã bàn trong mục trước. Mọi phương pháp ta đã nêu đến giờ nhìn chung đều có thể xếp vào nhóm các cách tiếp cận chú ý hiệu quả, vốn được dùng rộng rãi trong nhiều biến thể Transformer.



Hình 8.6: Minh họa các bộ nhớ đệm KV kích thước cố định trong LLM. Các hộp màu xanh lam biểu diễn các khóa và giá trị được sinh ra trong quá trình suy luận của LLM, các hộp màu xanh lục biểu diễn các khóa và giá trị được lưu trữ hoặc mã hóa trong bộ nhớ chính, còn các hộp màu cam biểu diễn các khóa và giá trị được lưu trữ hoặc mã hóa trong bộ nhớ nén. (Các nhãn con: (a) bộ nhớ đệm dựa trên cửa sổ; (b) bộ nhớ đệm dựa trên trung bình trượt; (c) mạng hồi quy làm bộ nhớ đệm; (d) bộ nhớ đệm lai (bộ nhớ nén + bộ nhớ cục bộ). "Keys/Values" = khóa/giá trị; "Size = 4×2" = kích thước bộ nhớ.)

2. Các mô hình dựa trên bộ nhớ

Việc mô hình hóa bộ nhớ đã bàn ở trên dựa trên các cập nhật đối với bộ nhớ đệm KV, và các mô hình thu được thường được gọi là **bộ nhớ trong** (*internal memories*). Bây giờ ta xét một họ mô hình khác, gọi là **bộ nhớ ngoài** (*external memories*), hoạt động độc lập để truy cập các ngữ cảnh quy mô lớn cho LLM. Nhiều cách tiếp cận trong số này bắt nguồn từ các **phương pháp dựa trên bộ nhớ** (*memory-based methods*), một khái niệm đã được nghiên cứu sâu rộng trong học máy cổ điển [Bishop, 2006]. Một ví dụ phổ biến là các thuật toán láng giềng gần nhất: ta lưu các biểu diễn ngữ cảnh trong một kho dữ liệu (*datastore*) và truy vấn nó để tìm những biểu diễn đã lưu giống nhất. Các biểu diễn ngữ cảnh được truy xuất này sau đó được đưa vào để tăng cường cơ chế chú ý cho truy vấn hiện tại.

Ở đây, ta xét phương pháp k **láng giềng gần nhất** (k-*nearest neighbors,* k-*NN*), một trong những phương pháp dựa trên bộ nhớ phổ biến nhất. Vì trọng tâm của mục này là mô hình hóa ngôn ngữ, ta định nghĩa một mẫu trong kho dữ liệu là một cặp khóa–giá trị ứng với một trạng thái ngữ cảnh nào đó. Lưu ý rằng "ngữ cảnh" ở đây là một khái niệm rộng, không chỉ là tiền tố của chuỗi trong sinh văn bản. Chẳng hạn, ta có thể xem toàn bộ tập dữ liệu là ngữ cảnh để dự đoán token. Điều này cho phép ta truy xuất ngữ cảnh giống nhất trong một tập các chuỗi, thay vì trong một tiền tố chuỗi cho trước. Dù ta sẽ giới hạn ở việc mô hình hóa ngữ cảnh cho một chuỗi duy nhất, trong tiểu mục này ta bàn về một trường hợp tổng quát hơn.

Giả sử ta có một tập các khóa $\{\mathbf{k}_j\}$ cùng các giá trị tương ứng $\{\mathbf{v}_j\}$, và giả sử ta lưu các cặp khóa–giá trị này trong một cơ sở dữ liệu vectơ (*vector database*)[11]. Với mỗi truy vấn $\mathbf{q}_i$, ta tìm k láng giềng gần nhất của nó bằng cách tăng dần bán kính của mặt cầu tâm $\mathbf{q}_i$ cho tới khi mặt cầu chứa k điểm dữ liệu trong $\{\mathbf{k}_j\}$. Kết quả là một tập gồm k khóa cùng các giá trị tương ứng, ký hiệu là Mem_{knn}. Như trước, ta ký hiệu Mem là bộ nhớ cục bộ của truy vấn, chẳng hạn bộ nhớ đệm KV của các token lân cận. Mục tiêu của ta là cho phép truy vấn $\mathbf{q}_i$ chú ý tới cả bộ nhớ cục bộ Mem lẫn bộ nhớ dài hạn Mem_{knn}. Dĩ nhiên có nhiều cách để đưa Mem và Mem_{knn} vào mô hình chú ý. Ví dụ, ta có thể đơn giản gộp chúng lại để tạo thành một bộ nhớ đệm KV duy nhất $[\text{Mem}, \text{Mem}_{\text{knn}}]$, rồi cho $\mathbf{q}_i$ chú

[11] Cơ sở dữ liệu vectơ (hay kho vectơ) là một cơ sở dữ liệu cung cấp các giao diện truy xuất được tối ưu hóa cao để tìm những vectơ đã lưu khớp gần nhất với một vectơ truy vấn.

ý tới $[\text{Mem}, \text{Mem}_{\text{knn}}]$ bằng phép chú ý QKV chuẩn. Hoặc ta có thể dùng Mem và Mem_{knn} trong các bước chú ý riêng rẽ. Một ví dụ cho cách tiếp cận này là mô hình do Wu và cộng sự [2021] phát triển. Mô hình này kết hợp tuyến tính hai loại chú ý, cho bởi

$$\text{Att}(\mathbf{q}_i, \text{Mem}, \text{Mem}_{\text{knn}}) = \mathbf{g} \odot \text{Att}_{\text{local}} + (1 - \mathbf{g}) \odot \text{Att}_{\text{knn}} \tag{8.64}$$

$$\text{Att}_{\text{local}} = \text{Att}(\mathbf{q}_i, \text{Mem}) \tag{8.65}$$

$$\text{Att}_{\text{knn}} = \text{Att}(\mathbf{q}_i, \text{Mem}_{\text{knn}}). \tag{8.66}$$

Ở đây $\mathbf{g} \in \mathbb{R}^d$ là vectơ hệ số, có thể là đầu ra của một cổng (*gate*) được học.

Chú giải của người dịch: Ý nghĩa của Phương trình (8.64)

Ký hiệu $\odot$ là phép nhân theo từng phần tử. Mỗi thành phần g của vectơ $\mathbf{g}$ nằm trong khoảng $[0, 1]$ và đóng vai trò "núm vặn" trộn hai nguồn thông tin. Ví dụ với một thành phần: nếu $g = 0.8$, kết quả chú ý cục bộ cho 5 và kết quả từ k-NN cho 10, thì đầu ra là $0.8 \cdot 5 + 0.2 \cdot 10 = 6$, nghiêng về thông tin cục bộ. Vì $\mathbf{g}$ là vectơ, mỗi chiều có thể có tỉ lệ trộn riêng.

Với mô hình bộ nhớ dựa trên k-NN mô tả ở trên, việc còn lại là xác định những cặp khóa–giá trị nào được giữ trong kho dữ liệu. Với các tác vụ mô hình hóa ngôn ngữ chuẩn, ta coi các token đã thấy trước đó trong chuỗi là ngữ cảnh, nên có thể thêm khóa và giá trị của tất cả các token này vào kho dữ liệu. Trong trường hợp đó, mô hình chú ý dựa trên k-NN thu được về bản chất tương đương với một mô hình chú ý thưa [Gupta và cộng sự, 2021].

Mặt khác, ta có thể mở rộng ngữ cảnh từ một chuỗi thành một tập các chuỗi. Ví dụ, ta có thể thu thập mọi cặp khóa–giá trị trên các chuỗi trong một tập dữ liệu huấn luyện và thêm chúng vào kho dữ liệu để mô hình hóa một ngữ cảnh lớn hơn. Nhờ vậy, LLM có thể dự đoán token dựa trên một ngữ cảnh tổng quát hóa. Một vấn đề của cách này là chi phí tính toán có thể lớn nếu có nhiều chuỗi tham gia. Vì các chuỗi này là một phần dữ liệu huấn luyện của ta, ta có thể xây dựng và tối ưu một chỉ mục (*index*) cho các vectơ trong kho dữ liệu trước khi chạy LLM. Kết quả là việc truy xuất các vectơ tương tự có thể rất hiệu quả, như thường được cài đặt trong các cơ sở dữ liệu vectơ hiện đại.

Thực ra, tất cả các phương pháp kể trên đều là những trường hợp cụ

thể của mô thức tăng cường truy xuất rộng hơn. Ngoài việc đưa trực tiếp các vectơ được truy xuất vào cơ chế chú ý, việc truy xuất còn có thể được áp dụng ở mức phân phối đầu ra. Một ứng dụng điển hình là **mô hình hóa ngôn ngữ k-NN** (*k-NN language modeling, k-NN LM*) [Khandelwal và cộng sự, 2020]. Trực giác là: các trạng thái ẩn giống nhau trong một mạng Transformer có khả năng dự báo rất tốt rằng token tiếp theo cũng giống nhau, dù ở các ngữ cảnh khác nhau. Trong một k-NN LM, mỗi mục trong kho dữ liệu được lưu dưới dạng một bộ $(\mathbf{z}, w)$, trong đó $\mathbf{z}$ là trạng thái ẩn cuối cùng của LLM tại một vị trí cụ thể, còn w là token tiếp theo thực sự quan sát được tại vị trí đó. Thông thường, kho dữ liệu này được xây dựng *ngoại tuyến* (trước khi sử dụng) bằng cách cho toàn bộ tập dữ liệu huấn luyện đi qua LLM.

Trong khi suy luận, với một tiền tố chuỗi cho trước, LLM sinh ra trạng thái ẩn hiện tại $\mathbf{h}_i$. Ta truy vấn kho dữ liệu bằng $\mathbf{h}_i$ để lấy ra k trạng thái khớp gần nhất, thu được một tập các bộ $\{(\mathbf{z}_1, w_1), \cdots, (\mathbf{z}_k, w_k)\}$. Các token đích $\{w_1, \cdots, w_k\}$ đóng vai trò các đầu ra tham chiếu. Để tận dụng các tham chiếu này, ta chuyển các khoảng cách truy xuất thành một phân phối xác suất trên bộ từ vựng V. Gọi $d(\mathbf{h}_i, \mathbf{z}_j)$ là độ đo khoảng cách. Khối lượng xác suất mà phép truy xuất k-NN gán cho một token $v \in V$ được tính bằng cách cộng gộp các độ tương tự (đã chuẩn hóa bằng Softmax) của những láng giềng được truy xuất:

$$\Pr_{\text{knn}}(v|\mathbf{h}_i) \;\propto \sum_{\substack{(\mathbf{z}_j, w_j) \in \text{top-}k, \\ w_j = v}} \exp\left(-d(\mathbf{h}_i, \mathbf{z}_j)\right). \tag{8.67}$$

Nếu một token v không xuất hiện trong số k láng giềng được truy xuất thì xác suất $\Pr_{\text{knn}}(v|\mathbf{h}_i)$ của nó bằng 0. Cuối cùng, ta dùng một siêu tham số điều chỉnh được $\lambda \in [0, 1]$ để nội suy tuyến tính giữa phân phối dựa trên truy xuất này và phân phối tham số chuẩn $\Pr_{\text{lm}}(\cdot|\mathbf{h}_i)$ do đầu phân loại (*classification head*) của LLM sinh ra:

$$\Pr(\cdot|\mathbf{h}_i) \;=\; \lambda \cdot \Pr_{\text{knn}}(\cdot|\mathbf{h}_i) + (1 - \lambda) \cdot \Pr_{\text{lm}}(\cdot|\mathbf{h}_i). \tag{8.68}$$

Sau đó, như thường lệ, ta có thể chọn token tiếp theo y bằng cách cực đại hóa xác suất $\Pr(y|\mathbf{h}_i)$.

Chú giải của người dịch: Ví dụ tính tay cho k-NN LM

Giả sử $k = 3$ láng giềng được truy xuất là: ($\mathbf{z}_1$, "mèo") với khoảng cách $d = 1$, ($\mathbf{z}_2$, "mèo") với $d = 2$, và ($\mathbf{z}_3$, "chó") với $d = 1$. Theo (8.67), điểm của "mèo" là $e^{-1} + e^{-2} \approx 0.368 + 0.135 = 0.503$, điểm của "chó" là $e^{-1} \approx 0.368$. Ký hiệu $\propto$ ("tỉ lệ với") nghĩa là ta còn phải chia cho tổng $0.503 + 0.368 = 0.871$ để được xác suất: $\Pr_{\text{knn}}(\text{mèo}) \approx 0.58$, $\Pr_{\text{knn}}(\text{chó}) \approx 0.42$, mọi token khác có xác suất 0. Khoảng cách càng nhỏ thì e^{-d} càng lớn, tức láng giềng càng gần càng "có tiếng nói". Nếu LLM thông thường cho $\Pr_{\text{lm}}(\text{mèo}) = 0.3$ và chọn $\lambda = 0.5$ thì theo (8.68): $\Pr(\text{mèo}) = 0.5 \cdot 0.58 + 0.5 \cdot 0.3 = 0.44$.

Giống như các hệ thống truy xuất thông tin (*information retrieval, IR*), kho dữ liệu có thể lưu và truy xuất dữ liệu dạng văn bản. Ví dụ, ta có thể lưu một tập văn bản trong một máy tìm kiếm có chỉ mục toàn văn (*full-text indexing*), rồi tìm trong đó những văn bản khớp với một truy vấn dạng văn bản cho trước. Áp dụng các kỹ thuật IR cho LLM dẫn tới một khung tổng quát gọi là **sinh văn bản tăng cường truy xuất** (*retrieval-augmented generation, RAG*). Khung RAG hoạt động như sau. Ta dùng ngữ cảnh $\mathbf{x}$ làm truy vấn và tìm k đoạn văn bản (*document chunks*) liên quan nhất $\{\mathbf{c}_1, \cdots, \mathbf{c}_k\}$ từ kho dữ liệu bằng các kỹ thuật IR hiệu quả[12]. Các kết quả tìm kiếm này được kết hợp với ngữ cảnh gốc thông qua một khuôn mẫu prompt $g(\cdot)$[13], tạo thành một đầu vào được tăng cường cho LLM

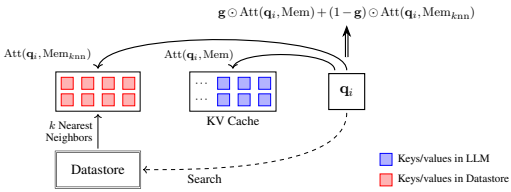
$$\mathbf{x}' = g(\mathbf{c}_1, \cdots, \mathbf{c}_k, \mathbf{x}). \tag{8.69}$$

Sau đó, ta dùng $\mathbf{x}'$ làm ngữ cảnh và dự đoán phần văn bản tiếp theo bằng mô hình $\Pr(\mathbf{y}|\mathbf{x}')$. Một ưu điểm của RAG là ta không cần sửa đổi kiến trúc của LLM, mà chỉ tăng cường đầu vào cho LLM thông qua một hệ thống IR bổ sung. Hình 8.7 so sánh việc sử dụng các loại bộ nhớ ngoài khác nhau trong LLM.

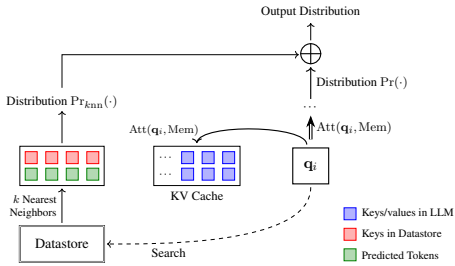
[12] Trong các ứng dụng thực tế, truy vấn thường được tạo bởi một hệ thống sinh truy vấn, hệ thống này có thể mở rộng truy vấn bằng các biến thể của token và ý định truy vấn.

[13] Ví dụ, khuôn mẫu có thể là:

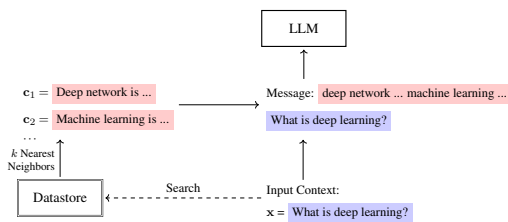
message (thông điệp) = {*$\mathbf{c}_1$*} $\cdots$ {*$\mathbf{c}_k$*}
input (đầu vào): {*$\mathbf{x}$*}
output (đầu ra): ___



(a) k-NN Search Augmented Attention



(b) k-NN Language Modeling



(c) Retrieval-augmented Generation

Hình 8.7: Minh họa các bộ nhớ ngoài (hay kho dữ liệu) cho mô hình hóa ngôn ngữ. (Các nhãn con: (a) chú ý được tăng cường bằng tìm kiếm k-NN; (b) mô hình hóa ngôn ngữ k-NN; (c) sinh văn bản tăng cường truy xuất. Các nhãn khác: "Datastore" = kho dữ liệu, "Search" = tìm kiếm, "k Nearest Neighbors" = k láng giềng gần nhất, "Keys/values in LLM / in Datastore" = khóa/giá trị trong LLM / trong kho dữ liệu, "Predicted Tokens" = các token dự đoán, "Output Distribution" = phân phối đầu ra. Trong (c), $\mathbf{c}_1$ = "Mạng sâu là …", $\mathbf{c}_2$ = "Học máy là …", ngữ cảnh đầu vào $\mathbf{x}$ = "Học sâu là gì?".)

3. Dung lượng bộ nhớ

Một mô hình bộ nhớ trong LLM, dù ở dạng một bộ nhớ đệm khóa–giá trị đơn giản hay một kho dữ liệu ngoài, nhìn chung đều có thể được xem là một bộ mã hóa thông tin ngữ cảnh. Lý tưởng nhất, trước khi nói rằng một mô hình bộ nhớ đại diện được cho toàn bộ ngữ cảnh trong việc dự đoán token, ta phải bảo đảm rằng mô hình có thể tái tạo chính xác bất kỳ phần nào của ngữ cảnh đó. Bộ nhớ đệm KV chuẩn là một mô hình như vậy, vì nó giữ lại toàn bộ lịch sử của chuỗi. Trong trường hợp này, mô hình được gọi là có dung lượng (*capacity*) đủ (hay không giới hạn) để ghi nhớ ngữ cảnh. Tuy nhiên, trong nhiều ứng dụng thực tế, việc ghi nhớ trọn vẹn là không cần thiết. Thay vào đó, mục tiêu chính là giúp LLM truy cập được những thông tin ngữ cảnh nổi bật. Do đó, các mô hình bộ nhớ hiệu quả và được nén đã được phát triển, như đã mô tả trong mục này. Lưu ý rằng chuỗi càng dài thì một mô hình bộ nhớ dung lượng thấp càng khó nắm bắt những thông tin ngữ cảnh then chốt. Vì vậy, một thực hành phổ biến là đơn giản tăng dung lượng bộ nhớ khi xử lý ngữ cảnh dài.

Dù các mô hình bộ nhớ dung lượng cao nhìn chung là đáng mong muốn, chúng lại tốn kém về tính toán khi huấn luyện và triển khai. Một tình huống khó khăn xảy ra khi các token đến thành một luồng (*stream*) liên tục, khiến ngữ cảnh tăng lên vô hạn. Phát triển LLM cho các tác vụ dạng luồng như vậy là khó, vì nó đòi hỏi huấn luyện Transformer trên những chuỗi cực dài. Một giải pháp khả dĩ là dùng các cách tiếp cận phi tham số (*non-parametric*), chẳng hạn các phương pháp dựa trên truy xuất. Ví dụ, như đã bàn ở trên, ta có thể dùng một cơ sở dữ liệu vectơ để lưu mọi cặp khóa–giá trị được sinh ra, nhờ đó biểu diễn hiệu quả ngữ cảnh thông qua một mô hình bộ nhớ ngoài. Mặc dù cách này né được giới hạn độ dài ngữ cảnh vốn có của Transformer, việc xây dựng và duy trì các mô hình bộ nhớ ngoài lại gây ra chi phí tính toán đáng kể. Hơn nữa, các mô hình này thường được áp dụng cho những bài toán mà ngữ cảnh được cung cấp trước và giữ nguyên trong khi suy luận, do đó không phù hợp với việc mô hình hóa ngữ cảnh động, dạng luồng.

Trong những trường hợp kích thước ngữ cảnh tăng liên tục, áp dụng các mô hình bộ nhớ kích thước cố định là một cách làm phổ biến. Ví dụ, trong các mô hình hồi quy, một chuỗi có độ dài tùy ý có thể được tóm tắt thành một tập các trạng thái ẩn, dẫn tới chi phí tính toán cố định cho mỗi bước. Mặc dù trong những ứng dụng ban đầu của học sâu vào NLP, các

mô hình hồi quy được thấy là không giỏi xử lý các phụ thuộc xa (*long-distance dependencies*) trong mô hình hóa chuỗi, những tiến bộ gần đây đã cho thấy các biến thể của chúng nay đã hiệu quả trong việc mô hình hóa các chuỗi cực dài [Bulatov và cộng sự, 2022; Hutchins và cộng sự, 2022; Munkhdalai và cộng sự, 2024; Ma và cộng sự, 2024].

Không có một định nghĩa chung nào về dung lượng bộ nhớ trong LLM. Một cách đơn giản có thể là xem xét lượng bộ nhớ lưu trữ được dùng để giữ thông tin ngữ cảnh. Ví dụ, dung lượng bộ nhớ có thể được định nghĩa bằng kích thước của bộ nhớ đệm KV trong Transformer hoặc của cơ sở dữ liệu vectơ dùng trong các phương pháp dựa trên truy xuất. Một khái niệm liên quan là độ phức tạp mô hình (*model complexity*). Trong học máy, có nhiều cách định nghĩa độ phức tạp mô hình. Một trong những cách đơn giản nhất là đếm số tham số. Tuy nhiên, cần nhấn mạnh rằng các mô hình bộ nhớ bàn ở đây chủ yếu dùng để lưu trữ thông tin, chứ không phải để thêm các tham số huấn luyện được. Do đó, một mô hình có dung lượng bộ nhớ lớn không nhất thiết phức tạp hơn. Dù vậy, trên thực tế, việc xác định dung lượng của một mô hình bộ nhớ không hề đơn giản. Nói chung, ta cần cân nhắc sự đánh đổi giữa việc tối đa hóa hiệu năng và kiểm soát lượng bộ nhớ chiếm dụng.

Chú giải của người dịch: Bộ nhớ trong và bộ nhớ ngoài – tóm tắt

Bộ nhớ trong giống trí nhớ ngắn hạn trong đầu: nhỏ gọn, nằm ngay trong mô hình (bộ nhớ đệm KV và các biến thể nén của nó). *Bộ nhớ ngoài* giống một thư viện hay cuốn sổ tay: có thể rất lớn, nằm ngoài mô hình, và mô hình "tra cứu" khi cần (k-NN, RAG). Bộ nhớ ngoài không làm tăng số tham số của mô hình, nhưng việc xây dựng chỉ mục và tra cứu lại tốn chi phí riêng.

8.3.4 Chia sẻ giữa các đầu và các tầng

Trong Transformer, bộ nhớ đệm KV là một cấu trúc dữ liệu có thể được điều chỉnh linh hoạt theo nhiều chiều, như số đầu, số tầng và độ dài chuỗi. Ví dụ, xét một LLM có L tầng. Mỗi tầng có τ đầu chú ý, và mỗi đầu tạo ra một đầu ra d_h chiều. Trong khi suy luận, ta lưu khóa và giá trị cho tối đa m token. Độ phức tạp không gian (*space complexity*) của cơ chế lưu đệm này là $O(L \cdot \tau \cdot d_h \cdot m)$. Như đã thấy ở trên, độ phức tạp này có thể được

giảm bằng cách lưu đệm khóa và giá trị cho ít token hơn. Ví dụ, trong chú ý cửa sổ trượt (*sliding window attention*), một cửa sổ có kích thước cố định được dùng để lưu đệm khóa và giá trị trong ngữ cảnh cục bộ. Mô hình cửa sổ trượt có độ phức tạp không gian $O(L \cdot \tau \cdot d_h \cdot m_w)$, với m_w là kích thước cửa sổ.

Chú giải của người dịch: Ký hiệu $O(\cdot)$ và một ví dụ số

Ký hiệu $O(L \cdot \tau \cdot d_h \cdot m)$ (đọc là "bậc của …") cho biết lượng bộ nhớ tăng *tỉ lệ* với tích các đại lượng đó, bỏ qua hằng số. Ví dụ, một mô hình có $L = 32$ tầng, $\tau = 32$ đầu, $d_h = 128$, ngữ cảnh $m = 4096$ token cần lưu $32 \cdot 32 \cdot 128 \cdot 4096 \approx 5.4 \times 10^8$ số cho khóa, và chừng đó cho giá trị; với mỗi số chiếm 2 byte, tổng cộng khoảng 2 GB cho *một* chuỗi. Giảm bất kỳ thừa số nào (số token, số đầu, số tầng) đều giảm bộ nhớ theo cùng tỉ lệ.

Ngoài việc giảm m, ta cũng có thể giảm kích thước bộ nhớ đệm KV theo các chiều khác. Một cách làm được dùng rộng rãi là cho phép chia sẻ giữa các đầu trong tự chú ý đa đầu. Nhắc lại từ Mục 8.1.1 rằng tự chú ý đa đầu dùng nhiều bộ truy vấn, khóa và giá trị (mỗi bộ gọi là một *đầu*), mỗi bộ thực hiện cơ chế chú ý QKV như thường lệ. Điều này có thể được biểu diễn là

$$\text{Output} \;=\; \text{Merge}(\text{head}_1, \cdots, \text{head}_\tau)\mathbf{W}^{\text{head}}, \tag{8.70}$$

trong đó $\text{head}_j \in \mathbb{R}^{d_h}$ là đầu ra của đầu thứ j, được tính bằng hàm chú ý QKV chuẩn

$$\text{head}_j = \text{Att}_{\text{qkv}}(\mathbf{q}_i^{[j]}, \mathbf{K}_{\leq i}^{[j]}, \mathbf{V}_{\leq i}^{[j]}). \tag{8.71}$$

Ở đây $\mathbf{q}_i^{[j]}$ là vectơ truy vấn tại vị trí i, còn $\mathbf{K}_{\leq i}^{[j]}$ và $\mathbf{V}_{\leq i}^{[j]}$ là các ma trận khóa và giá trị tương ứng tính tới vị trí i, với mỗi vectơ được chiếu vào không gian con đặc trưng (*feature subspace*) thứ j. Vì vậy mô hình chú ý đa đầu này có thể được hiểu là thực hiện phép chú ý song song trên một nhóm các không gian con đặc trưng (xem Hình 8.8 (b)). Bộ nhớ đệm KV cần giữ khóa và giá trị cho tất cả các đầu này, tức là $\{(\mathbf{K}_{\leq i}^{[1]}, \mathbf{V}_{\leq i}^{[1]}), \cdots, (\mathbf{K}_{\leq i}^{[\tau]}, \mathbf{V}_{\leq i}^{[\tau]})\}$.

Một cải tiến của mô hình chú ý đa đầu, gọi là **chú ý đa truy vấn** (*multi-query attention, MQA*), là chia sẻ khóa và giá trị giữa các đầu, trong khi vẫn giữ truy vấn riêng cho từng đầu [Shazeer, 2019]. Trong MQA, chỉ có một bộ khóa và giá trị duy nhất $(\mathbf{K}_{\leq i}, \mathbf{V}_{\leq i})$. Ngoài ra, có τ truy vấn

$\{\mathbf{q}_i^{[1]}, \cdots, \mathbf{q}_i^{[\tau]}\}$, mỗi truy vấn ứng với một đầu khác nhau. Với mỗi đầu, ta có

$$\text{head}_j = \text{Att}_{\text{qkv}}(\mathbf{q}_i^{[j]}, \mathbf{K}_{\leq i}, \mathbf{V}_{\leq i}). \tag{8.72}$$

Hình 8.8 (c) minh họa mô hình này. Nhờ chia sẻ khóa và giá trị, kích thước bộ nhớ đệm KV sẽ là $O(L \cdot d_h \cdot m)$.

Chú ý truy vấn theo nhóm (*grouped query attention, GQA*) là một mở rộng tự nhiên của chú ý đa đầu và MQA [Ainslie và cộng sự, 2023]. Trong GQA, các đầu được chia thành n_g nhóm, mỗi nhóm ứng với một bộ khóa và giá trị dùng chung. Do đó ta có n_g bộ khóa và giá trị $\{(\mathbf{K}_{\leq i}^{[1]}, \mathbf{V}_{\leq i}^{[1]}), \cdots, (\mathbf{K}_{\leq i}^{[n_g]}, \mathbf{V}_{\leq i}^{[n_g]})\}$. Xem Hình 8.8 (d) để có minh họa. Gọi $g(j)$ là mã số nhóm của đầu thứ j. Mô hình GQA có thể được biểu diễn là

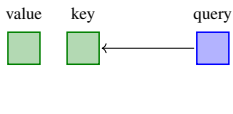
$$\text{head}_j = \text{Att}_{\text{qkv}}(\mathbf{q}_i^{[j]}, \mathbf{K}_{\leq i}^{[g(j)]}, \mathbf{V}_{\leq i}^{[g(j)]}). \tag{8.73}$$

Kích thước bộ nhớ đệm KV của GQA là $O(L \cdot n_g \cdot d_h \cdot m)$. Một lợi ích của GQA là ta có thể đánh đổi giữa hiệu quả tính toán và khả năng biểu đạt (*expressiveness*) của mô hình bằng cách điều chỉnh n_g. Khi $n_g = \tau$, mô hình trở thành mô hình chú ý đa đầu chuẩn. Ngược lại, khi $n_g = 1$, nó trở thành mô hình MQA.

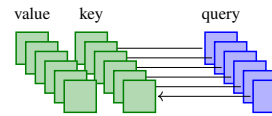
Chú giải của người dịch: So sánh MHA, MQA, GQA bằng số

Giả sử mỗi tầng có $\tau = 8$ đầu. Chú ý đa đầu chuẩn lưu 8 bộ khóa–giá trị. MQA chỉ lưu 1 bộ (giảm 8 lần bộ nhớ đệm), nhưng 8 đầu buộc phải "nhìn" cùng một khóa–giá trị nên mô hình kém linh hoạt hơn. GQA với $n_g = 2$: đầu 1–4 dùng chung bộ 1, đầu 5–8 dùng chung bộ 2 (tức $g(1) = \cdots = g(4) = 1$, $g(5) = \cdots = g(8) = 2$), lưu 2 bộ – giảm 4 lần mà vẫn giữ được một phần sự đa dạng. Truy vấn thì vẫn riêng cho từng đầu vì truy vấn không cần lưu đệm (chỉ dùng cho token hiện tại).

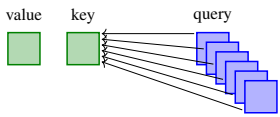
Việc chia sẻ cũng có thể được thực hiện giữa các tầng. Phương pháp như vậy thuộc họ các phương pháp chia sẻ trọng số (*shared weight*) và chia sẻ giá trị kích hoạt (*shared activation*), vốn đã được dùng rộng rãi trong Transformer [Dehghani và cộng sự, 2018; Lan và cộng sự, 2020]. Ví dụ, ta có thể chia sẻ các giá trị kích hoạt KV hoặc các trọng số chú ý giữa các tầng để giảm cả lượng tính toán lẫn bộ nhớ chiếm dụng [Xiao và cộng sự, 2019; Brandon và cộng sự, 2024]. Hình 8.8 (e) minh họa phương pháp này, trong đó một truy vấn ở một tầng truy cập trực tiếp bộ nhớ đệm



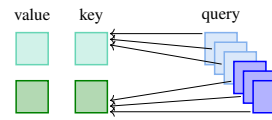
(a) Single-head Attention



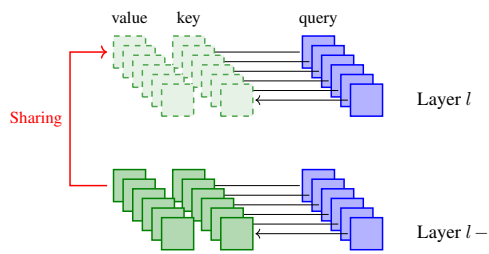
(b) Multi-head Attention



(c) Multi-query Attention



(d) Grouped Query Attention



(e) Cross-layer Multi-head Attention

Hình 8.8: Minh họa phép chú ý QKV dựa trên các cơ chế đa đầu và chia sẻ khác nhau. (a) = chú ý một đầu, và (b–e) = chú ý với nhiều đầu. (Các nhãn con: (a) chú ý một đầu; (b) chú ý đa đầu; (c) chú ý đa truy vấn; (d) chú ý truy vấn theo nhóm; (e) chú ý đa đầu xuyên tầng. "value/key/query" = giá trị/khóa/truy vấn; "Sharing" = chia sẻ; "Layer l" = tầng l.)

KV của một tầng thấp hơn.

8.3.5 Ngoại suy và nội suy vị trí

Vì các tầng Transformer bất biến theo thứ tự (*order-invariant*) đối với đầu vào, ta cần một cách nào đó để mã hóa thông tin vị trí vào các token đầu vào. Để làm điều này, cách phổ biến là cộng các vectơ nhúng vị trí vào các vectơ nhúng token, rồi đưa các vectơ nhúng kết hợp này vào chồng các tầng Transformer. Khi đó, vectơ nhúng tại vị trí i có thể được biểu diễn là

$$\mathbf{e}_i \;=\; \mathbf{x}_i + \mathrm{PE}(i), \tag{8.74}$$

trong đó $\mathbf{x}_i \in \mathbb{R}^d$ là vectơ nhúng token, và $\mathrm{PE}(i) \in \mathbb{R}^d$ là vectơ nhúng vị trí tại vị trí i. Vì vectơ nhúng token $\mathbf{x}_i$ không phụ thuộc vị trí, vectơ nhúng vị trí $\mathrm{PE}(i)$ mã hóa ngữ cảnh tuyệt đối hoặc tương đối của token. Một cách cài đặt trực tiếp là coi $\mathrm{PE}(i)$ là một tập các biến học được, được huấn luyện cùng với các tham số còn lại của mô hình. Điều này cho phép mô hình học một biểu diễn vectơ riêng cho mỗi vị trí rời rạc, nhờ đó phân biệt các token dựa trên chỗ chúng xuất hiện trong chuỗi.

Dùng các vectơ học được để biểu diễn vị trí tuyệt đối hoạt động khá tốt khi độ dài chuỗi gặp phải lúc suy luận gần khớp với độ dài đã thấy khi huấn luyện. Tuy nhiên, trong tiền huấn luyện LLM thực tế, ta phải áp đặt các giới hạn độ dài tối đa nghiêm ngặt (chẳng hạn m_l) lên các chuỗi huấn luyện để giữ cho việc tính toán khả thi, nhưng ta lại thường muốn áp dụng mô hình đã huấn luyện cho những chuỗi dài hơn đáng kể khi triển khai ở các tác vụ hạ nguồn. Trong những tình huống như vậy, dùng các vectơ nhúng vị trí học được có nhược điểm rõ ràng, vì không có vectơ nhúng nào đã được huấn luyện cho những vị trí không xuất hiện trong giai đoạn huấn luyện.

Một cách tiếp cận khác để mô hình hóa thông tin vị trí là phát triển các vectơ nhúng vị trí có khả năng tổng quát hóa (*generalize*): một khi đã huấn luyện, mô hình nhúng có thể được dùng để xử lý các chuỗi dài hơn. Giả sử ta huấn luyện một mô hình nhúng vị trí trên các chuỗi có độ dài tối đa m_l, và ta muốn áp dụng mô hình đã huấn luyện cho một chuỗi có độ dài m ($m \gg m_l$). Nếu mô hình chỉ được huấn luyện trên một khoảng vị trí giới hạn thì mô hình này sẽ đơn giản là thất bại khi xử lý dữ liệu mới nằm ngoài khoảng đó. Xem Hình 8.9 (a) để thấy một minh họa, trong đó

mô hình nhúng đã học không thể mô hình hóa các điểm dữ liệu nằm ngoài miền huấn luyện nếu nó thiếu khả năng ngoại suy.

Có một số cách tiếp cận giúp các mô hình nhúng vị trí tổng quát hóa. Chúng có thể được chia thành hai nhóm.

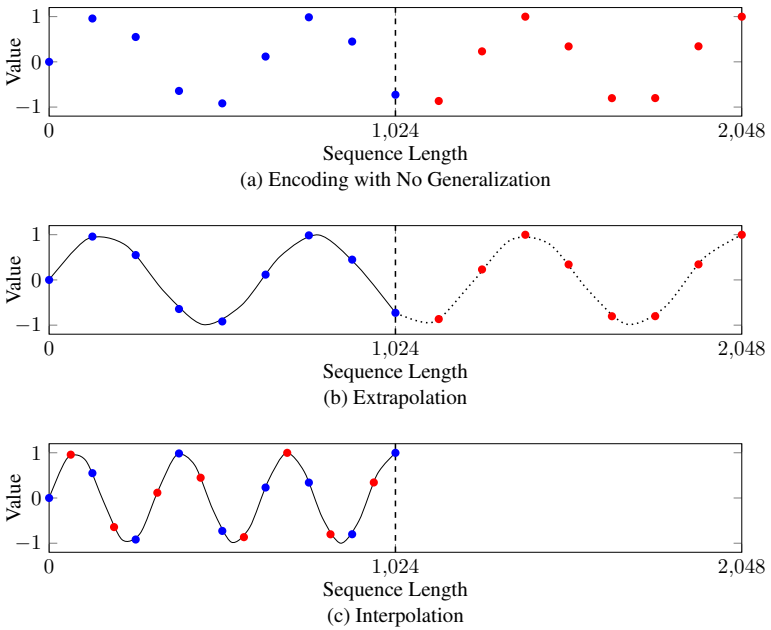
- **Ngoại suy** (*Extrapolation*). Mô hình học được trên các điểm dữ liệu đã quan sát (tức là các vị trí) có thể được dùng trực tiếp để gán giá trị có ý nghĩa cho các điểm dữ liệu nằm ngoài khoảng ban đầu. Chẳng hạn, nếu một mô hình học được một quan hệ hàm liên tục cho các vị trí $\{1, \cdots, 10\}$ thì về lý thuyết nó có thể tính chính hàm đó tại vị trí 15, mà vẫn giữ đúng khoảng cách tương đối và các tính chất hình học so với các token phía trước. Hình 8.9 (b) minh họa khái niệm này, trong đó một hàm được khớp trên miền huấn luyện được kéo dài một cách trơn tru để ước lượng giá trị cho các tọa độ ngoài phân phối (*out-of-distribution*).

- **Nội suy** (*Interpolation*). Cách tiếp cận này ánh xạ một khoảng điểm dữ liệu lớn hơn vào khoảng quan sát ban đầu. Ví dụ, giả sử ta có một mô hình được thiết kế cho các số trong khoảng $[1, 10]$. Khi gặp một khoảng mới $[1, 20]$, ta có thể thu nhỏ nó bằng cách chia mọi số cho 2, nhờ đó đưa tất cả các số vào $[1, 10]$. Phép co giãn này cho phép ta dùng mô hình đã huấn luyện trên khoảng $[1, 10]$ để mô tả các điểm dữ liệu trong khoảng mở rộng $[1, 20]$. Xem Hình 8.9 (c) để có minh họa cho cách tiếp cận này.

Chú giải của người dịch: Ngoại suy và nội suy – liên hệ THPT

Hãy hình dung bạn biết giá trị của hàm số $y = \sin x$ tại các điểm $x = 1, 2, \ldots, 10$. *Ngoại suy* là dùng chính công thức đó để tính y tại $x = 15$ – nằm ngoài vùng đã biết. *Nội suy* thì làm ngược lại: khi cần xử lý x từ 1 đến 20, ta "nén" trục bằng cách thay x bởi $x/2$, nên mọi điểm mới rơi vào vùng $[0{,}5;\ 10]$ quen thuộc (nằm *giữa* các điểm đã biết). Cái giá phải trả của nội suy là các vị trí kề nhau trở nên "sát" nhau hơn, nên mô hình phải phân biệt những khác biệt nhỏ hơn.

Thực ra, các vectơ nhúng vị trí trong nhiều hệ thống đã đạt được một mức tổng quát hóa nhất định. Ví dụ, mã hóa hình sin (*sinusoidal encoding*), phương pháp nhúng vị trí phổ biến nhất, dùng các hàm sin và cos vốn có thể tự nhiên mở rộng cho chuỗi có độ dài bất kỳ. Mặc dù cách



Hình 8.9: Minh họa các hành vi khác nhau của vectơ nhúng vị trí trên một dải độ dài chuỗi liên tục. Các điểm màu xanh lam là những vị trí được quan sát tường minh trong khi huấn luyện, còn các điểm màu đỏ là những vị trí chưa thấy gặp phải lúc suy luận. Trong hình con (a), mô hình nhúng ghi nhớ miền huấn luyện và không thể tổng quát hóa. Trong các hình con (b) và (c), mô hình tổng quát hóa thành công tới các độ dài chưa thấy, lần lượt bằng ngoại suy và nội suy. (Nhãn: "Value" = giá trị, "Sequence Length" = độ dài chuỗi; (a) mã hóa không tổng quát hóa; (b) ngoại suy; (c) nội suy.)

tiếp cận này có vẻ trực tiếp và đơn giản, nó không hoạt động tốt khi ta kéo dài đáng kể các chuỗi cần xử lý. Trong tiểu mục này, ta sẽ bàn về một số phương pháp thay thế dựa trên ngoại suy hoặc nội suy.

1. Chú ý với các độ lệch học được

Một vấn đề với Phương trình (8.74) là mô hình nhúng xử lý mỗi token một cách độc lập và do đó bỏ qua khoảng cách giữa các token khác nhau. Một cải tiến phổ biến cho mô hình này, gọi là nhúng vị trí tương đối (*relative positional embedding*), là xét quan hệ theo cặp giữa các token [Shaw và cộng sự, 2018]. Ý tưởng chung là tính độ lệch giữa hai vị trí bất kỳ và đưa nó vào mô hình tự chú ý. Một trong những dạng đơn giản nhất của tự chú ý với nhúng vị trí tương đối được cho bởi

$$\text{Att}_{\text{qkv}}(\mathbf{q}_i, \mathbf{K}_{\leq i}, \mathbf{V}_{\leq i}) = \sum_{j=0}^{i} \alpha(i,j)\mathbf{v}_j \tag{8.75}$$

$$\alpha(i,j) = \text{Softmax}\left(\frac{\mathbf{q}_i \mathbf{k}_j^\top + \text{PE}(i,j)}{\sqrt{d}} + \text{Mask}(i,j)\right). \tag{8.76}$$

Điểm khác biệt duy nhất giữa mô hình này và mô hình tự chú ý gốc là trong mô hình mới, một số hạng độ lệch $\text{PE}(i,j)$ được cộng vào tích truy vấn–khóa. Về mặt trực giác, $\text{PE}(i,j)$ có thể được hiểu là một mức phạt khoảng cách (*distance penalty*) cho cặp vị trí i và j. Khi khoảng cách giữa i và j tăng, $\text{PE}(i,j)$ giảm.

$\text{PE}(i,j)$ có thể được định nghĩa theo nhiều cách khác nhau. Ở đây, ta xét phiên bản nhúng vị trí tương đối của T5, gọi là độ lệch T5 (*T5 bias*) [Raffel và cộng sự, 2020]. Với mỗi cặp truy vấn $\mathbf{q}_i$ và khóa $\mathbf{k}_j$, độ lệch vị trí giữa chúng được định nghĩa là[14]

$$d(i,j) \;=\; i - j. \tag{8.77}$$

Một cách xây dựng ngây thơ cho độ lệch $\text{PE}(i,j)$ là gán một số vô hướng học được riêng cho mỗi độ lệch rời rạc có thể có, tức là $\text{PE}(i,j) = u_{i-j}$, trong đó u_{i-j} là tham số gắn với đúng khoảng cách $i-j$. Tuy nhiên,

[14] Với mô hình hóa ngôn ngữ, một truy vấn chỉ được phép chú ý tới ngữ cảnh bên trái của nó, nên ta có $i - j \geq 0$. Trong trường hợp tổng quát hơn của tự chú ý, khi một token có thể chú ý tới mọi token trong chuỗi, ta có thể có độ lệch âm khi $i < j$.

chỉ đơn giản gán một giá trị riêng cho mỗi độ lệch sẽ giới hạn mô hình vào những độ lệch đã quan sát trong khi huấn luyện. Khi $i - j$ lớn hơn độ lệch lớn nhất đã được huấn luyện, mô hình không thể tổng quát hóa.

Độ lệch T5 đưa ra một phiên bản tổng quát hóa của cách tiếp cận này. Thay vì cấp một tham số riêng cho mọi độ lệch nguyên có thể có, nó gom các độ lệch vị trí này vào các "thùng" (*buckets*) riêng biệt, trong đó mọi độ lệch được ánh xạ vào cùng một thùng sẽ dùng chung một tham số học được. Cụ thể, với n_b thùng, chiến lược phân bổ được định nghĩa như sau:

- Với nửa đầu số thùng (chỉ số từ 0 đến $\lfloor \frac{n_b}{2} \rfloor - 1$), phép ánh xạ là một–một. Mỗi độ lệch được ánh xạ vào một thùng riêng: thùng $0 \leftrightarrow$ độ lệch 0, thùng $1 \leftrightarrow$ độ lệch 1, và cứ thế. Ta có thể biểu diễn phép ánh xạ tuyến tính này là $b(i - j) = i - j$.
- Với nửa còn lại của số thùng (chỉ số từ $\lfloor \frac{n_b}{2} \rfloor$ đến $n_b - 1$), kích thước thùng tăng theo hàm logarit. Kết quả là các độ lệch lớn hơn được gom vào những ngăn ngày càng rộng. Với một độ lệch $i - j \geq \lfloor \frac{n_b}{2} \rfloor$, chỉ số thùng được tính là:

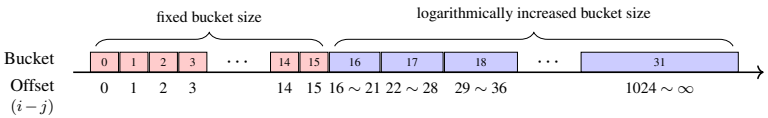
$$b(i-j) \;=\; \lfloor \frac{n_b}{2} \rfloor + \left\lfloor \frac{\log(i-j) - \log(\lfloor \frac{n_b}{2} \rfloor)}{\log(\text{dist}_{\max}) - \log(\lfloor \frac{n_b}{2} \rfloor)} \cdot \left(n_b - \lfloor \frac{n_b}{2} \rfloor - 1 \right) \right\rfloor, \tag{8.78}$$

  trong đó $\text{dist}_{\max}$ là một siêu tham số cho biết ngưỡng độ lệch tối đa dự kiến.
- Khi $i - j \geq \text{dist}_{\max}$, độ lệch bị cắt và được đặt vào thùng cuối cùng $n_b - 1$. Nói cách khác, thùng này đóng vai trò "thu gom tất cả" cho mọi khoảng cách lớn tùy ý vượt quá độ dài chuỗi dự kiến.

Gộp lại, hàm chia thùng được định nghĩa là

$$b(i-j) = \begin{cases} i - j & \text{nếu } 0 \leq i - j < \lfloor \frac{n_b}{2} \rfloor, \\ \min\left(n_b - 1,\ \lfloor \frac{n_b}{2} \rfloor + \left\lfloor \frac{\log(i-j) - \log(\lfloor \frac{n_b}{2} \rfloor)}{\log(\text{dist}_{\max}) - \log(\lfloor \frac{n_b}{2} \rfloor)} \cdot \left(n_b - \lfloor \frac{n_b}{2} \rfloor - 1 \right) \right\rfloor \right) & \text{nếu } i - j \geq \lfloor \frac{n_b}{2} \rfloor. \end{cases} \tag{8.79}$$

Hình 8.10 minh họa các thùng này. Ta thấy ở nửa đầu số thùng, mỗi thùng chỉ gắn với một giá trị của $i - j$, còn ở nửa sau, kích thước thùng tăng lên khi $i - j$ tăng. Thùng cuối cùng được thiết kế để xử lý các chuỗi có độ dài tùy ý.



Hình 8.10: Minh họa việc phân phối các độ lệch truy vấn–khóa vào các thùng trong mô hình T5 ($n_b = 32$ và $\text{dist}_{\max} = 1024$). Các hộp biểu diễn các thùng. Ở nửa đầu số thùng, ta dùng kích thước thùng cố định. Ở nửa sau, ta tăng kích thước thùng theo hàm logarit. Thùng cuối cùng chứa mọi độ lệch truy vấn–khóa chưa được các thùng trước bao phủ. (Nhãn: "Bucket" = thùng, "Offset" = độ lệch, "fixed bucket size" = kích thước thùng cố định, "logarithmically increased bucket size" = kích thước thùng tăng theo logarit.)

Chú giải của người dịch: Tính thử chỉ số thùng

Ký hiệu $\lfloor x \rfloor$ là phần nguyên (làm tròn xuống). Lấy $n_b = 32$, $\text{dist}_{\max} = 1024$ như Hình 8.10, khi đó $\lfloor n_b/2 \rfloor = 16$ và $n_b - 16 - 1 = 15$. Với độ lệch $i - j = 5 < 16$: thùng 5. Với $i - j = 64$: $\frac{\log 64 - \log 16}{\log 1024 - \log 16} = \frac{\log 4}{\log 64} = \frac{2}{6} \approx 0.333$ (logarit cơ số nào cũng được vì là tỉ số), nhân 15 được 5, nên $b = 16 + 5 = 21$. Với $i - j = 5000 > 1024$: công thức cho số lớn hơn 31, nên min cắt về thùng 31. Như vậy các khoảng cách gần được phân biệt rất "tỉ mỉ", còn các khoảng cách xa được gộp thô – giống việc ta nói "hôm qua", "hôm kia" nhưng chỉ nói "mấy năm trước" cho chuyện xa.

Mọi khoảng cách tương đối được ánh xạ vào cùng một thùng $b(i-j)$ dùng chung một số vô hướng độ lệch học được $u_{b(i-j)}$. Thay $\text{PE}(i,j) = u_{b(i-j)}$ trở lại Phương trình (8.76), việc tính trọng số chú ý cuối cùng trở thành[15]

$$\alpha(i,j) = \text{Softmax}\left(\frac{\mathbf{q}_i \mathbf{k}_j^\top + u_{b(i-j)}}{\sqrt{d}} + \text{Mask}(i,j)\right), \tag{8.81}$$

Các tham số $\{u_0, \cdots, u_{n_b-1}\}$ được học cùng với các tham số khác của mô hình. Cần nhấn mạnh rằng mô hình này có thể tổng quát hóa tới các chuỗi dài. Lý do là các $\text{PE}(i,j)$ có độ lệch truy vấn–khóa tương tự

[15] Lưu ý rằng trong mô hình T5 của Raffel và cộng sự [2020], phép co giãn lại tích truy vấn–khóa bị loại bỏ. Trọng số chú ý $\alpha(i,j)$ khi đó được cho bởi

$$\alpha(i,j) = \text{Softmax}(\mathbf{q}_i \mathbf{k}_j^\top + u_{b(i-j)} + \text{Mask}(i,j)). \tag{8.80}$$

nhau dùng chung một tham số, và chiến lược chia sẻ này đặc biệt quan trọng để đạt được khả năng tổng quát hóa tốt, bởi các độ lệch truy vấn–khóa lớn hiếm khi xuất hiện trong huấn luyện. Trên thực tế, ta thường đặt n_b ở mức vừa phải, nhờ đó giúp kiểm soát hiện tượng quá khớp của các mô hình nhúng vị trí.

2. Chú ý với các độ lệch cố định

Các mô hình nhúng vị trí tương đối dựa trên một tập các độ lệch học được, cộng vào tích truy vấn–khóa trong tự chú ý. Một cách tiếp cận khác là gán các giá trị cố định cho những độ lệch này thông qua các quy tắc kinh nghiệm định sẵn, thay vì học chúng từ dữ liệu. Một lợi ích của cách tiếp cận dựa trên quy tắc kinh nghiệm này là nó không phụ thuộc vào huấn luyện, nên có thể áp dụng trực tiếp cho bất kỳ chuỗi nào một khi các độ lệch đã được đặt.

Một ví dụ cho cách tiếp cận như vậy là phương pháp của Press và cộng sự [2022], gọi là **chú ý với độ lệch tuyến tính** (*attention with linear biases*), viết tắt là **ALiBi**. Trong cách tiếp cận ALiBi, số hạng độ lệch được định nghĩa là độ lệch truy vấn–khóa được co giãn và lấy dấu âm

$$\begin{aligned}\text{PE}(i,j) &= -\beta \cdot (i-j) \\ &= \beta \cdot (j-i),\end{aligned} \tag{8.82}$$

trong đó β là hệ số co giãn. Cộng số hạng này vào tích truy vấn–khóa, ta thu được một dạng mới của trọng số chú ý

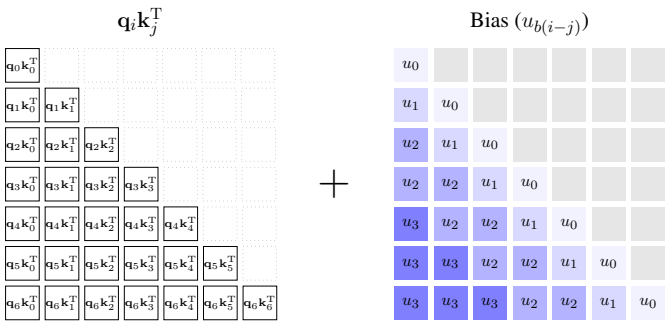
$$\alpha(i,j) \;=\; \text{Softmax}\left(\frac{\mathbf{q}_i\mathbf{k}_j^\top + \beta\cdot(j-i)}{\sqrt{d}} + \text{Mask}(i,j)\right). \tag{8.83}$$

Mô hình này có thể được hiểu là áp đặt một mức phạt tuyến tính lên điểm chú ý $\mathbf{q}_i\mathbf{k}_j^\top$ cho mỗi bước khóa j dịch xa khỏi truy vấn i. Xem Hình 8.11 để so sánh độ lệch T5 và độ lệch ALiBi.

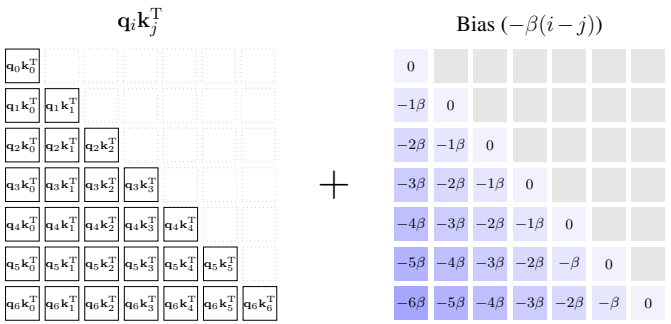
Về nguyên tắc, siêu tham số vô hướng β có thể được điều chỉnh bằng tìm kiếm lưới (*grid search*) trên một tập dữ liệu kiểm định. Tuy nhiên, Press và cộng sự [2022] đã chứng minh rằng việc đặt β một cách có hệ thống cho các đầu chú ý mang lại hiệu năng mạnh trên nhiều tác vụ khác nhau mà không cần điều chỉnh gì. Cụ thể, với một tầng tự chú ý đa đầu gồm n_{head} đầu, các giá trị β giảm theo cấp số nhân. Số vô hướng cho đầu

thứ k được định nghĩa là

$$\beta_k \;=\; 2^{-\frac{8k}{n_{\text{head}}}}. \tag{8.84}$$



(a) The T5 bias ($n_b = 3$ and $\text{dist}_{\max} = 5$)



(b) The ALiBi bias

Hình 8.11: Các tích truy vấn–khóa cùng với độ lệch (trên = độ lệch T5 và dưới = độ lệch ALiBi). Thang màu của các độ lệch đi từ xanh nhạt, biểu thị giá trị tuyệt đối nhỏ, tới xanh đậm, biểu thị giá trị tuyệt đối lớn. (Nhãn con: (a) độ lệch T5 ($n_b = 3$ và $\text{dist}_{\max} = 5$); (b) độ lệch ALiBi. "Bias" = độ lệch.)

Chú giải của người dịch: ALiBi bằng số

Với $n_{\text{head}} = 8$, theo (8.84) ta có $\beta_k = 2^{-k}$: đầu 1 có $\beta = 1/2$, đầu 2 có $1/4$, …, đầu 8 có $1/256$ – đúng là một cấp số nhân công bội $1/2$ mà bạn đã học.

Với đầu 1, khóa cách truy vấn 4 bước bị trừ $4 \cdot 0.5 = 2$ điểm trước khi qua Softmax; vì Softmax dùng hàm mũ, trừ 2 điểm nghĩa là trọng số bị nhân với $e^{-2} \approx 0.135$. Đầu có β lớn vì vậy chủ yếu "nhìn gần", còn đầu có β nhỏ vẫn "nhìn xa" được. Vì mức phạt chỉ phụ thuộc vào $i - j$ theo một công thức cố định, ta tính được nó cho mọi độ dài, kể cả độ dài chưa từng gặp khi huấn luyện.

Cách tiếp cận ALiBi cung cấp một dạng đơn giản của nhúng vị trí tương đối. Có những phương pháp tương tự khác để thiết kế độ lệch truy vấn–khóa dựa trên độ lệch $i - j$. Bảng 8.4 so sánh các độ lệch như vậy. Nhân tiện, đáng lưu ý rằng dạng của vế phải Phương trình (8.82) tương tự các đặc trưng dựa trên khoảng cách được dùng trong các hệ thống dựa trên đặc trưng truyền thống. Ví dụ, trong các hệ thống dịch máy thống kê (*statistical machine translation*), những đặc trưng như vậy được dùng rộng rãi để mô hình hóa bài toán sắp xếp lại trật tự từ (*word reordering*), cho ra những mô hình tổng quát hóa tốt trên nhiều tác vụ dịch khác nhau [Koehn, 2010].

Bảng 8.4: Các độ lệch truy vấn–khóa dùng làm nhúng vị trí tương đối. β, β_1, β_2, $\bar{d}$ và m_{len} là các siêu tham số. Trong mô hình T5, $b(i-j)$ là thùng được gán cho $i - j$. Trong mô hình FIRE, $\psi(\cdot)$ là một hàm chặn đơn điệu tăng, chẳng hạn $\psi(x) = \log(cx + 1)$, còn $f(\cdot)$ là một FFN.

Mô hình	Độ lệch truy vấn–khóa ($\text{PE}(i, j)$)
T5 [Raffel và cộng sự, 2020]	$u_{b(i-j)}$
ALiBi [Press và cộng sự, 2022]	$-\beta \cdot (i - j)$
Kerple [Chi và cộng sự, 2022]	$-\beta_1 (i-j)^{\beta_2}$ (dạng lũy thừa) $-\beta_1 \log(1 + \beta_2 (i-j))$ (dạng logarit)
Sandwich [Chi và cộng sự, 2023]	$\sum_{k=1}^{\bar{d}/2} \cos\left((i-j)/10000^{2k/\bar{d}}\right)$
FIRE [Li và cộng sự, 2024b]	$f\left(\psi(i-j)/\psi(\max(m_{\text{len}}, i))\right)$

3. Nhúng vị trí xoay

Tương tự các vectơ nhúng hình sin, nhúng vị trí xoay (*rotary positional embeddings*) dùng các thang tần số định sẵn [Su và cộng sự, 2024]. Nhắc lại rằng trong mô hình nhúng hình sin, các vị trí tuyệt đối được biểu diễn bằng các vectơ cố định tạo thành từ các hàm sin và cos với tần số khác nhau, rồi được cộng vào các vectơ nhúng token. Ngược lại, nhúng vị trí xoay mô hình hóa ngữ cảnh vị trí bằng cách áp dụng các phép quay có ý nghĩa hình học lên vectơ nhúng token trong miền số phức. Điều này dẫn tới một cách biểu diễn dựa trên các tương tác dạng nhân:

$$\mathbf{e}_i \;=\; \mathbf{x}_i R(i), \tag{8.85}$$

trong đó $R(i) \in \mathbb{R}^{d\times d}$ là ma trận quay (*rotation matrix*) biểu diễn các phép quay thực hiện trên vectơ nhúng token $\mathbf{x}_i \in \mathbb{R}^d$.

Để đơn giản, trước hết ta xét các vectơ nhúng chỉ có hai chiều, rồi sẽ quay lại bàn về cách biểu diễn tổng quát hơn sau. Giả sử ta có một vectơ nhúng token hai chiều được biểu diễn dưới dạng vectơ hàng $\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}$. Ta có thể biểu diễn nó như một vectơ trong mặt phẳng, có gốc tại gốc tọa độ $(0, 0)$ và ngọn tại (x_1, x_2). Một phép quay ngược chiều kim đồng hồ của vectơ này là thao tác di chuyển vectơ quanh gốc tọa độ trong khi giữ nguyên độ dài của nó, như minh họa trong Hình 8.12 (a). Mức độ quay thường được xác định bởi một góc cụ thể, ký hiệu là θ. Phép quay có thể được biểu diễn bằng toán học dưới dạng

$$\begin{aligned}\mathrm{Ro}(\mathbf{x}, \theta) &= \mathbf{x}R_\theta \\ &= \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \\ &= \begin{bmatrix} \cos\theta \cdot x_1 - \sin\theta \cdot x_2 & \sin\theta \cdot x_1 + \cos\theta \cdot x_2 \end{bmatrix}, \end{aligned} \tag{8.86}$$

trong đó $R_\theta = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$ là ma trận quay. Nếu thực hiện hai hay nhiều phép quay trên cùng một vectơ, ta có thể quay vectơ đi xa hơn nữa. Điều này suy ra từ việc hợp của các phép quay liên tiếp bản thân nó cũng là một phép quay. Một cách hình thức hơn, việc quay một vectơ một góc θ, lặp lại t lần, có thể được biểu diễn là

$$\mathrm{Ro}(\mathbf{x}, t\theta) = \mathbf{x}R_{t\theta}$$

$$= \begin{bmatrix} \cos t\theta \cdot x_1 - \sin t\theta \cdot x_2 & \sin t\theta \cdot x_1 + \cos t\theta \cdot x_2 \end{bmatrix}. \tag{8.87}$$

Chú giải của người dịch: Phép quay – nối với lượng giác lớp 10–11

Ở THPT, quay điểm (x_1, x_2) quanh gốc tọa độ một góc θ ngược chiều kim đồng hồ cho điểm $(x_1 \cos\theta - x_2 \sin\theta,\ x_1 \sin\theta + x_2 \cos\theta)$ – đúng là kết quả của (8.86); ma trận R_θ chỉ là cách viết gọn công thức này. Ví dụ, $\mathbf{x} = (1, 0)$, $\theta = 90°$: $\cos\theta = 0, \sin\theta = 1$, ta được $(0, 1)$ – vectơ nằm ngang quay thành thẳng đứng, độ dài vẫn bằng 1. Quay t lần mỗi lần θ thì bằng quay một lần góc $t\theta$ (công thức cộng góc lượng giác). Ý tưởng RoPE: token ở vị trí t được quay góc $t\theta$. Hai token cách nhau 7 vị trí thì luôn lệch nhau góc 7θ, dù ở đầu hay cuối câu (Hình 8.12 (c)).

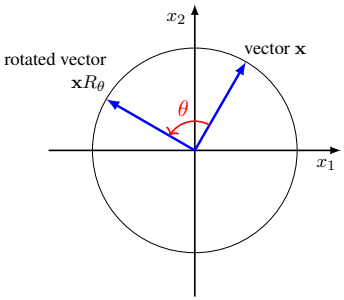
Nếu ta hiểu t là vị trí của một token (được biểu diễn bởi $\mathbf{x}$) trong một chuỗi, ta sẽ thấy phương trình trên định nghĩa một mô hình nhúng vị trí đơn giản. Như trong Hình 8.12 (b), ta bắt đầu dịch chuyển token từ vị trí 0. Mỗi khi tiến thêm một bước, vectơ được quay thêm một góc θ. Khi tới vị trí t, biểu diễn của token kèm ngữ cảnh vị trí được cho bởi $\mathrm{Ro}(\mathbf{x}, t\theta)$. Vì các phép quay không thay đổi độ lớn của vectơ nhúng, "ý nghĩa" ban đầu của token được giữ lại. Thông tin vị trí được đưa vào vectơ nhúng khi nó bị quay.

Một cách phổ biến để hiểu phép quay vectơ là thông qua số phức. Dễ dàng biến đổi mỗi vectơ $\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}$ trong không gian Euclid hai chiều $\mathbb{R}^2$ thành một số phức $\mathbf{x}' = x_1 + \mathrm{i}x_2$ trong không gian phức $\mathbb{C}$ thông qua một ánh xạ tuyến tính song ánh (*bijective linear map*). Khi đó, phép quay $\mathbf{x}$ một góc $t\theta$ tương ứng với phép nhân với $e^{\mathrm{i}t\theta}$. Vì $e^{\mathrm{i}t\theta} = \cos t\theta + \mathrm{i}\sin t\theta$, phép quay có thể được viết lại dưới dạng

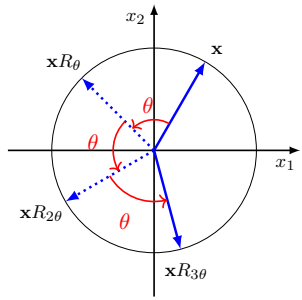
$$\begin{aligned} \mathbf{x}R_{t\theta} &\mapsto \mathbf{x}' e^{\mathrm{i}t\theta} \\ &= (x_1 + \mathrm{i}x_2)(\cos t\theta + \mathrm{i}\sin t\theta) \\ &= \cos t\theta \cdot x_1 - \sin t\theta \cdot x_2 + \mathrm{i}(\sin t\theta \cdot x_1 + \cos t\theta \cdot x_2). \end{aligned} \tag{8.88}$$

Ở đây ta ký hiệu biểu diễn token $\mathbf{x}' e^{\mathrm{i}t\theta}$ là $C(\mathbf{x}, t\theta)$. Tích trong của các biểu diễn của các token tại vị trí t và s có thể được viết là

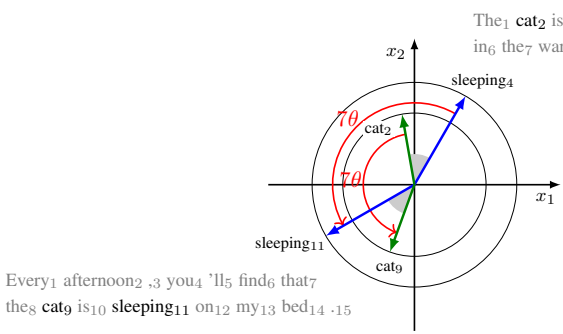
$$\langle C(\mathbf{x}, t\theta), C(\mathbf{y}, s\theta) \rangle \ = \ (\mathbf{x}'\overline{\mathbf{y}'}) e^{\mathrm{i}(t-s)\theta}, \tag{8.89}$$



(a) Single-step Rotation



(b) Multi-step Rotation



(c) Angles between embeddings of two tokens at different positions

Hình 8.12: Minh họa các phép quay vectơ trong mặt phẳng. Các hình con (a) và (b) lần lượt cho thấy phép quay một vectơ trong một bước và trong nhiều bước. Hình con (c) cho thấy các vectơ nhúng của token *cat* (con mèo) và *sleeping* (đang ngủ) trong hai câu khác nhau. Ta viết các câu này với một chỉ số dưới gắn vào mỗi token để cho biết vị trí của nó. Nếu biểu diễn token bằng vectơ, ta có thể thêm thông tin vị trí bằng cách quay các vectơ này. Phép quay này bảo toàn "khoảng cách" giữa các vectơ. Ví dụ, vì khoảng cách giữa *cat* và *sleeping* là như nhau trong cả hai câu, góc giữa các vectơ nhúng của chúng cũng giữ nguyên qua phép quay. (Nhãn con: (a) quay một bước; (b) quay nhiều bước; (c) góc giữa vectơ nhúng của hai token ở các vị trí khác nhau. Hai câu ví dụ: "The cat is sleeping peacefully in the warm sunlight." – *Con mèo đang ngủ yên bình dưới nắng ấm.*; "Every afternoon, you'll find that the cat is sleeping on my bed." – *Chiều nào bạn cũng sẽ thấy con mèo đang ngủ trên giường tôi.*)

trong đó $\overline{\mathbf{y}'}$ là số phức liên hợp (*complex conjugate*) của $\mathbf{y}'$. Kết quả của tích trong này chứa số hạng $t-s$, nên nó có thể mô hình hóa độ lệch giữa hai token.

Chú giải của người dịch: Số phức và công thức Euler

Số phức $x_1+\mathrm{i}x_2$ (với $\mathrm{i}^2=-1$) có thể xem như điểm (x_1, x_2) trên mặt phẳng (chương trình Toán 12 có giới thiệu). Công thức Euler $e^{\mathrm{i}\varphi}=\cos\varphi+\mathrm{i}\sin\varphi$ là một số phức có độ dài 1 và "góc" φ; nhân một số phức với nó nghĩa là quay số phức đó một góc φ. Số phức liên hợp của $a+b\mathrm{i}$ là $a-b\mathrm{i}$ ("quay ngược"). Vì vậy trong (8.89), góc $t\theta$ của $\mathbf{x}$ và góc $-s\theta$ (do liên hợp) của $\mathbf{y}$ cộng lại thành $(t-s)\theta$: chỉ còn *hiệu* hai vị trí – đúng là thông tin vị trí tương đối.

Bây giờ ta quay lại các biểu diễn trong không gian Euclid hai chiều. Tích vô hướng của $\mathrm{Ro}(\mathbf{x}, t\theta)$ và $\mathrm{Ro}(\mathbf{y}, s\theta)$ có thể được viết thành một hàm của $(t-s)\theta$

$$\begin{aligned}\mathrm{Ro}(\mathbf{x}, t\theta)[\mathrm{Ro}(\mathbf{y}, s\theta)]^\top &= \mathbf{x}R_{t\theta}[\mathbf{y}R_{s\theta}]^\top \\ &= \mathbf{x}R_{t\theta}[R_{s\theta}]^\top\mathbf{y}^\top \\ &= \mathbf{x}R_{(t-s)\theta}\mathbf{y}^\top. \end{aligned} \tag{8.90}$$

Với kết quả này, nếu ta coi $\mathrm{Ro}(\mathbf{x}, t\theta)$ và $\mathrm{Ro}(\mathbf{y}, s\theta)$ lần lượt là truy vấn và khóa thì phép tự chú ý sẽ ngầm bao hàm việc mô hình hóa ngữ cảnh vị trí tương đối.

Nhúng vị trí xoay này có thể được mở rộng cho các vectơ nhúng nhiều chiều. Với một vectơ nhúng token d chiều $\mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_d \end{bmatrix}$, ta có thể coi nó là một vectơ phức $\frac{d}{2}$ chiều $\mathbf{x}' = \begin{bmatrix} x'_1 & x'_2 & \cdots & x'_{d/2} \end{bmatrix} = \begin{bmatrix} x_1+\mathrm{i}x_2 & x_3+\mathrm{i}x_4 & \cdots & x_{d-1}+\mathrm{i}x_d \end{bmatrix}$, trong đó mỗi cặp phần tử liên tiếp tạo thành một số phức. Khi đó, nhúng vị trí xoay trong không gian phức được cho bởi

$$C(\mathbf{x}, t\theta) = \sum_{k=1}^{d/2} x'_k e^{\mathrm{i}t\theta_k}\vec{e}_k, \tag{8.91}$$

trong đó $\vec{e}_k$ là vectơ cơ sở chuẩn, chỉ có một giá trị khác 0 tại tọa độ thứ k và bằng 0 ở mọi tọa độ khác [Biderman và cộng sự, 2021].

Mặc dù công thức này có biểu thức phức tạp, dạng tương đương của

nó trong không gian Euclid d chiều lại khá dễ hiểu. Ta có thể viết

$$\text{Ro}(\mathbf{x}, t\theta) = \begin{bmatrix} x_1 & x_2 & \cdots & x_d \end{bmatrix} \begin{bmatrix} R_{t\theta_1} & & & \\ & R_{t\theta_2} & & \\ & & \ddots & \\ & & & R_{t\theta_{d/2}} \end{bmatrix}, \tag{8.92}$$

trong đó $R_{t\theta_k} = \begin{bmatrix} \cos t\theta_k & \sin t\theta_k \\ -\sin t\theta_k & \cos t\theta_k \end{bmatrix}$. $\theta = \begin{bmatrix} \theta_1, \cdots, \theta_{d/2} \end{bmatrix}$ là các tham số điều khiển góc quay ở các chiều khác nhau. Thông thường, θ_k được đặt bằng $10000^{-\frac{2(k-1)}{d}}$, tương tự cách thiết lập trong nhúng hình sin.

Trong cài đặt thực tế, Phương trình (8.92) có thể được viết lại thành một dạng chỉ dựa vào phép nhân theo từng phần tử và phép cộng vectơ:

$$\text{Ro}(\mathbf{x}, t\theta) = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{d-1} \\ x_d \end{bmatrix}^\top \odot \begin{bmatrix} \cos t\theta_1 \\ \cos t\theta_1 \\ \vdots \\ \cos t\theta_{d/2} \\ \cos t\theta_{d/2} \end{bmatrix}^\top + \begin{bmatrix} -x_2 \\ x_1 \\ \vdots \\ -x_d \\ x_{d-1} \end{bmatrix}^\top \odot \begin{bmatrix} \sin t\theta_1 \\ \sin t\theta_1 \\ \vdots \\ \sin t\theta_{d/2} \\ \sin t\theta_{d/2} \end{bmatrix}^\top. \tag{8.93}$$

Cuối cùng, ta viết lại Phương trình (8.85) để có dạng vectơ nhúng tại vị trí i

$$\mathbf{e}_i = \text{Ro}(\mathbf{x}_i, i\theta). \tag{8.94}$$

Chú giải của người dịch: RoPE nhiều chiều: quay từng cặp

Phương trình (8.92)–(8.93) nói rằng: chia vectơ d chiều thành $d/2$ cặp $(x_1, x_2), (x_3, x_4), \ldots$ và quay *mỗi cặp* như một vectơ 2 chiều, nhưng với tốc độ quay khác nhau θ_k. Với $d = 4$: $\theta_1 = 10000^0 = 1$ và $\theta_2 = 10000^{-2/4} = 0.01$. Cặp thứ nhất quay nhanh (1 radian mỗi vị trí, giống kim giây), cặp thứ hai quay rất chậm (0.01 radian mỗi vị trí, giống kim giờ). Phối hợp nhiều "kim" với tốc độ khác nhau giúp phân biệt cả khoảng cách gần lẫn xa, như đồng hồ dùng kim giây, kim phút, kim giờ.

4. Nội suy vị trí

Trong nội suy vị trí (*position interpolation*), mục tiêu của ta là ánh xạ các vị trí trong chuỗi mới sao cho khớp với khoảng đã quan sát khi huấn luyện. Giả sử mô hình được huấn luyện trên các chỉ số vị trí trong $[0, m_l]$. Khi suy luận cần xử lý một độ dài $m > m_l$, ta nén các vị trí trong khoảng $[0, m]$ sao cho biểu diễn của chúng vừa với miền ban đầu $[0, m_l]$.

Để minh họa, xét mô hình nhúng vị trí xoay mô tả ở trên. Vectơ nhúng của mỗi token được cho bởi phép biến đổi $\text{Ro}(\mathbf{x}_i, i\theta)$, trong đó vectơ tham số là $\theta = \left[\theta_1, \cdots, \theta_{d/2}\right]$. Như trong Phương trình (8.93), $\text{Ro}(\mathbf{x}_i, i\theta)$ dựa vào sự kết hợp các hàm tuần hoàn trên các chiều đặc trưng khác nhau

$$\cos i\theta = \begin{bmatrix}\cos i\theta_1 & \cdots & \cos i\theta_{d/2}\end{bmatrix} \tag{8.95}$$

$$\sin i\theta = \begin{bmatrix}\sin i\theta_1 & \cdots & \sin i\theta_{d/2}\end{bmatrix}, \tag{8.96}$$

θ_k có dạng hàm mũ

$$\theta_k \;=\; b^{-\frac{2(k-1)}{d}}, \tag{8.97}$$

trong đó b là cơ số (*base*). Chu kỳ của $\cos i\theta_k$ và $\sin i\theta_k$ là

$$T_k \;=\; 2\pi \cdot b^{\frac{2(k-1)}{d}}. \tag{8.98}$$

Trực giác then chốt đằng sau nội suy vị trí tuyến tính là điều chỉnh các chu kỳ này sao cho các vị trí mở rộng mới có thể được mã hóa trong phạm vi đã tiền huấn luyện $[0, m_l]$. Cách trực tiếp nhất để làm điều này là tăng tuyến tính T_k theo tỉ lệ ngoại suy $\frac{m}{m_l}$, thu được:

$$T'_k \;=\; \frac{m}{m_l} \cdot 2\pi \cdot b^{\frac{2(k-1)}{d}}. \tag{8.99}$$

Làm như vậy, mọi chỉ số vị trí trong $[0, m]$ được nén đều vào $[0, m_l]$. Phép co giãn tuyến tính này có thể được cài đặt bằng cách chia chỉ số vị trí trong mô hình nhúng cho hệ số co giãn [Chen và cộng sự, 2023c]. Mô hình được sửa đổi với nội suy vị trí tuyến tính trở thành

$$\text{Ro}'(\mathbf{x}_i, i\theta) \;=\; \text{Ro}\left(\mathbf{x}_i, \frac{m_l}{m} i\theta\right). \tag{8.100}$$

Chú giải của người dịch: Chu kỳ và nội suy tuyến tính bằng số

Ở lớp 11, hàm $y = \cos(\omega x)$ có chu kỳ $T = 2\pi/\omega$; vì $\theta_k = b^{-2(k-1)/d}$ nên $T_k = 2\pi/\theta_k = 2\pi b^{2(k-1)/d}$, đúng như (8.98). Giả sử mô hình được huấn luyện với $m_l = 2048$ và ta muốn dùng $m = 8192$, tỉ lệ $m/m_l = 4$. Theo (8.100), vị trí $i = 8000$ được "giả vờ" là vị trí $8000/4 = 2000$ – nằm trong vùng quen thuộc $[0, 2048]$. Mọi chu kỳ dài ra 4 lần, nên các vị trí mới nằm *xen giữa* các vị trí cũ (vị trí 1 được coi như 0.25), đúng tinh thần nội suy ở Hình 8.9 (c).

Một phương pháp nội suy vị trí khác là co giãn cơ số b, một kỹ thuật được gọi là **RoPE co giãn có nhận biết NTK** (*NTK-aware scaled RoPE*)[16]. Giả sử cơ số b được co giãn theo hệ số λ. Mục tiêu là buộc chu kỳ của mô hình mới này ở chiều tần số cao nhất (tức là chiều cuối cùng, nơi $k = \frac{d}{2}$) khớp với chu kỳ của mô hình nội suy vị trí tuyến tính. Thay $k = \frac{d}{2}$ vào số mũ ta được $\frac{2(d/2-1)}{d} = \frac{d-2}{d}$. Cho hai chu kỳ bằng nhau ta có:

$$2\pi \cdot (\lambda b)^{\frac{d-2}{d}} = \frac{m}{m_l} \cdot 2\pi \cdot b^{\frac{d-2}{d}}. \tag{8.101}$$

Giải phương trình này, ta có thể tách riêng λ:

$$\lambda^{\frac{d-2}{d}} = \frac{m}{m_l}. \tag{8.102}$$

Khi đó, ta có

$$\lambda = \left(\frac{m}{m_l}\right)^{\frac{d}{d-2}}. \tag{8.103}$$

Cách tiếp cận này cho ra một mô hình nhúng được sửa đổi:

$$\mathrm{Ro}'(\mathbf{x}_i, i\theta) = \mathrm{Ro}(\mathbf{x}_i, i\theta'), \tag{8.104}$$

trong đó

$$\theta' = \left[(\lambda b)^{-\frac{0}{d}}, (\lambda b)^{-\frac{2}{d}}, \cdots, (\lambda b)^{-\frac{d-2}{d}}\right]. \tag{8.105}$$

Lưu ý rằng việc co giãn cơ số mang lại một cách co giãn *không đều* các chu kỳ trên các chiều khác nhau của θ. Phương pháp này đã được thấy

[16] Phương pháp này lần đầu được đề xuất trên một diễn đàn trực tuyến: `https://www.reddit.com/r/LocalLLaMA/comments/14lz7j5/ntkaware_scaled_rope_allows_llama_models_to_have/`

là hữu ích trong việc mở rộng LLM cho các chuỗi dài hơn, và một số cải tiến đã được phát triển [Peng và cộng sự, 2024; Ding và cộng sự, 2024].

Chú giải của người dịch: Vì sao co giãn cơ số là "không đều"?

Lấy $d = 4$, $m/m_l = 4$. Theo (8.103): $\lambda = 4^{4/2} = 16$. Với $k = 1$, số mũ $\frac{0}{d} = 0$ nên $\theta_1' = (\lambda b)^0 = 1 = \theta_1$: chiều quay nhanh nhất *không đổi* (vẫn phân biệt tốt các token kề nhau). Với $k = 2$ (chiều cuối), $\theta_2' = (16b)^{-1/2} = \theta_2/4$: chu kỳ dài ra đúng 4 lần như nội suy tuyến tính. Như vậy, khác với (8.100) vốn kéo dài *mọi* chu kỳ 4 lần, cách này giữ nguyên các "kim giây" và chỉ kéo dài các "kim giờ". (Phép giải (8.102)–(8.103): chia hai vế (8.101) cho $2\pi b^{(d-2)/d}$, rồi nâng hai vế lên lũy thừa $\frac{d}{d-2}$.)

8.3.6 Nhận xét

Trong mục này, chúng ta đã trình bày nhiều phương pháp cho mô hình hóa ngôn ngữ với ngữ cảnh dài. Ta khép lại mục này bằng việc thảo luận một số vấn đề thú vị liên quan tới các phương pháp này.

1. Nhu cầu về ngữ cảnh dài

Một trong những mục tiêu tối thượng của LLM ngữ cảnh dài là giúp các mô hình này mã hóa chính xác ngữ cảnh vô hạn (*infinite context*). Ở đây, thuật ngữ *ngữ cảnh vô hạn* chỉ khả năng của một LLM liên tục xử lý các token đang đến. Điều này thúc đẩy việc phát triển các LLM có thể xử lý ngữ cảnh rất dài hoặc dữ liệu dạng luồng. Như đã bàn ở Mục 8.3.3, việc dùng các mô hình bộ nhớ kích thước cố định để xử lý ngữ cảnh mở rộng liên tục là phổ biến. Nhiều hệ thống như vậy dựa trên các kiến trúc hồi quy hoặc biến thể của chúng, vì bản chất chúng phù hợp để mô hình hóa dữ liệu tuần tự, nơi ảnh hưởng của các đầu vào trong quá khứ kéo dài mãi mãi. Một cách khác để đạt được bộ nhớ vô hạn là phát triển các phương án thay thế cho mô hình tự chú ý, chẳng hạn ta có thể dùng các mô hình chú ý trong không gian liên tục (*continuous-space attention*) để mã hóa ngữ cảnh, giúp loại bỏ sự phụ thuộc vào độ dài ngữ cảnh [Martins và cộng sự, 2022].

Khi nghiên cứu các LLM ngữ cảnh dài, ta tự nhiên sẽ thắc mắc những cơ chế nào có thể giải thích việc sử dụng ngữ cảnh dài trong mô hình hóa ngôn ngữ. Liệu ta có thể nén biểu diễn của ngữ cảnh vô hạn vào một mô

hình tương đối nhỏ không? Có phải mọi token ngữ cảnh đều hữu ích cho việc dự đoán token tiếp theo? Liệu ta có thể biết trước thông tin ngữ cảnh nào là then chốt cho dự đoán? Câu trả lời tổng quát cho tất cả các câu hỏi này không hiển nhiên, nhưng chúng gợi mở những nghiên cứu tiếp theo về các mô hình có thể giải thích (*explainable models*), và một số kết quả thú vị đã được tìm ra. Ví dụ, Deletang và cộng sự [2024] đã tiến hành nhiều thí nghiệm để cho thấy LLM là những bộ nén mạnh mẽ trong ngữ cảnh. Mặc dù việc xem các mô hình dự đoán như các mô hình nén đã được nghiên cứu từ lâu trong học máy, nó cũng mang lại những hiểu biết sâu sắc cho việc ta hiểu các định luật co giãn (scaling law) của LLM. Hơn nữa, Pal và cộng sự [2023] và Wu và cộng sự [2024] đã nghiên cứu xem liệu các đặc trưng nội tại học được tính tới bước tính toán hiện tại, dù không được tối ưu tường minh cho mục đích đó, đã đủ để dự đoán chính xác các token ở nhiều bước trong tương lai hay chưa. Lưu ý rằng nhu cầu về ngữ cảnh dài trong mô hình hóa ngôn ngữ phụ thuộc nhiều vào bài toán mà ta giải quyết. Một vấn đề liên quan là áp dụng LLM ở đâu và đánh giá chúng như thế nào. Ví dụ, trong các tác vụ tóm tắt, ta có thể chỉ cần chắt lọc và tập trung vào vài khía cạnh chính của văn bản, trong khi ở các tác vụ giống truy xuất, ta cần "ghi nhớ" toàn bộ ngữ cảnh để có thể truy cập thông tin liên quan. Ta sẽ bàn về vấn đề đánh giá ở phần sau của tiểu mục này.

Chú giải của người dịch: Dự đoán và nén có liên quan gì?

Nếu một mô hình đoán rất chuẩn chữ tiếp theo, ta chỉ cần ghi lại những chỗ nó đoán sai là đủ để khôi phục cả văn bản – tức là văn bản được "nén" lại. Ví dụ, với câu "Hà Nội là thủ đô của Việt __", một mô hình tốt chắc chắn đoán "Nam", nên chữ này gần như không tốn chỗ lưu. Vì vậy, dự đoán giỏi đồng nghĩa với nén giỏi; đây là lý do các nhà nghiên cứu xem LLM như những bộ nén.

2. Tiền huấn luyện hay thích nghi LLM?

Việc huấn luyện LLM đòi hỏi tài nguyên tính toán rất lớn. Mặc dù huấn luyện LLM trên các chuỗi dài là việc làm trực tiếp, chi phí huấn luyện sẽ trở nên quá đắt đỏ đối với các tập dữ liệu lớn. Thực hành phổ biến là tiền huấn luyện LLM trên các tập dữ liệu tổng quát, rồi thích nghi (*adapt*) chúng bằng một lượng tinh chỉnh vừa phải. Ví dụ, các LLM có nhúng vị trí tương đối hoặc nhúng vị trí xoay có thể được huấn luyện trực tiếp trên dữ liệu quy mô lớn trong giai đoạn tiền huấn luyện. Mặc dù các mô hình thu được có thể thể hiện một số khả năng ngoại suy độ dài (*length extrapolation*) trong giai đoạn suy luận, việc tinh chỉnh chúng trên các chuỗi dài hơn có thể hiệu quả hơn.

Lý tưởng nhất, ta muốn tiền huấn luyện LLM với các kiến trúc Transformer chuẩn rồi thích nghi chúng cho các tác vụ mới. Điều này cho phép ta dùng nhiều LLM có sẵn và thích nghi chúng một cách hiệu quả để xử lý các chuỗi dài. Tuy nhiên, khi áp dụng các kiến trúc mới, dường như không tránh khỏi việc phải huấn luyện các mô hình này từ đầu. Điều này gây khó khăn thực tế cho việc phát triển LLM ngữ cảnh dài, vì ta không thể tận dụng các mô hình tiền huấn luyện đã được phát triển tốt mà phải tự huấn luyện chúng. Mặt khác, tinh chỉnh vẫn là một cách hiệu quả để thích nghi các LLM có một số kiến trúc khác với kiến trúc dùng trong tiền huấn luyện. Một ví dụ là các mô hình được tăng cường bằng bộ nhớ ngoài. Trong các mô hình này, LLM đã tiền huấn luyện được giữ cố định, và trọng tâm là cách LLM tương tác với các mô-đun bộ nhớ ngoài. Chẳng hạn trong RAG, người ta thường tinh chỉnh LLM để cải thiện cách chúng sử dụng các đầu vào được tăng cường truy xuất. Một ví dụ khác là huấn luyện một LLM với chú ý đầy đủ, rồi thay thế nó bằng chú ý thưa trong giai đoạn tinh chỉnh. LLM đã tiền huấn luyện cung cấp giá trị khởi tạo cho các tham số của mô hình mới, và mô hình này sau đó được tinh chỉnh như thường lệ.

3. Đánh giá LLM ngữ cảnh dài

Đánh giá LLM ngữ cảnh dài là quan trọng, nhưng đây là một bài toán tương đối mới trong NLP. Ý tưởng chung là: nếu ta đưa một ngữ cảnh dài vào LLM, ta có thể xem xét đầu ra của LLM để xác định liệu nó có hiểu toàn bộ ngữ cảnh và tận dụng ngữ cảnh đó khi dự đoán các token tiếp theo hay không. Trong nghiên cứu NLP truyền thống, những đánh giá như vậy thường nhằm kiểm tra khả năng của mô hình NLP trong việc xử lý các phụ thuộc xa. Tuy nhiên, độ dài ngữ cảnh dùng trong các LLM gần đây lớn hơn nhiều so với độ dài dùng trong các hệ thống NLP cách đây vài năm. Điều này thúc đẩy các nhà nghiên cứu phát triển các bộ đánh giá chuẩn (benchmark) và độ đo mới cho LLM ngữ cảnh dài.

Một cách tiếp cận là dùng độ đo độ hỗn loạn (*perplexity*). Tuy nhiên, dù đơn giản, phương pháp này có xu hướng phản ánh khả năng của LLM trong việc tận dụng ngữ cảnh cục bộ hơn là ngữ cảnh toàn cục. Vì vậy, người ta muốn phát triển các phương pháp đánh giá dành riêng cho LLM ngữ cảnh dài. Các phương pháp phổ biến bao gồm nhiều tác vụ tổng hợp (*synthetic tasks*), trong đó dữ liệu được sinh nhân tạo hoặc được sửa đổi được dùng để đánh giá những năng lực cụ thể của LLM ngữ cảnh dài. Chẳng hạn, trong các tác vụ mò kim đáy bể (*needle-in-a-haystack*)[17] và truy xuất mật mã (*passkey retrieval*) [Mohtashami và Jaggi, 2024; Chen và cộng sự, 2023c], LLM được yêu cầu xác định và trích ra một mẩu thông tin nhỏ, có liên quan từ một khối lượng lớn văn bản cho trước. Giả định ở đây là một LLM có đủ bộ nhớ sẽ nhớ được các phần trước đó của văn bản khi xử lý thông tin mới. Nhờ vậy LLM có thể nhặt ra từ văn bản những chi tiết liên quan, vốn có thể thưa thớt và ẩn giữa rất nhiều thông tin không liên quan. Ngoài ra, trong các tác vụ sao chép bộ nhớ (*copy memory tasks*) (gọi tắt là tác vụ sao chép), LLM được yêu cầu lặp lại văn bản đầu vào hoặc một đoạn cụ thể nhiều lần. Các tác vụ này ban đầu được đề xuất để kiểm tra mức độ mà các mô hình hồi quy có thể lưu giữ và nhớ lại các token đã thấy trước đó [Hochreiter và Schmidhuber, 1997; Arjovsky và cộng sự, 2016], và đã được dùng để đánh giá các LLM gần đây [Bulatov và cộng sự, 2022; Gu và Dao, 2023].

[17] `https://github.com/gkamradt/LLMTest_NeedleInAHaystack`

Chú giải của người dịch: Bài kiểm tra "mò kim đáy bể"

Người ta chèn một câu "kim" (ví dụ: "Mật khẩu bí mật là 7429.") vào một vị trí ngẫu nhiên giữa hàng chục nghìn token văn bản "rơm" không liên quan, rồi hỏi mô hình: "Mật khẩu bí mật là gì?". Thay đổi độ dài văn bản và vị trí chèn kim, ta vẽ được bản đồ cho biết mô hình "nhớ" tốt tới đâu. Như sách nói ở đoạn sau, làm tốt bài này chưa chắc mô hình đã *hiểu* toàn bộ ngữ cảnh.

Một cách tiếp cận khác để đánh giá LLM ngữ cảnh dài là kiểm tra chúng trên các tác vụ NLP có chuỗi đầu vào rất dài. Ví dụ bao gồm tóm tắt văn bản dài hoặc tóm tắt nhiều văn bản, hỏi đáp trên văn bản dài, hoàn thiện mã nguồn, v.v. Một lợi ích của cách tiếp cận này là nó có thể làm cho việc đánh giá phù hợp với kỳ vọng của người dùng.

Mặc dù nhiều phương pháp đã được phát triển, vẫn chưa có một cách tổng quát để đánh giá LLM ngữ cảnh dài [Liu và cộng sự, 2024c]. Một vấn đề là hầu hết các phương pháp này tập trung vào những khía cạnh cụ thể của LLM, thay vì năng lực nền tảng trong việc mô hình hóa ngữ cảnh rất dài. Ngay cả khi một LLM có thể nhặt ra đúng đoạn văn bản thích hợp từ đầu vào, ta cũng không thể nói rằng nó thực sự hiểu toàn bộ ngữ cảnh. Thay vào đó, nó có thể chỉ nhớ một số phần quan trọng của ngữ cảnh, hoặc thậm chí đơn giản là nhớ lại câu trả lời thông qua mô hình đã học trong tiền huấn luyện. Hơn nữa, dữ liệu dùng trong nhiều tác vụ có quy mô nhỏ và còn tương đối sơ khai, dẫn tới sự chênh lệch giữa kết quả đánh giá và hiệu năng ứng dụng thực tế. Một vấn đề đáng chú ý hơn là kết quả của LLM chịu ảnh hưởng bởi nhiều yếu tố khác và các thiết lập thí nghiệm, chẳng hạn dùng các prompt khác nhau có thể dẫn tới kết quả rất khác nhau. Điều này khiến việc đánh giá càng khó khăn hơn, vì các cải thiện có thể không hoàn toàn đến từ việc mô hình hóa ngữ cảnh dài tốt hơn, và có nguy cơ khẳng định quá mức mức tăng hiệu năng. Dù vậy, vẫn còn nhiều câu hỏi mở trong việc phát triển và đánh giá LLM ngữ cảnh dài. Ví dụ, các mô hình này vẫn gặp những hạn chế như độ dài ngữ cảnh bị giới hạn và độ trễ (*latency*) cao. Nghiên cứu các vấn đề này nhiều khả năng sẽ là một hướng đi giá trị trong tương lai.

8.4 Tóm tắt

Trong chương này, chúng ta đã bàn về khái niệm LLM và các kỹ thuật liên quan. Đây có thể được xem là một phần giới thiệu tổng quát, dù chưa toàn diện, về LLM, đặt nền móng cho những thảo luận sâu hơn về các chủ đề nâng cao trong các chương sau. Hơn nữa, chúng ta đã tìm hiểu hai hướng để mở rộng quy mô LLM. Hướng thứ nhất tập trung vào tiền huấn luyện LLM quy mô lớn, điều then chốt để phát triển các mô hình tiên tiến nhất. Hướng thứ hai tập trung vào các phương pháp thích nghi LLM cho đầu vào dài, bao gồm tối ưu các mô hình chú ý, thiết kế các bộ nhớ đệm KV hiệu quả và được nén hơn, đưa vào các mô hình bộ nhớ, và khám phá các vectơ nhúng vị trí tốt hơn.

Sức mạnh của LLM nằm ở khả năng phá vỡ giới hạn của việc huấn luyện các mô hình NLP cho một số ít tác vụ cụ thể. Thay vào đó, LLM học từ lượng văn bản khổng lồ thông qua một tác vụ đơn giản là dự đoán token – dự đoán token tiếp theo trong chuỗi khi biết ngữ cảnh phía trước nó. Một quan điểm chung là: bằng cách lặp lại tác vụ dự đoán token này rất nhiều lần, LLM có thể thu nhận một số tri thức về thế giới và ngôn ngữ, rồi áp dụng chúng cho các tác vụ mới. Kết quả là LLM có thể được prompt để thực hiện bất kỳ tác vụ nào bằng cách diễn đạt tác vụ đó thành bài toán dự đoán các token tiếp theo khi cho trước prompt. Năng lực đột sinh (*emergent ability*) này của các mô hình ngôn ngữ đến từ nhiều chiều, chẳng hạn mở rộng quy mô huấn luyện, kích thước mô hình và kích thước ngữ cảnh. Không thể phủ nhận rằng các định luật co giãn hiện là nguyên lý nền tảng được áp dụng trong phát triển mô hình ngôn ngữ lớn, mặc dù việc chỉ đơn thuần tăng kích thước mô hình vẫn chưa được chứng minh là đủ để đạt tới trí tuệ nhân tạo tổng quát (*AGI*). Các LLM được mở rộng liên tục này đã được thấy là có năng lực hiểu ngôn ngữ, sinh ngôn ngữ và lập luận (reasoning) đa mục đích. Gần đây hơn, người ta thấy rằng việc tăng lượng tính toán lúc suy luận (tức tính toán lúc kiểm tra (*test-time compute*)) cũng có thể dẫn tới những cải thiện đáng kể trong các tác vụ suy luận phức tạp [OpenAI, 2024].

Với sức mạnh đáng kinh ngạc của mình, LLM đã thu hút sự quan tâm đáng kể, cả về kỹ thuật lẫn ứng dụng. Kết quả là sự bùng nổ quan tâm nghiên cứu về LLM cũng dẫn tới một số lượng khổng lồ các kỹ thuật và mô hình mới. Tuy nhiên, chúng tôi không cố gắng đưa ra một tổng quan

tài liệu toàn diện về mọi khía cạnh của LLM, do lĩnh vực này phát triển rất nhanh. Dù vậy, bạn đọc vẫn có thể tìm hiểu về LLM từ các bài tổng quan chung [Zhao và cộng sự, 2023; Minaee và cộng sự, 2024] hoặc các thảo luận tập trung hơn vào những chủ đề cụ thể [Ruan và cộng sự, 2024].

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.